DOCUMENTATION SECTION

The Hackathon Story

The project story from start to finish — how a single-lane test track became a thousand-lane highway for financial contract evaluation, told through the car factory analogy.

- Car Factory & Highway Analogy
- Development Timeline & Milestones
- Key Decisions & Trade-Offs
- Challenges & Solutions
- Results & Performance Benchmarks

Table of Contents

The Hackathon Story

Competition Submission — ACTUS-I

Personal Note

Understanding CPU and GPU — The Car Factory

Monte Carlo Test Sample

Building the Highway — Development Timeline

Key Decisions — Designing the Highway

Mac Docker — PAM Monte Carlo Scaling Study

Roadblocks — Challenges and How I Solved Them

Computational Complexity Reference

Test Results — What the Highway Delivered

Scenario / CPU-GPU / CalcDate Causality Demo

Benchmark Analysis Report

Code Resources

Installation & Getting Started

Resources & Downloads

Terminology & Glossary

Hackathon

The Hackathon Story

The Hackathon Story

How It Started

In late 2025, I was introduced to the ACTUS standard — Algorithmic Contract Types Unified Standards — by Francis Gross and Willi Brammertz. I had never worked with ACTUS before, and concepts like Monte Carlo simulation, financial contract projections, and deterministic cash flow modelling were entirely new to me. What I did bring was deep experience in the insurance domain, where I had seen the same fundamental challenge play out over and over: the need for both **accuracy** and **speed** when evaluating large portfolios.

When I learned about the **ACTUS Algorithmic Financial Contracts Use Case Competition**, I saw a chance to prove that this challenge could be solved — not just in theory, but with a working system.

I started in December 2025. The competition deadline was March 16, 2026. This is the story of what I built.

> *A note on the Technical Concepts*
>
> *Before going deeper, it is worth acknowledging something important.*
>
> *The implementation behind this project involves concepts such as CPU parallelism, GPU computing, vectorization, and large-scale simulation pipelines. These are common topics in high-performance computing, but they are not concepts everyone works with daily — especially in finance, insurance, or business roles where the focus is usually on models, contracts, and outcomes rather than hardware execution strategies.*
>
> *Because of that, I will not explain the system purely in technical terms.*

> *Instead, I will use an analogy to explain both the reasoning and the architecture behind the implementation. The analogy is intentionally simple, but it maps directly to how the system actually works. If you understand the analogy, you will understand the design decisions that made the system fast.*
>
> *For readers who are interested in the deeper technical details — GPUs, thread blocks, memory layouts, and simulation kernels — those will be explained later as well. But first, the analogy.*

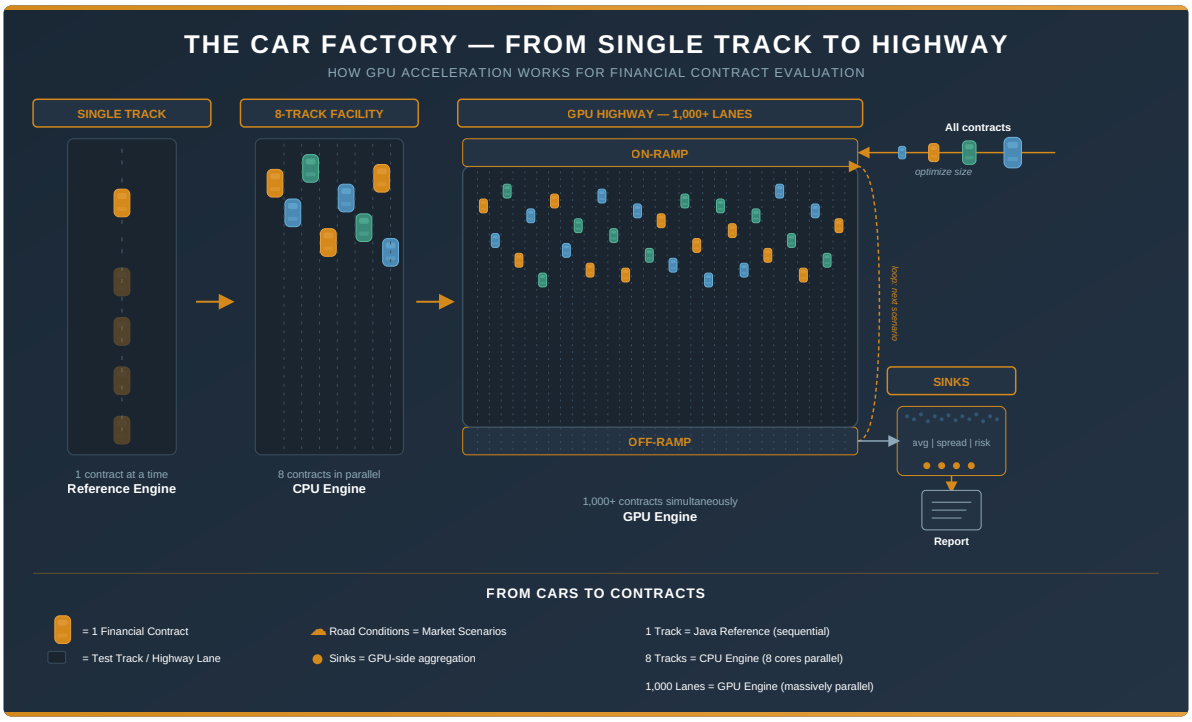
Imagine a Car Factory

You run a car factory. You have built 100,000 different car models and you need to know how every single one of them performs — on dry roads, wet roads, icy roads, in extreme heat, in extreme cold, uphill, downhill, fully loaded, and empty. Each combination of car and road condition is a test run that produces data: fuel consumption, brake wear, tyre grip, engine temperature, and dozens more measurements.

That is exactly the problem financial institutions face. Each "car" is a financial contract — a loan, a deposit, a bond, an insurance policy. Each "road condition" is a possible future — different interest rates, different economic climates. Each "test run" projects the contract's cash flows under that future and produces risk metrics. To understand the risk in a portfolio, you need to test every contract under every scenario.

With 100,000 contracts and 1,000 scenarios, that is **100 million test runs**.

If you have one test track and run them one by one, it takes hours. This project was about building a **thousand-lane highway** that runs them all at the same time — and proving that every car still produces exactly the same test results as it would on the original single track.



What is ACTUS?

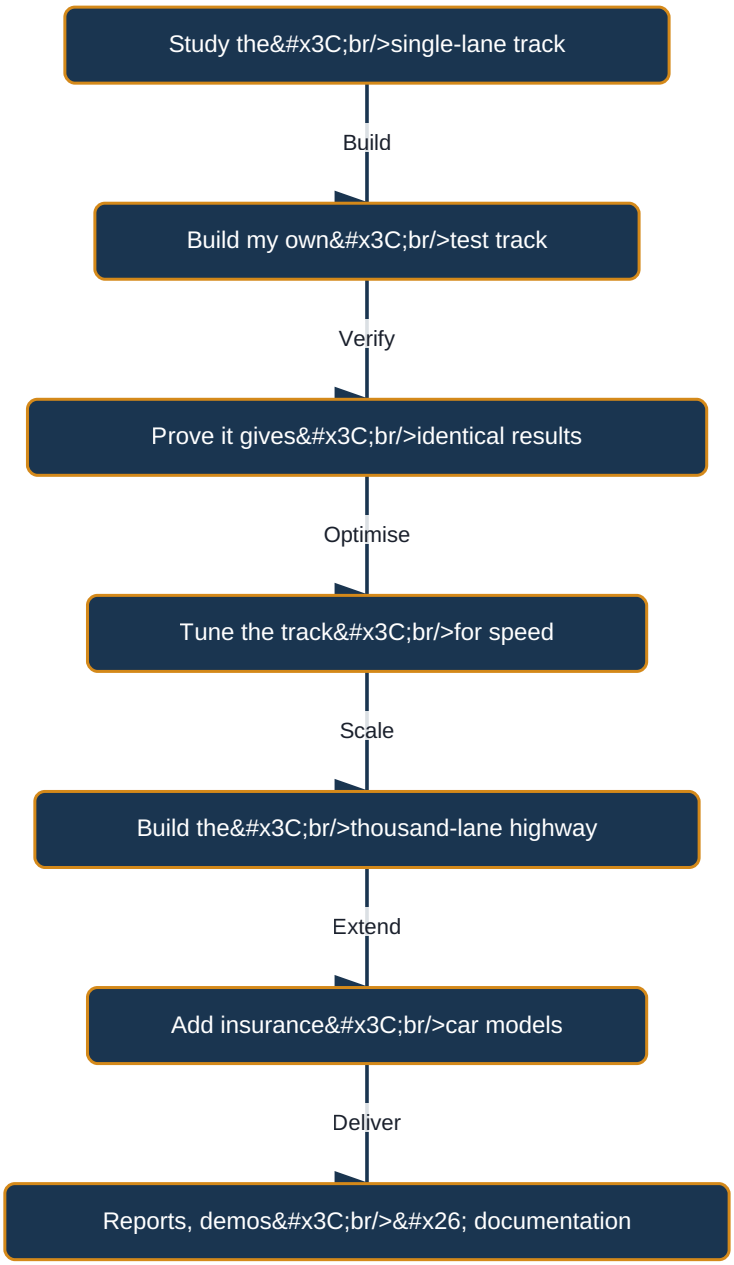
ACTUS (Algorithmic Contract Types Unified Standards) is the rulebook for the test track. It defines, in precise mathematical terms, how every type of financial contract behaves: when payments occur, how interest accrues, what happens at maturity, and how the contract responds to changing market conditions.

The critical property of ACTUS is **determinism**: given the same contract and the same market data, the rules always produce exactly the same cash flows. No interpretation, no ambiguity. This is what regulators, auditors, and risk managers need — numbers they can trust and independently verify.

The ACTUS standard already had a reference implementation — a working single-lane test track written in Java. It is correct and well-tested. But it processes one contract at a time. For institutional-scale portfolios, it is simply too slow.

What I Built

Working largely on my own, with a little help from one other person, I followed a disciplined sequence — like upgrading a test facility one step at a time, verifying that test results stay identical at every stage:



The result:

In the Car Factory	In the Financial World
42 reference cars tested on both tracks — identical results to 10 decimal places	All 42 official ACTUS test cases pass on both the CPU and GPU engines
The highway handles 100,000 cars simultaneously, over 2× faster than the 8-track facility	GPU evaluates 100,000 life insurance policies in 273 ms vs. 735 ms on CPU (2.7× faster)

In the Car Factory	In the Financial World
Cars stay on the highway and loop through 1,000 road conditions without re-entering	Monte Carlo: 10,000 contracts × 10,000 scenarios in 28 seconds (5× faster than CPU)
New car types added — insurance vehicles with different test programmes	Life insurance contracts with state transitions and actuarial tables
Highway sinks aggregate measurements before sending them off the highway	Risk metrics computed on the GPU before transfer — no off-ramp congestion
Full reporting: results by car type, by road condition, by factory division	Excel-friendly exports grouped by segment, region, product line

Why Insurance?

My background is in the insurance industry. I came to this project not from the banking or finance side, but from having seen first-hand how insurers struggle with the same problems ACTUS was designed to solve — just in a different product domain. Accuracy is non-negotiable (regulators demand it), but so is speed (actuaries need answers in seconds, not hours).

ACTUS gave me a rigorous, deterministic framework that could handle financial contracts. The question I wanted to answer was: can this framework be extended to insurance, and can it be made fast enough for real institutional use? The hackathon gave me the deadline and the motivation to find out.

The Guiding Rule

Throughout the entire project, one rule was never broken: **every car must produce the same test results on the highway as it does on the original single track.** If any optimisation changed even one measurement by more than a billionth, it was rejected.

This is what makes the system trustworthy. It is not just fast — it is provably correct.

Continue Reading

Start with the car factory story — it explains everything else:

1. Understanding CPU and GPU — the full car factory analogy: single track, eight tracks, the highway, the on-ramp bottleneck, staying on the highway, sinks on the off-ramp
2. Development Timeline — how the facility was built, phase by phase

3. Key Decisions — why I built a new track, how I designed the highway, and other critical choices
4. Challenges & Solutions — the problems I ran into and how I solved them
5. Outcomes & Benchmarks — the test results: how fast, how correct, and what it means
6. Installation & Getting Started — how to install and run the system: Docker demo, CLI tool, developer build, and documentation site

Competition Submission — ACTUS-I

Competition Submission

Selected Proposal: ACTUS-I — Extending Algorithmic Financial Contracts into Insurance with High-Performance Portfolio Projections

---

Project Solution Description (≤2000 characters)

The project introduces **ACTUS-I**, an extension of the ACTUS Algorithmic Financial Contract Standard that enables insurance contracts to be represented and projected using the same deterministic, machine-readable framework already used for banking products. The solution demonstrates that banking and insurance contracts can be modeled, simulated, and analyzed within a **single unified projection engine**, enabling transparent and scalable financial risk analysis.

ACTUS-I defines insurance-specific contract structures while preserving the core ACTUS lifecycle model. It introduces insurance attributes such as premiums, coverage limits, deductibles, and reinsurance terms, together with state variables for claims, benefits, reserves, and policy status. Contract lifecycles are represented through events such as premium payments, claim triggers, benefit payouts, and policy state transitions, allowing insurance cash flows to be computed deterministically from contract definitions.

To model the probabilistic nature of insurance outcomes, the system incorporates a **Markov state-machine representation of policy lifecycles**. At each time step, the model propagates probabilities across states (for example active, lapsed, claim open, or benefit paid) using actuarial hazard tables for mortality, lapse, and disability. This produces probability-weighted expected cash flows while preserving the reproducibility required by the ACTUS standard.

The contracts are executed in a **high-performance projection engine** capable of processing large portfolios of mixed banking and insurance contracts. The engine supports both deterministic projections and Monte Carlo scenario simulations, enabling forward-looking portfolio analysis, tail-risk measurement, and economic capital estimation. To ensure consistency between portfolio projections and individual contract calculations, the same computational kernel can run on both CPU and GPU architectures.

By combining a standardized contract representation with scalable projection technology, the solution enables institutions to perform fast, transparent, and reproducible portfolio

analysis across both assets and liabilities. The result is a unified analytical framework that supports risk management, regulatory reporting, and forward-looking financial decision-making at scale.

URL to Demonstration Video

Video of the presentationapplication demo

URL to Solution

Demo site with information and resources

Core Team Members

Frans Van Ek — Led the core setup, programmed the GPU kernels, converted the site documentation into the new layout, and developed the PAM and insurance contract extensions.

Cristina Mudura - Contributed on the deployment, creation of the base documentation site, and validation of the implementations.

Key Innovations and Unique Value Proposition

A major innovation lies in the execution architecture of the projection engine. Large-scale portfolio valuation requires enormous computational throughput: projecting millions of policies across hundreds of time steps with stochastic decrements is inherently parallel and naturally suited for GPU acceleration. However, GPU-only implementations introduce a serious practical problem: calculation drift.

In most actuarial systems, individual contract calculations (for example policy servicing, illustration, or audit checks) are executed on the CPU, while portfolio projections run on specialized GPU code. Even when both implementations aim to follow the same formulas, small differences in code paths, floating-point behavior, or update order inevitably appear over time. This creates reconciliation problems: the projection engine may produce results that cannot be reproduced by the single-policy calculation engine.

The innovation in this project is that the exact same computational kernel is used for both CPU and GPU execution.

The other innovation is the **principled extension of the ACTUS standard into the insurance domain** — something that has not previously existed in a production-grade open implementation.

ACTUS was designed for banking contracts, where every cash flow follows a deterministic schedule. Insurance contracts are fundamentally different: whether a policy pays a death benefit, lapses, or matures depends on probabilities, not on a fixed date. Bridging these two paradigms required solving several distinct problems simultaneously.

1. A Markov state machine for insurance contracts

The project introduces a 10-state Markov model as the formal representation of insurance policy lifecycle. Rather than projecting a single fixed path, the model carries forward a probability distribution across all possible states at every monthly time step — Active, Lapsed, Grace Period, Claim Open, Death Claim Paid, and others. At each step the model applies competing-risk decrements from actuarial hazard tables (mortality via Gompertz-Makeham, lapse by duration, disability by age) and produces a probability-weighted expected cash flow. This is mathematically rigorous, reproducible, and fully compatible with the ACTUS design principle of determinism: given the same actuarial assumptions, the same expected cash flows are always produced.

2. A domain-specific language for insurance product rules

Actuarial tables alone cannot capture the full variation between insurance products. Grace period lengths, paid-up conversion formulas, reinstatement eligibility windows, and underwriting loadings are product-specific and cannot be expressed in a simple lookup table. The project includes a complete DSL — lexer, recursive-descent parser, evaluator — that allows product rules to be defined in structured text without modifying the engine. New products are added by writing a new rule pack; the projection infrastructure is unchanged. This separates product design from engineering and allows actuaries to work independently of the development team.

3. Monte Carlo simulation on a parallel architecture

A single best-estimate projection is insufficient for risk management. The project supports full stochastic Monte Carlo simulation: thousands of scenarios are constructed by perturbing actuarial assumptions (mortality shocks, lapse shocks, disability shocks), each scenario runs the complete Markov projection independently, and the collection of results builds a probability distribution over portfolio outcomes. This makes the 95th percentile loss, tail risk quantification, and economic capital calculation directly computable — not approximated.

4. A unified architecture across banking and insurance

Both banking contracts (via the existing ACTUS standard) and insurance contracts share the same underlying projection infrastructure. A portfolio containing both fixed-rate bonds and life insurance policies can be evaluated in a single run, producing a consolidated cash flow view across the entire balance sheet. This is valuable for insurers that also hold investment portfolios and for banks that offer insurance products — the boundary between the two domains has historically been a reporting and systems barrier. This project removes it.

5. Configurability through the Markov graph definition

The state diagram itself is an externally defined resource, not hard-coded. By changing the graph definition, different product structures — term life, whole life, critical illness, disability income — can be modelled without any engine changes. This makes the architecture product-agnostic and future-proof.

Possible Next Steps and Subsequent Developments

The work completed in the competition establishes a foundation that can be extended in several important directions.

Broader contract type coverage. The current implementation covers life insurance with mortality, lapse, and disability hazards. Natural extensions include critical illness contracts (with specific illness-event triggers), disability income (with recovery transitions from claim back to active), and group insurance (where the unit of projection is a cohort rather than an individual policy).

Regulatory integration. Insurance regulation increasingly requires stochastic solvency projections — Solvency II in Europe and analogous frameworks elsewhere mandate that insurers demonstrate solvency under a range of adverse scenarios. The Monte Carlo engine built in this project is directly applicable to producing the scenario outputs required for regulatory capital calculations. Standardising the scenario definitions and output format to match regulatory specifications would make the tool immediately usable in compliance workflows.

Reinsurance modelling. The current model projects direct insurance policies. Reinsurance arrangements — quota share, excess of loss, stop loss — modify the cash flow profile at portfolio level. Layering reinsurance structures on top of the gross projection would allow insurers to evaluate the net impact of their reinsurance programme under different scenarios.

Integration with ACTUS financial contract data. Insurers hold investment portfolios alongside their insurance liabilities. Connecting the insurance projection output to the existing ACTUS financial contract engine would allow asset-liability management analysis: projecting both sides of the balance sheet under the same scenario assumptions and measuring the mismatch.

Actuarial table marketplace. The current implementation uses built-in parametric models. A natural evolution is a pluggable table registry where insurers can supply their own experience-based mortality and lapse tables, or access standardised population tables by jurisdiction. This would let the tool be deployed across different markets without rebuilding the actuarial inputs.

User interface for actuaries. The current work is an engine and a data standard extension. A practical next step is a workbench interface — web or desktop — that allows actuaries to define products, load portfolios, configure scenarios, and visualise results without writing code. The DSL and rule pack structure is already designed to support this.

Challenges in Realising the Solution

Conceptual challenge — bridging deterministic and probabilistic contract paradigms. ACTUS is designed for deterministic financial contracts with predefined event schedules. Insurance contracts behave differently: policy outcomes depend on probabilistic events such as death, lapse, or disability. The challenge was to extend the ACTUS framework without losing its key property of reproducibility. This required introducing a Markov state-machine representation of the insurance lifecycle while ensuring that the final projected cash flows remain deterministic expected values.

Technical challenge — high-performance computation with consistent results. The engine needed to support large-scale GPU parallelism while maintaining exact agreement with CPU calculations. Financial contracts also contain heterogeneous data structures (dates, text labels, variable-length events) that GPUs cannot process directly. In addition, floating-point rounding differences between execution environments created small numerical discrepancies during validation.

Data challenge — reliable and consistent inputs. Financial models are highly sensitive to input interpretation. Even minor differences in numeric formatting or configuration could lead to large calculation errors. Ensuring that interest rates, actuarial tables, and reference datasets were interpreted consistently across environments required strict standardisation of numeric parsing and data handling.

Scope challenge — balancing performance goals with practical constraints. GPU acceleration offers significant performance gains for large portfolios but introduces setup overhead. For small workloads this overhead can outweigh the performance benefits, making CPU execution faster.

How Challenges Were Overcome

On the paradigm bridge: Insurance contracts were represented as a Markov state machine driven by actuarial hazard tables. Instead of simulating individual stochastic paths, the engine propagates state probabilities and computes expected cash flows. This preserves the deterministic ACTUS guarantee: identical inputs always produce identical outputs.

On the technical challenges: A dedicated transformation layer converts contracts into compact numeric structures suitable for parallel execution. Sensitive calculations such as discount factors are computed once and reused to avoid numerical drift, and explicit event priority rules ensure deterministic processing when multiple events occur on the same date.

On the data challenge: All numeric inputs are parsed using a fixed, locale-independent convention. Reference tests were executed across different system configurations to confirm consistent results.

On scope and performance: The system supports both CPU and GPU execution. Smaller workloads run efficiently on the CPU, while large portfolios benefit from GPU parallelism, allowing the platform to scale without sacrificing practicality.

Results and Success Rating

The project delivered a working prototype that demonstrates the feasibility of **ACTUS-I — extending the ACTUS algorithmic financial contract framework into the insurance domain while maintaining the standard's deterministic and reproducible design principles**.

The implementation includes the core architectural elements required to validate the vision of a unified contract projection framework:

- **Markov state-machine model** representing the lifecycle of insurance contracts.
- Integration of **actuarial hazard tables** and probability-based state transitions.
- **domain-specific language (DSL)** enabling product rules to be defined independently of the projection engine.

- **Monte Carlo scenario simulation** integrated with the ACTUS execution infrastructure.
- **GPU-enabled projection engine** capable of processing portfolio-scale calculations while preserving deterministic results consistent with CPU execution.

Most importantly, the system demonstrates that **banking contracts and insurance contracts can be processed within the same ACTUS-based projection engine**, supporting the broader vision of a unified algorithmic financial contract standard capable of large-scale forward-looking portfolio analysis.

The project also produced a structured **documentation and demonstration website** that explains the architecture, implementation decisions, and results. This website serves as the primary evidence artifact for the project and documents the complete design and working prototype.

The insurance extension has been **implemented and validated using a full working product example**, proving the technical feasibility of the ACTUS-I contract model. While only one product type was implemented within the available timeframe, the DSL-based architecture allows additional products to be added without changes to the projection engine.

The reporting and monitoring infrastructure described in the vision has been **partially implemented**. Initial reporting pipelines and demonstration outputs exist, but the full automated reporting, alerting, and early-warning framework remains future work.

The primary objective — **demonstrating that ACTUS can be extended into the insurance domain while remaining standards-compatible, reproducible, and computationally scalable** — was successfully achieved. The working prototype executes insurance projections within the ACTUS engine and demonstrates GPU-accelerated portfolio calculations.

Self-assessed success rating: 4 / 5

All core architectural objectives were achieved: the ACTUS-I insurance extension, unified projection engine, GPU-enabled portfolio processing, and demonstration of insurance contract projections. Remaining stretch goals mainly concern completing the automated reporting and early-warning capabilities envisioned for the platform.

Additional Materials

Repositories

- **Actus-Insurance.Documentation** — Public
  - The documentation you are reading now. Contains all conceptual and technical documentation for the insurance extension: the Markov model, DSL and product rules, Monte Carlo simulation, and the life insurance projection model.
- **Actus-Insurance.Core** — Public
  - The C# implementation of the insurance contract extensions. Contains the Markov state machine, the DSL interpreter, actuarial lookup tables, the rule pack loader, and the insurance contract adapter that connects to the ACTUS contract engine.
- **Actus-Insurance.GPU** — Private
  - The high-performance GPU kernel that runs the portfolio projection. This repository is proprietary and not publicly available. It publishes compiled packages to GitHub Packages, which the other repositories consume as dependencies.
- **Actus-Insurance.DemoAndSamples** — Public
  - End-to-end examples showing how to use the insurance extension in your own solution. This is the recommended starting point for implementers. It consumes the published GPU packages as a dependency and demonstrates how the Core, GPU, and Documentation components work together in practice.

Suggested materials to include if available:

- **Technical architecture document** — overview of the system design, component responsibilities, and data flows
- **Actuarial validation report** — comparison of model output against known actuarial benchmarks or independently computed reference values
- **Test portfolio results** — sample output showing projected cash flows, state probability distributions, and Monte Carlo percentile bands for a representative policy or portfolio
- **DSL rule pack examples** — sample product definitions illustrating how term life and whole life products are expressed in the rule language
- **Slides or poster** — visual summary of the innovation and results suitable for a pitch or poster session

[TODO: insert links or upload references for any of the above that exist]

Value and Benefits Realised

The participation in the competition crystallised something that the ACTUS community has discussed but not yet implemented: a unified standard that covers both financial and insurance contracts within the same formal framework.

The value demonstrated is concrete and at multiple levels.

For the ACTUS standard itself, the work shows that the core design principles — determinism, reproducibility, separation of data standard from algorithmic standard — extend naturally to the insurance domain. The extension did not require abandoning or compromising those principles. It validated that the framework is genuinely general.

For insurers, the combination of a formal contract standard, a configurable product rule language, and a high-performance stochastic projection engine addresses a real and expensive problem: most insurers today run their liability projections on systems that are slow, inflexible, and proprietary. A standards-based open approach reduces vendor lock-in, improves auditability (the rules are readable, not buried in compiled code), and makes it practical to run the thousands of scenarios that modern risk management and regulation require.

For regulators and supervisors, a standardised machine-readable representation of insurance contracts offers the same benefits it provides in banking: the ability to collect and compare contract data across institutions in a common format, and to apply scenario analysis systematically rather than institution by institution.

For the broader fintech and insurtech ecosystem, demonstrating a working open implementation lowers the barrier for other developers to build on the standard — whether as analytics tools, pricing engines, ALM systems, or regulatory reporting solutions. There is access needed to be able to run this repo. it is merely for showcasing the code.

Available to Pitch Virtually?

- ■ Yes
- □ No

Personal Note

Where It Began

Starting with nothing — only a basic curiosity and the opportunity to attend the CORA Summit in Zagreb — I began my journey into the ACTUS world. At that moment, the terminology, the structure, and even the way financial contracts were reasoned about were completely new to me.

Everything changed during a breakfast conversation where I received a kind of crash course in ACTUS from Francis Gross and Willi Brammertz. Listening to them explain the reasoning behind the standard, I slowly started to see the elegance and potential behind it. The idea that financial contracts could be described in a structured, algorithmic way immediately resonated with me.

Recognising the Connection to Insurance

Coming from the insurance domain, I quickly began to recognise similarities. Insurance contracts, just like financial contracts, are fundamentally defined by events, state transitions, obligations, and cashflows. That realisation sparked the thought that the ACTUS approach might also be applicable beyond traditional finance — particularly in the insurance world.

Choosing a Technical Path

When I later learned about the hackathon, I started thinking about what I could realistically contribute. My background is in software development and software architecture, so I decided to focus on something technical. I could have taken the route of proposing conceptual ideas for insurance extensions to ACTUS, but without a working implementation that would remain theoretical. Instead, I chose a technically challenging path: building something that actually works.

The primary objective therefore became ensuring that the implementation behaves correctly according to the ACTUS definitions for financial contracts. From there, I explored how these concepts could be extended into the insurance domain. In that part of the work, the focus was less on perfect correctness and more on identifying the structural requirements necessary to model insurance contracts within an ACTUS-style framework. Many of those explorations are documented in the chapters that follow, including the challenges encountered along the way.

What I Learned

Through this process I learned a tremendous amount. The journey required stepping into two unfamiliar worlds at the same time: the financial modelling domain and a new technical architecture to support it. Seeing those two domains gradually come together into a working model is something I am genuinely proud of and gives me a strong sense of achievement.

Where This Can Go

But this work is not an endpoint. The goal was never only to build a proof-of-concept. The real objective is to explore a path forward where ACTUS can serve as a foundation for insurance modelling as well. The insurance industry has specific needs — particularly the ability to change products, rules, and regulatory requirements quickly and frequently. To address that, I experimented with implementing a Markov graph model and a domain-specific language that allows business rules to be defined without requiring software development. This brings product configuration closer to the business domain and within reach of non-developers.

I hope you can appreciate both the effort and the results presented in this documentation. More importantly, I hope this work can serve as a starting point for further exploration.

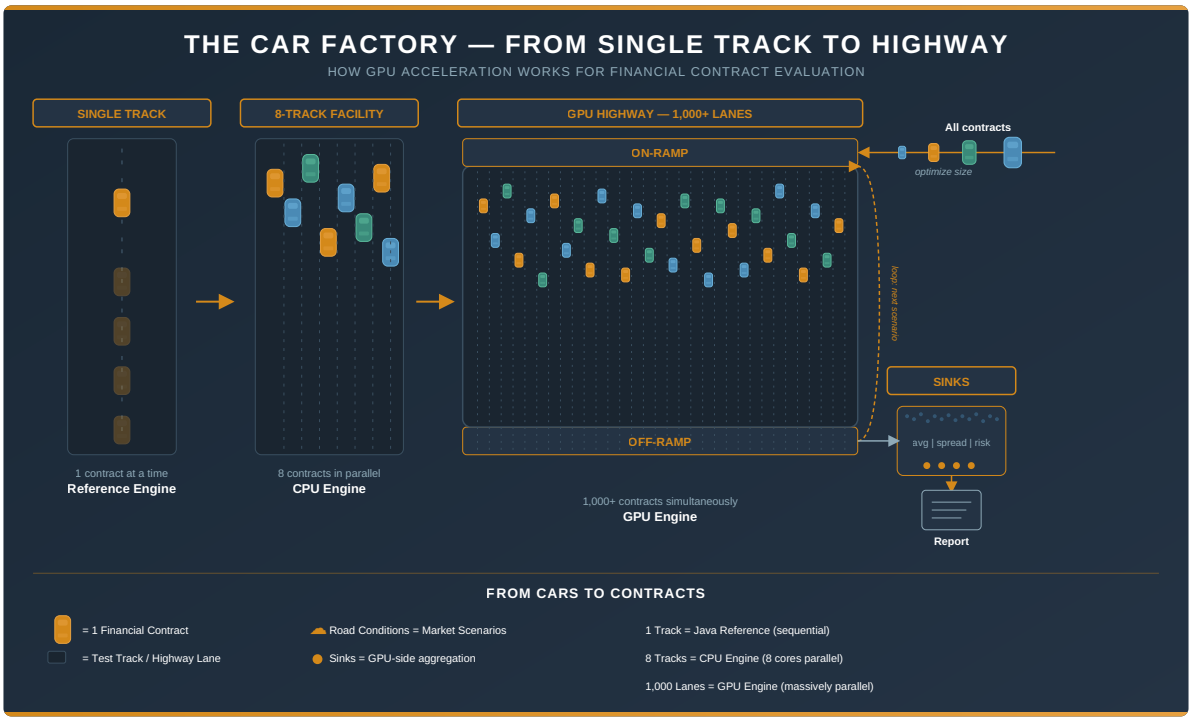
Let's see where this can lead us.

Frans

Understanding CPU and GPU — The Car Factory

Understanding CPU and GPU — The Car Factory

This is the foundational story for the entire hackathon section. If you read one document, read this one. Every concept in the other documents maps back to the analogy introduced here.



The Factory Floor

You are the head of testing at a car factory. Your company has designed many different car models — some are small city cars, some are heavy trucks, some are luxury sedans. Before any of them can go to market, you need to know how they perform under every imaginable condition: dry asphalt, wet roads, icy mountain passes, scorching desert heat, freezing Nordic winters, city stop-and-go, and open highway cruising.

For each combination of car and road condition, your team puts the car on the track, runs the scenario, and collects data: fuel consumption, brake wear, tyre grip, engine temperature, suspension load, emissions, and dozens of other measurements.

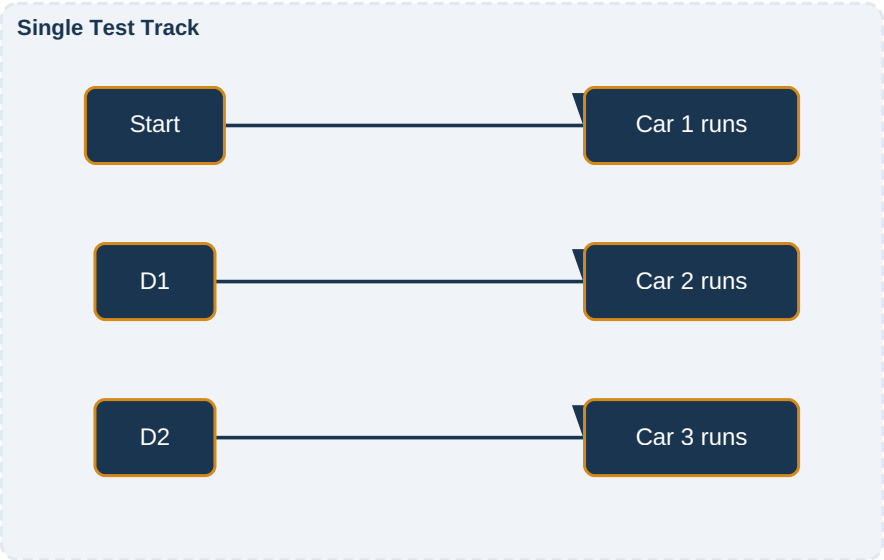
This is **exactly** what financial contract evaluation does:

Car Factory	Financial World
A car model	A financial contract (loan, bond, deposit, insurance policy)
A road condition (wet, icy, hot)	A market scenario (rising rates, falling rates, crisis)
A test run on the track	Evaluating a contract's cash flows under one scenario
Measurements collected	Cash flow amounts, present values, risk metrics
The full fleet of cars	The institution's portfolio
All road conditions tested	Monte Carlo simulation across thousands of scenarios

Now the question is: how fast can you test the entire fleet?

Stage 1 — One Car, One Track

In the simplest setup, you have a single test track. One car goes out, drives the scenario, comes back. The next car goes out. One at a time.

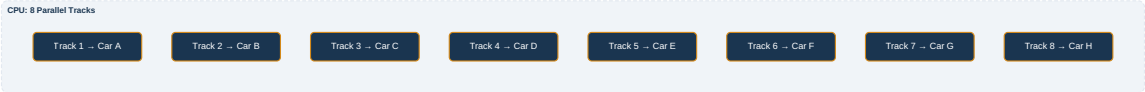


For 100,000 cars under 1,000 road conditions, that is 100 million runs. Sequentially. You will be here for a very long time.

This is how the original ACTUS reference implementation works. It evaluates one contract at a time, one scenario at a time. The results are correct — the track itself is excellent — but the throughput is limited to the speed of one track.

Stage 2 — Eight Parallel Tracks (the CPU)

A modern computer processor — the CPU — is not one track. It has multiple cores, typically 8 to 16, each of which can run a test independently. Think of it as upgrading your facility from one track to eight:



Now 8 cars are tested at the same time. Each track is powerful and versatile — it can handle complex test programmes, change conditions mid-run, and adapt to very different car types. But there are only 8 tracks.

For 100,000 cars, this is 8 times faster than before. "Impossibly long" becomes "long but feasible."

This is the optimised CPU engine I built during the hackathon. Same ACTUS calculations, but running on all available processor cores simultaneously.

Stage 3 — The Thousand-Lane Highway (the GPU)

A GPU is a fundamentally different kind of facility. Instead of 8 sophisticated tracks, it is a **massive highway with thousands of lanes** where thousands of cars drive at the same time.

Each individual lane is simpler than a CPU track. A lane cannot handle complex mid-run changes the way a CPU track can. But the sheer number of lanes — a thousand or more running at once — means you can test the entire fleet in a fraction of the time.

There is one crucial property that makes this possible: **every car's test is independent.** The result of testing Car A on wet roads has no effect on the result of testing Car B on icy roads. They do not need to wait for each other or share information. This kind of workload — thousands of completely independent tasks — is exactly what a highway is built for.

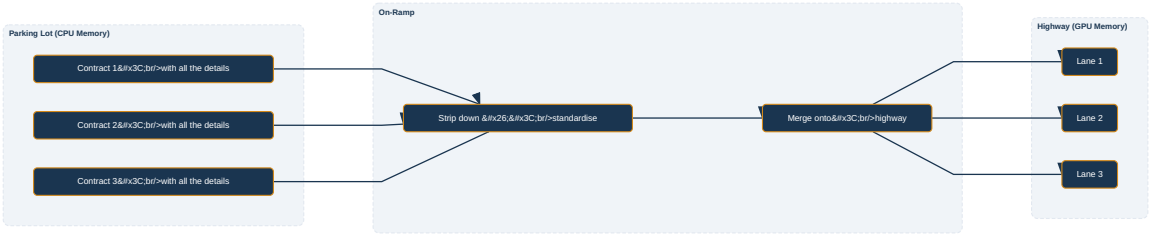
This is the GPU engine. The same ACTUS calculations, but running on thousands of GPU lanes simultaneously.

The First Bottleneck: The On-Ramp

Here is where it gets interesting. The highway is enormously fast, but the on-ramp is narrow.

Before a car can enter the highway, it has to be prepared. In a real factory, that might mean fuelling up, checking tyre pressure, and programming the navigation system. In the financial world, it means **converting each contract from its natural, human-readable format into the compact format the highway demands**.

The highway lanes are uniform and standardised. They do not accept cars with custom bodywork sticking out, trailers attached, or luggage strapped to the roof. Every car must be stripped down to a sleek, uniform shape that fits perfectly into a lane.



No matter how many lanes the highway has, if the on-ramp is congested, lanes sit empty waiting for traffic. **The on-ramp is often the real bottleneck — not the highway itself.**

The solution: make the cars as small and uniform as possible.

In the financial world, this means converting contract data into the most compact possible form:

What the contract carries naturally	What it looks like on the highway
The text "Annual" for payment frequency	The number 12
A date like "March 15, 2025"	A single number (a tick count)
A list of 20 scheduled events	A position and count in a shared flat list
A label like "Fixed Rate"	A code: 1

The smaller and more uniform the packages, the faster they flow through the on-ramp. I built a dedicated translation layer that handles this compression automatically — the engine works with contracts in their natural form; only when the highway is needed does the streamlining happen.

Staying on the Highway

After a car completes one test run, it needs to run the next scenario. If it exits the highway, drives all the way back to the parking lot, gets re-prepared, and enters through the on-ramp again — you have just doubled the on-ramp congestion.

The much better approach: **keep the car on the highway.** After completing one run, each car loops back to the start of its lane and begins the next scenario immediately. No off-ramp, no parking lot, no on-ramp. Just a change of road conditions.

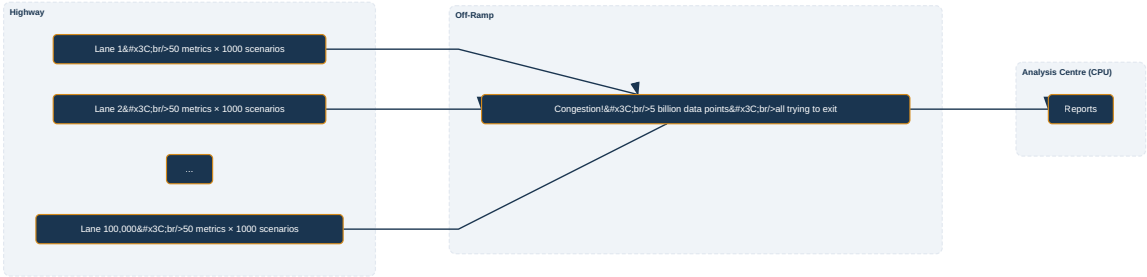
The contract data stays on the highway — it is loaded once and reused for every scenario. Only the scenario parameters (the road conditions) change between runs. This is a lightweight update: instead of moving the entire car, you just change the weather on the track.

This is enormously powerful for Monte Carlo simulation, where the same portfolio of 100,000 contracts is tested under 1,000 or even 10,000 different interest rate scenarios. The portfolio crosses the on-ramp once. The scenarios flow in as small, fast updates.

The Second Bottleneck: The Off-Ramp

Now the highway is running at full speed. Thousands of cars are completing their runs and producing data. But here is the new problem: **every car generates a huge number of measurements during its run.**

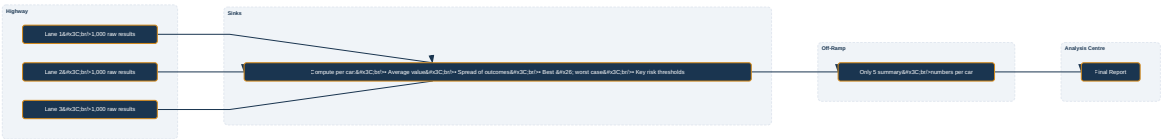
For 100,000 cars, each producing 50 data points per scenario, under 1,000 scenarios, that is **5 billion individual measurements** trying to get off the highway and back to the analysis centre.



The off-ramp becomes the new bottleneck. If you are not careful, the time spent getting data off the highway exceeds the time saved by running on it.

The solution: aggregate before leaving the highway.

Instead of sending every raw measurement off the highway, I built **sinks** — structured collection points that gather the raw results and then compute compact summaries. Each sink takes the thousand scenario results for one car and produces just a handful of numbers: the average, the spread, the best case, the worst case, and a few key risk percentiles.

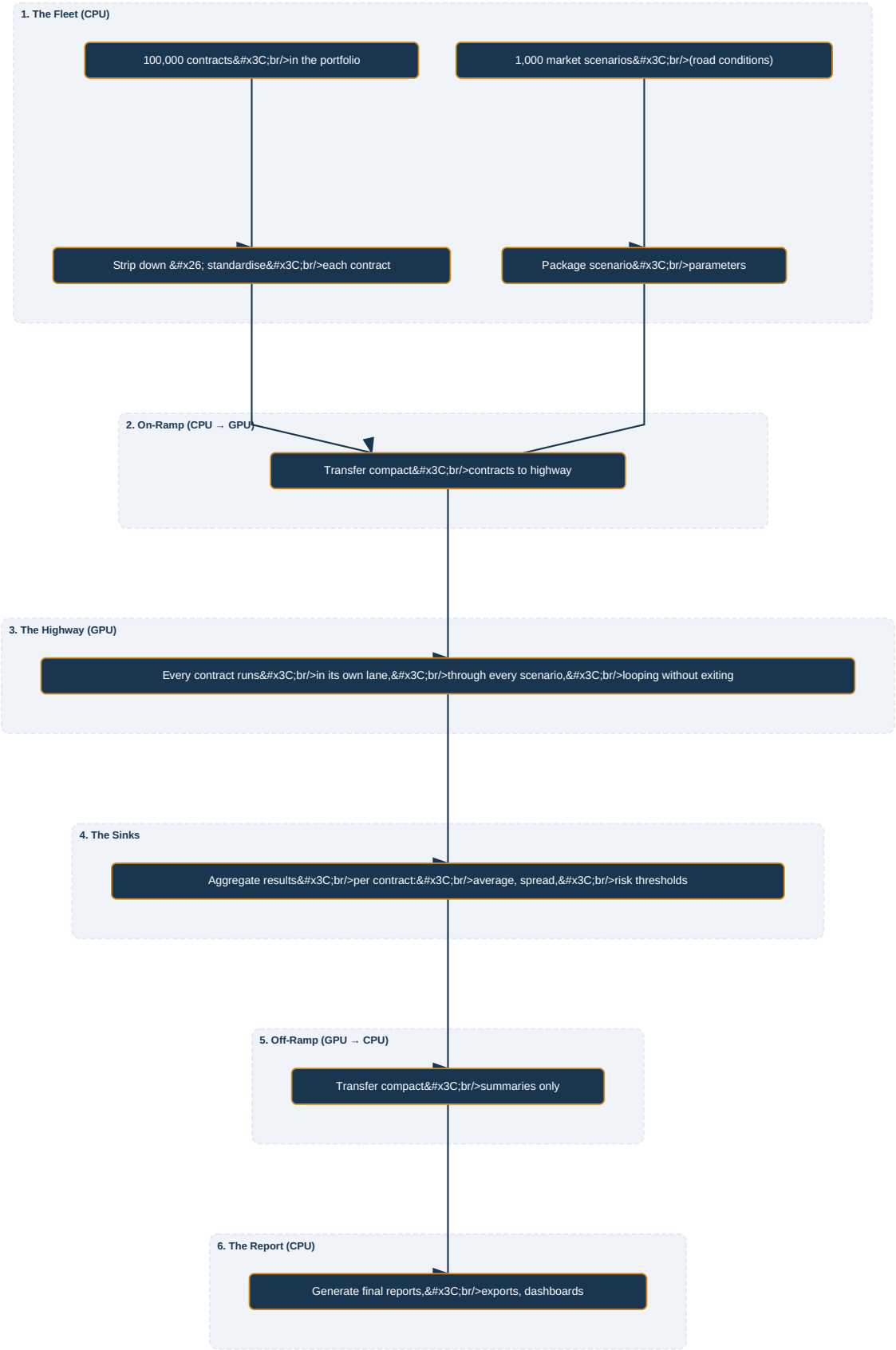


Now instead of 5 billion raw data points, only 500,000 summary values cross the off-ramp (5 numbers per car × 100,000 cars). That is a **10,000× reduction** in off-ramp traffic. The analysis centre receives exactly what it needs for reporting — compact, pre-digested, ready to use.

The key design insight is that the raw measurements are collected in a structured, flat format that makes aggregation extremely efficient. The sinks are not an afterthought — they are a core part of the highway architecture, designed from the start to prevent the off-ramp from becoming the bottleneck.

The Complete Journey

Here is the full picture, from fleet to final report:



The bottlenecks are never on the highway itself — the GPU handles the raw computation easily. The engineering challenge is entirely in the **on-ramp** (getting data onto the highway efficiently) and the **off-ramp** (getting results back without congestion):

Bottleneck	Problem	Solution
On-ramp	Contracts must be converted to highway-compatible format	Strip to compact numeric form — make the cars as small as possible
Staying on	Re-entering the on-ramp for every scenario wastes time	Keep contracts resident on the highway — loop, don't exit
Off-ramp	Raw results from millions of runs create congestion	Sinks aggregate the results — exit with just the summary

---

From Cars to ACTUS Contracts

Everything in this analogy maps directly to the ACTUS financial contract engine I built during the hackathon:

Car Factory	ACTUS Engine
A car model	A financial contract (PAM loan, insurance policy, etc.)
The test track rulebook	The ACTUS standard — deterministic rules for every contract type
Road conditions (dry, wet, icy)	Market scenarios (different interest rate paths)
A single test run	Projecting one contract's cash flows under one scenario
Measurements (fuel, brakes, grip)	Financial metrics (payments, present value, risk)
One track (original)	The reference Java implementation — one contract at a time
8 parallel tracks (CPU)	8 CPU cores evaluating contracts simultaneously
1,000-lane highway (GPU)	Thousands of GPU threads evaluating contracts simultaneously
Stripping cars for the on-ramp	Converting contracts to compact GPU-compatible format
Looping without exiting	Reusing contract data in GPU memory across all scenarios
Sinks at the off-ramp	Aggregating risk metrics before transfer
The analysis centre report	Excel exports, dashboards, grouped portfolio summaries

The hackathon proved two things: first, that the highway produces **exactly the same test results** as the original track for every car (all 42 ACTUS test cases pass to 10 decimal places). And second, that for large fleets — the kind real institutions manage — the highway is **dramatically faster**.

The other documents in this section tell the story of how the highway was built:

- Development Timeline — the construction phases
- Key Decisions — why I built it this way
- Challenges & Solutions — the roadblocks I hit
- Outcomes & Benchmarks — the final test results

Monte Carlo Test Sample

PAM Monte Carlo — 50-Year Demo Run

> ***Engine:*** *ActusCoreCsharp*
>
> ***GPU:*** *NVIDIA GeForce RTX 3060 Ti*
>
> ***Contracts:*** *10,000* · ***Scenarios:*** *1,000* · ***Horizon:*** *600 months (50 years)*
>
> ***Seed:*** *12345* · ***Backend:*** *Both (CPU + GPU)* · ***CalcDate:*** *2020-01-01 (month 0)*

Executive Summary

This sample run executes the full PAM Monte Carlo pipeline against a synthetic portfolio of 10,000 contracts over a 50-year horizon (600 monthly steps), with 1,000 Vasicek interest-rate scenarios. Both the CPU and GPU backends ran for every configuration, producing **bit-identical risk metrics** while the GPU delivered a **7.5× speedup** on the core computation kernel.

Three headline findings:

Finding	Evidence
GPU delivers **7.5× speedup** on 10M contract-scenario pairs	CPU 13,530 ms vs GPU 1,796 ms (calc only)
CPU and GPU produce **bit-identical results**	PV mean, VaR99, ES99 match to the cent on both runs
GPU speedup **scales with workload** — 3.1× at 50k, 7.5× at 10M pairs	Fixed kernel-launch cost amortised over more work

---

1 · Vasicek Scenario Generation

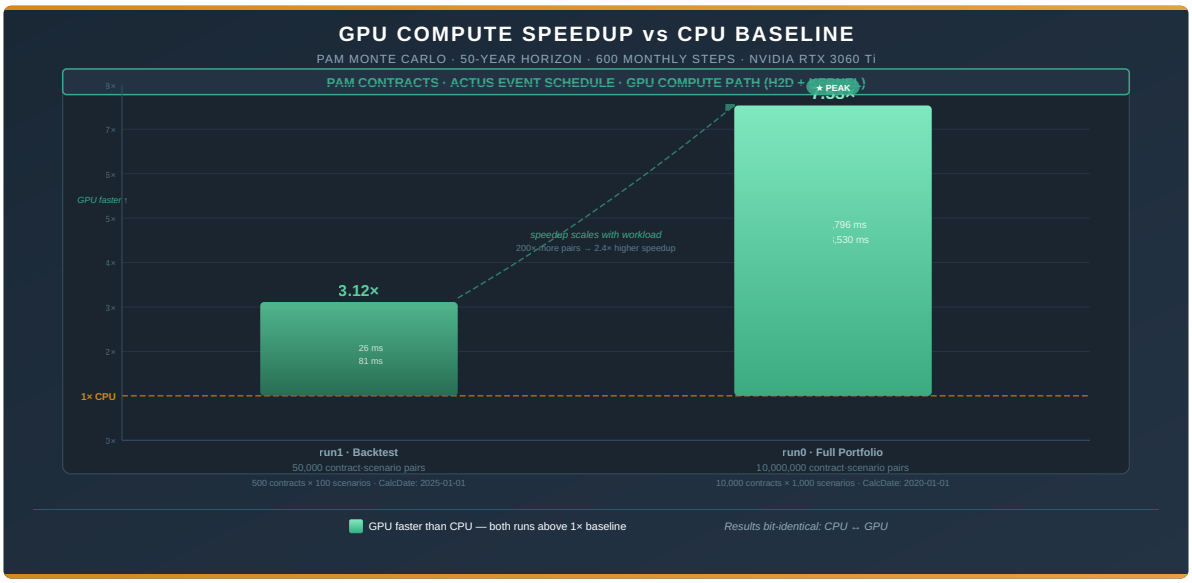
1,000 paths · 600 monthly steps · generated in 47 ms

The Vasicek model is mean-reverting: starting at 3.00%, it converges toward a long-run mean of ~5% over the 50-year horizon.

Checkpoint	Value	Interpretation
Mean short rate t=0	3.00%	Starting conditions (2020-01-01)
Mean short rate t=300 (25y)	4.77%	Rate environment mid-simulation
Mean discount factor t=120 (10y)	0.6782	10-year PV factor
Scenario count	1,000	Paths sampled from Vasicek distribution
Generation time	47 ms	Wall time for all 1,000 × 600 = 600,000 steps

---

2 · GPU Speedup



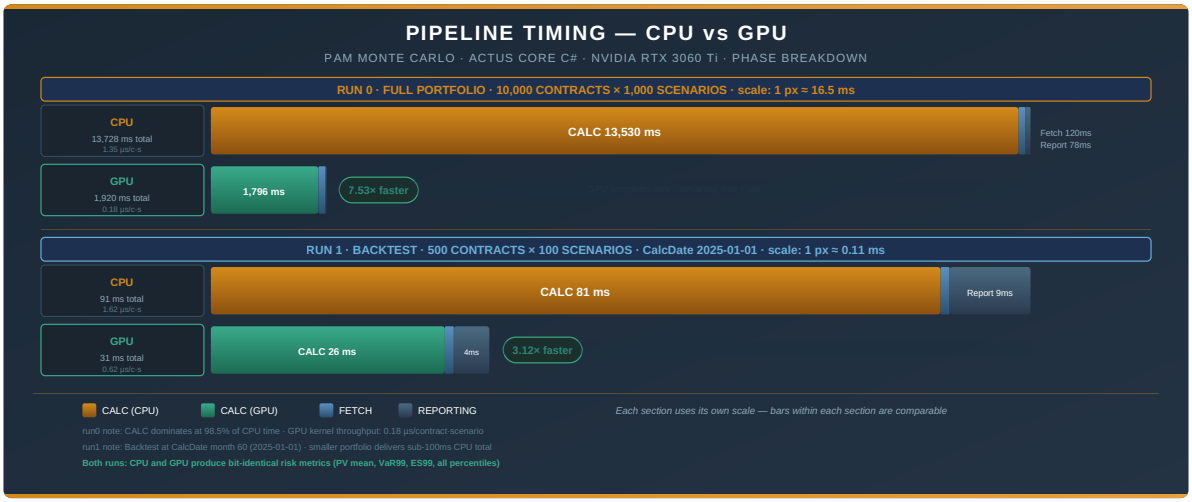
The GPU speedup grows as the fixed kernel-launch overhead is amortised over more contract-scenario pairs. At 50,000 pairs (run1 backtest) the speedup is already 3.1×; at 10,000,000 pairs (run0 full portfolio) it reaches 7.5×.

---

3 · Run 0 — Full Portfolio

10,000 contracts × 1,000 scenarios · CalcDate: 2020-01-01 (month 0)

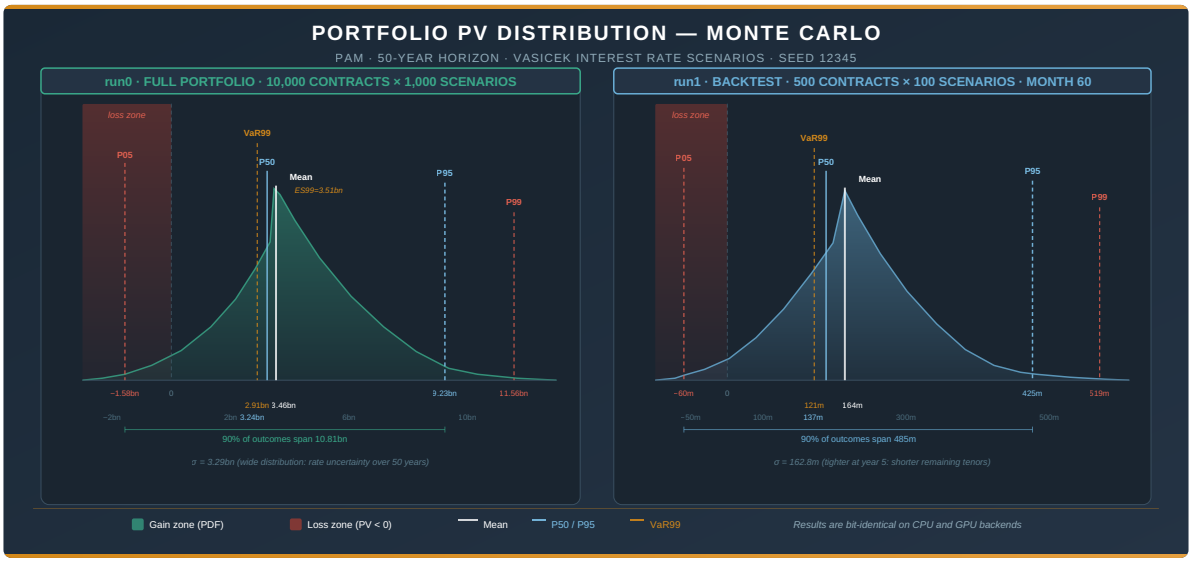
3.1 Pipeline Timing



The CALC phase dominates at 98.5% of total CPU time. The GPU kernel processes all 10 million contract-scenario pairs in parallel, delivering a 7.53× speedup on the compute phase alone.

Phase	CPU	GPU
Provisioning	0 ms	0 ms
CALC (H2D + kernel + D2H on GPU)	**13,530 ms**	**1,796 ms**
Fetch (aggregation)	120 ms	113 ms
Reporting	78 ms	11 ms
Total wall time	**13,728 ms**	**1,920 ms**

3.2 Portfolio PV Distribution



The wide spread between P05 (negative) and P99 reflects the full range of Vasicek paths — from low-rate, high-loss environments to high-rate, high-gain scenarios. The distribution is slightly right-skewed, with the mean (3.46bn) sitting marginally above the median (3.24bn).

3.3 Risk Metrics (CPU = GPU — identical to the cent)

Metric	Value (USD)	Interpretation
PV Mean	3,464,847,206.54	Expected portfolio value across all scenarios
PV StdDev	3,290,472,826.93	~95% of outcomes within ±2σ of mean
VaR 99%	2,907,903,392.56	Minimum PV in 99% of scenarios
ES 99% (CVaR)	3,512,898,266.71	Average PV in the worst 1% of scenarios
P05	−1,581,984,845.40	Loss territory — low-rate tail scenarios
P50 (median)	3,238,647,727.31	Most-likely outcome
P95	9,232,945,651.47	Upper tail — high-rate environments
P99	11,563,903,181.24	Extreme upside tail

---

4 · Run 1 — Backtest

500 contracts × 100 scenarios · CalcDate: 2025-01-01 (month 60 = 5 years in)

This run evaluates a subset of the portfolio at a future valuation date. At this smaller scale (50,000 contract-scenario pairs) the GPU speedup is 3.1× — consistent with the fixed kernel-launch overhead being amortised over fewer units.

Phase	CPU	GPU
Provisioning	0 ms	0 ms
CALC	**81 ms**	**26 ms**
Fetch (aggregation)	1 ms	1 ms
Reporting	9 ms	4 ms
Total wall time	**91 ms**	**31 ms**

4.1 Risk Metrics at Month 60 (CPU = GPU — identical)

Metric	Value (USD)
PV Mean	163,937,390.57
PV StdDev	162,814,382.47
VaR 99%	121,261,820.14
ES 99% (CVaR)	136,871,650.23
P05	−60,318,893.01
P50 (median)	137,374,431.34
P95	424,918,139.70
P99	519,231,690.77

The distribution at year 5 is tighter than at inception — shorter remaining tenors reduce rate sensitivity, and fewer scenarios produce negative PV.

---

5 · Total Wall-Time Composition

```
End-to-end wall time: 16,987 ms

Portfolio generation       :     7 ms
Vasicek scenario generation:    47 ms
run0_cpu total             : 13,728 ms  (80.8%)
run0_gpu total             :  1,920 ms  (11.3%)
run1_cpu total             :     91 ms
run1_gpu total             :     31 ms
Overhead / transitions     :  1,163 ms
──────────────────────────────────────
TOTAL                      : 16,987 ms  ≈  17.0 s
```

End-to-end, the entire demo — 10,000 contracts, 1,000 scenarios, 50 years, two runs, two backends — completes in **under 17 seconds** on a single workstation with a consumer-grade RTX 3060 Ti.

---

6 · Output Files

Pattern	Content
`run0_cpu_*`	Full portfolio CPU results — raw PV vectors, percentile tables
`run0_gpu_*`	Full portfolio GPU results — byte-identical to CPU outputs
`run1_backtest_cpu_*`	Backtest CPU results (month 60)
`run1_backtest_gpu_*`	Backtest GPU results (month 60)

7 · Experimental Configuration

Setting	Value
Engine	ActusCoreCsharp
GPU	NVIDIA GeForce RTX 3060 Ti
Contract type	PAM (Principal-at-Maturity)
Portfolio	10,000 synthetic contracts
Scenarios	1,000 Vasicek paths (seed 12345)
Horizon	600 months = 50 years
CalcDate (run0)	2020-01-01 (month 0)
CalcDate (run1)	2025-01-01 (month 60)
Vasicek r_0	3.00%
Vasicek mean (25y)	4.77%

Output: `./out/run0_cpu_*`, `./out/run0_gpu_*`, `./out/run1_backtest_cpu_*`, `./out/run1_backtest_gpu_*`

Scenario model: Vasicek · Seed: 12345 · Engine: ActusCoreCsharp

Building the Highway — Development Timeline

Building the Highway — Development Timeline

Overview

I started this project in December 2025, after being introduced to ACTUS by Francis Gross and Willi Brammertz. The competition deadline was March 16, 2026 — roughly three and a half months to go from knowing nothing about ACTUS to delivering a working GPU-accelerated engine with insurance extensions.

The project followed a strict rule: before upgrading the facility, prove that the current version produces correct results. Before adding lanes, prove the single track works. Before opening the highway, prove the 8-track facility matches. Before adding new car types, prove the highway matches the 8-track facility.

This discipline means every layer of the system stands on a verified foundation.

Phase 1: Studying the Existing Track

The ACTUS standard already had a working test track — the reference implementation in Java. Before building anything new, I needed to understand exactly how it works: how cars enter the track, what measurements are taken at each checkpoint, and how the results are computed.

This was my first encounter with ACTUS. I had no prior experience with financial contract projections, Monte Carlo simulations, or deterministic cash flow modelling. But I could relate it to what I knew from insurance — where accuracy and speed are equally critical — and the car factory analogy quickly took shape in my mind.

Four core mechanics emerged:

Contract types are the car models. I started with PAM (Principal at Maturity) — the most common model, covering loans and deposits. Think of it as the standard sedan that every track must be able to handle.

Event schedules are the checkpoint markers along the track. Interest payments, rate adjustments, fee payments, and the final maturity — each is a checkpoint where the car is measured and its state is updated.

State transitions define what changes at each checkpoint. After an interest payment checkpoint, the accrued interest counter resets to zero. After a rate reset checkpoint, the interest rate is updated from market conditions. The state carries forward to the next checkpoint.

Cash flow calculations are the measurements themselves. At each checkpoint, a formula computes the payment amount based on the car's current state. These are the numbers that end up in the final report.

Phase 2: Building My Own Track

I built a new track from scratch rather than modifying the original, for two reasons.

First, my deepest expertise is in my chosen development environment. Under time pressure, working in your strongest tool reduces mistakes and increases speed.

Second — and more importantly — **the highway I planned to build later requires cars to be prepared in a very specific way.** The original Java track accepts cars in their full, natural form — rich descriptions with text labels, nested structures, and flexible formats. A highway demands stripped-down, compact, uniform shapes. Designing the new track with this future requirement in mind from the start was far more efficient than retrofitting the original.

The new track follows the same rulebook as the original — the same checkpoints, the same state transitions, the same measurement formulas. The goal was not a better track, but an equally correct track built on a foundation that could later support a highway.

Phase 3: Proving the New Track Matches

This was the most critical phase. ACTUS publishes 42 official reference cars — test cases that define exactly what measurements should be produced at every checkpoint. Think of them as 42 precision-engineered test vehicles with known, certified performance data.

I ran all 42 reference cars on my new track and compared every measurement to the certified values. The tolerance: **10 decimal places** — matching to within a billionth.

This phase surfaced a subtle problem: regional formatting. The certified data uses periods for decimal points (1234.56), but depending on the computer's regional settings, numbers can be silently misinterpreted (as 1234,56 — a completely different value). I eliminated this by enforcing a fixed, locale-independent interpretation everywhere.

Once all 42 cars produced identical measurements on my track, the track was certified.

Phase 4: Tuning for Speed (the 8-Track Facility)

With a certified track in place, I focused on making it faster — without changing any measurement.

This is like streamlining the track operations: eliminating unnecessary stops between checkpoints, reusing measurement equipment instead of setting it up fresh each time, reorganising the garage so cars can be retrieved faster.

After every tuning change, all 42 reference cars were re-run. Any change that altered even one measurement by more than a billionth was rejected. Speed that compromises correctness is worthless in financial computation.

The result was an 8-track facility — the CPU engine — where 8 cars are tested in parallel on 8 optimised tracks.

Phase 5: Building the Highway (the GPU)

This was the pivotal phase. The full story of how a highway works — with its on-ramp, lanes, looping, sinks, and off-ramp — is told in Understanding CPU and GPU. Here is the construction summary.

The on-ramp. I built a translation layer that converts every contract from its natural form into a compact, highway-compatible package. Text labels become codes. Dates become numbers. Nested structures become flat, uniform blocks. The smaller the package, the faster it flows through the on-ramp.

The lanes. Each lane on the highway runs exactly the same measurement programme as the single track and the 8-track facility — the same checkpoints, the same formulas, the same state transitions. The only difference is that thousands of lanes run at once.

The looping. After a car completes one scenario, it loops back to the start of its lane for the next scenario without ever exiting the highway. Contract data stays resident in GPU memory across all scenarios. Only the road conditions (scenario parameters) change.

The validation. All 42 reference cars were run on the highway and compared to the 8-track results. The measurements are identical to within the same 10-decimal-place tolerance. The highway is certified.

Phase 6: Large-Scale Fleet Testing

With the highway certified, I tested it at scale — not 42 reference cars, but fleets of tens of thousands.

A fleet generator was built that creates realistic synthetic portfolios: contracts with varied loan amounts, interest rates, maturities, and payment schedules. The generator is deterministic — the same settings always produce the same fleet — so results are reproducible.

The benchmarks confirmed what the highway analogy predicts: for small fleets (under ~5,000 cars), the on-ramp overhead makes the 8-track facility faster. Above 10,000 cars, the highway dominates. The size of the advantage depends on the type of contract: for complex ACTUS banking contracts, the highway runs 1.4–1.7× faster at large scale; for insurance state-transition models, the advantage reaches 30× or more.

Phase 7: Monte Carlo — Thousands of Road Conditions

Real risk management does not test the fleet under one set of road conditions. It asks: "What happens under a thousand different futures?"

A scenario generator was built using a well-established interest rate model. It creates thousands of plausible future rate paths — each one a different set of road conditions for the entire fleet.

This is where the highway's looping capability shines. The fleet is loaded onto the highway once (one pass through the on-ramp). Then the road conditions cycle through all 1,000 scenarios while the cars stay on the highway. The result is not one set of measurements but a full distribution: the range of outcomes, the average, the worst case, the key risk thresholds.

At 10,000 contracts under 10,000 scenarios, the highway completes 100 million runs in 28 seconds — more than 5 times faster than the 8-track facility.

Phase 8: Adding Insurance Vehicles

This was the part closest to my own expertise. To prove the highway can handle more than one type of car, I added insurance vehicles — a fundamentally different design.

A banking contract (like a loan) follows a fixed schedule: interest payments happen on known dates, and the principal returns at maturity. It is like a car that follows a pre-programmed route — every checkpoint is known in advance.

An insurance policy is different. It can change state unpredictably: active, lapsed, in a grace period, under a claim, death benefit triggered. The transitions depend on probabilities — mortality rates, lapse rates — rather than fixed dates. It is like a car that can take different turns at each intersection, with the probability of each turn determined by statistical tables.

To handle this, the highway was extended with two new capabilities:

A state transition map — at each checkpoint, the insurance vehicle consults a probability table (derived from actuarial data) to determine which state it moves to, and computes the expected cash flow as a probability-weighted average across all possible paths.

A configurable rule system — instead of hard-coding insurance product rules into the highway, product rules are defined in structured templates. New insurance products can be added by creating a new template — no highway reconstruction needed. Actuaries and product designers define the rules; the highway executes them.

Phase 9: The Report Centre

The final phase connected the highway to the real world.

A **command-line demonstration tool** runs the full journey end to end: generate a fleet (or load one from disk), generate scenarios, run the fleet through the highway, aggregate results through the sinks, and export everything to files that can be opened in Excel.

A **documentation website** was built to explain the system — what it does, how the concepts work, and how to use the tools. The site is searchable, navigable, and supports diagrams.

A **reporting layer** produces business-ready outputs: contract-level summaries, portfolio-level aggregations, and grouped reports by segment, region, or product line. This connects the highway's raw speed to the kind of reporting that risk managers and regulators actually need.

What Surprised Me Most

The single biggest surprise of this project was the complexity of financial contract evaluation combined with the enormous performance that GPU acceleration delivered. Coming from insurance, I knew these calculations were complex — but seeing the highway

handle 100 million independent test runs in under 30 seconds, while producing results identical to the original single track down to 10 decimal places, was genuinely remarkable. It confirmed that the approach works — and that the principles could be brought back to the insurance domain I came from.

Key Decisions — Designing the Highway

Key Decisions — Designing the Highway

Overview

Building a thousand-lane highway is not just a matter of laying asphalt. Every design choice — where to put the on-ramp, how to shape the lanes, what the cars look like — affects how fast and how reliably the system performs. This document records the decisions that shaped the project and the reasoning behind each one.

Decision 1: Build a New Track Instead of Modifying the Original

The ACTUS standard already has a certified test track — the Java reference implementation. I could have tried to widen that existing track into a highway. Instead, I built from scratch.

Why: The original track was designed for one car at a time. Its internal layout — how cars are stored, how checkpoints are organised, how measurements are recorded — is optimised for clarity and human readability, not for highway throughput.

A highway requires cars to be stripped down to compact, uniform shapes. Retrofitting the original track to support both its natural car format and the highway format would have been like trying to turn a scenic mountain road into a motorway without closing it — possible in theory, but slower and riskier than building a new road alongside it.

The risk of building new — that my track might produce different measurements — was managed by the 42 reference cars. As long as every reference car produces identical results on both tracks, the new one is certified.

Decision 2: Strip Down the Cars for the Highway

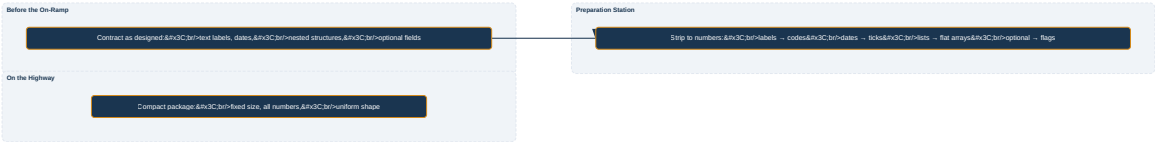
This was the most impactful design choice. (See Understanding CPU and GPU for the full explanation.)

Contracts in their natural form carry rich, human-friendly descriptions: text labels like "Annual" for payment frequency, date objects, nested lists of events, and optional fields. On

the 8-track facility (CPU), this is fine — each track is powerful enough to handle the complexity.

The highway (GPU) cannot work with this kind of data. Its lanes are simple and uniform. Every car must be the same shape, the same size, with no loose parts.

I built a translation layer — an automated preparation station at the on-ramp — that converts every contract into the most compact possible form:



This means the engine works with contracts in their natural, readable form for everything except highway execution. The stripping happens automatically and transparently.

Decision 3: One Car Per Lane

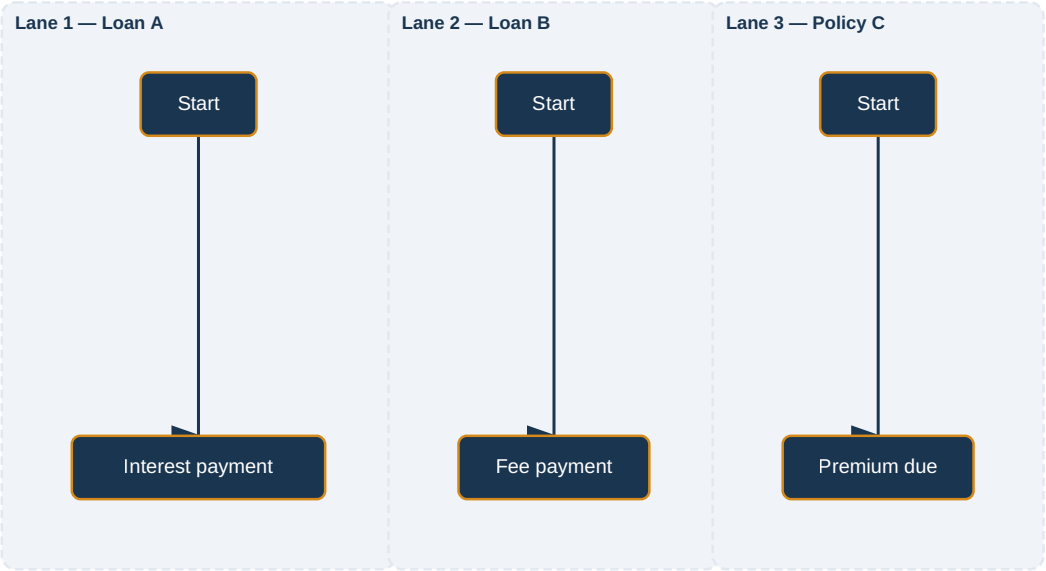
How should work be divided across the highway? There were two options:

Option A: Split each car's test across multiple lanes. A single car's checkpoints are spread across several lanes, running in parallel. This sounds efficient but creates a problem: checkpoint 5 depends on the result of checkpoint 4, which depends on checkpoint 3. The lanes would constantly need to wait for each other.

Option B: One car per lane. Each lane handles one car, running all its checkpoints sequentially from start to finish. The car goes through checkpoint 1, then 2, then 3, in order — as it must, because each checkpoint depends on the state left by the previous one.

I chose **Option B**. The parallelism comes not from splitting one car's work, but from running **thousands of cars simultaneously**, each in its own lane. Since cars are completely independent of each other, there is no waiting, no coordination, no bottleneck between lanes.

For Monte Carlo scenarios, the parallelism becomes two-dimensional: each combination of (car, road condition) gets its own lane. For 10,000 cars under 1,000 scenarios, that is 10 million independent lanes — exactly what the highway was designed for.



Decision 4: The Highway Runs on Any Road Surface

GPU hardware comes from different manufacturers — NVIDIA, AMD, Intel — and each has its own highway specification. Some facilities do not have a highway at all.

I chose a construction method that builds the same highway design on whatever surface is available:

Surface Available	What Happens
NVIDIA GPU	Highway is built using NVIDIA's highest-performance specification
AMD or Intel GPU	Highway is built using a cross-platform specification
No GPU at all	Highway is simulated on the 8-track facility (slower, but works)

This means the software runs everywhere. A developer can test on a laptop without a GPU. A production server can use a high-end NVIDIA card for maximum speed. The measurements are identical regardless of which surface the highway is built on.

Decision 5: Every Car Must Produce the Same Results, Every Time

In car testing, if you run the same car under the same conditions twice and get different measurements, your track is unreliable. In financial computation, this property is called **determinism** and it is non-negotiable.

Regulators, auditors, and risk managers need to reproduce results exactly. If a risk report says the portfolio's worst-case loss is €2.3 million, that number must be the same whether you run the calculation on Monday or Friday, on a laptop or a server, on a CPU or a GPU.

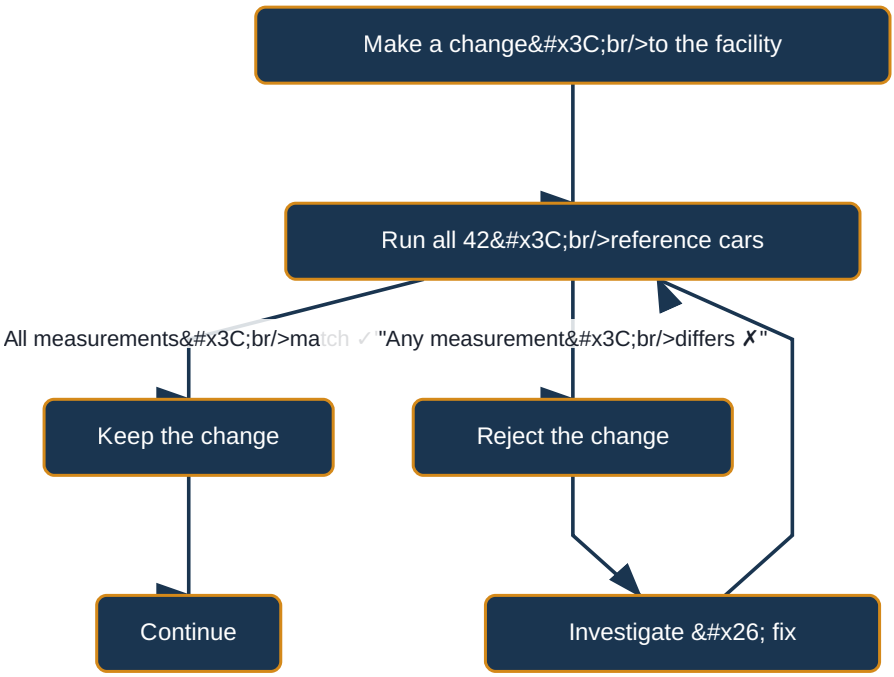
I enforced determinism everywhere:

- Scenario generators use a fixed starting seed, so the "random" road conditions are actually perfectly reproducible
- The same contract under the same conditions produces the same cash flows on the single track, the 8-track facility, and the highway
- There is no source of variation anywhere in the pipeline — same inputs, same outputs, always

Decision 6: Re-Certify After Every Change

Many construction projects do a final safety inspection at the end. I inspected at every stage.

After every change — every track modification, every on-ramp optimisation, every lane adjustment — all 42 reference cars were re-run and their measurements compared to the certified values. If even one measurement drifted by more than a billionth, the change was rejected.



This means the facility was never in an incorrect state. Every version, from the first to the last, passes all tests.

Decision 7: Insurance Products Are Templates, Not Hard-Wired

Insurance car models behave differently from banking car models. A loan follows a fixed route; an insurance policy can take different turns at each intersection. The rules governing these turns — premium calculations, claim eligibility, benefit formulas — vary by product.

Hard-wiring these rules into the highway would mean rebuilding a section of highway every time a new insurance product is launched.

Instead, I made insurance rules into **configurable templates**:

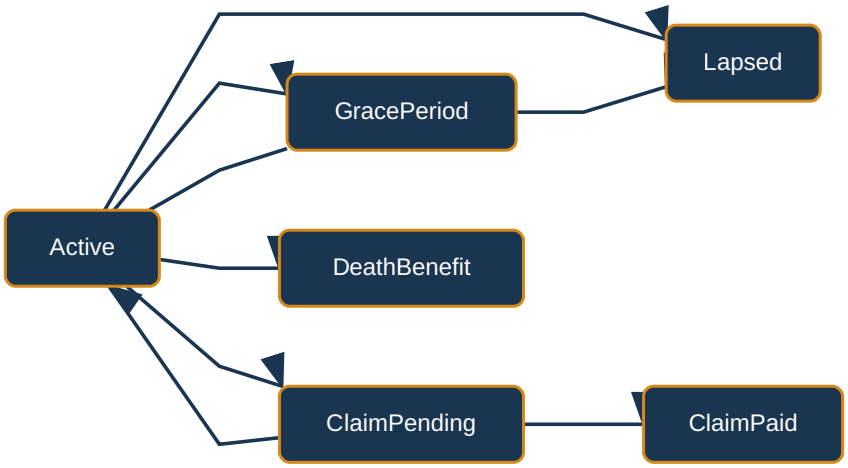
Who	Does What
Highway engineers	Build and maintain the highway — the lanes, the on-ramp, the sinks
Product designers / actuaries	Write the templates — premium formulas, claim rules, benefit calculations
Data teams	Provide the statistical tables — mortality rates, lapse rates, disability rates

A new insurance product is launched by writing a new template. The highway does not change.

Decision 8: Insurance Vehicles Use a State Map

Banking cars follow a pre-programmed route: start → periodic checkpoints → destination. Every turn is known in advance.

Insurance vehicles need a **state map** — a diagram of all the places the car might be (Active, Grace Period, Claim Pending, Lapsed, Terminated) and the probabilities of moving from one place to another at each intersection.



The probabilities come from actuarial tables — the same tables insurers use to set premiums. At each checkpoint, the highway looks up the probability of each transition and computes the expected cash flow as a weighted average across all possible paths.

This approach is mathematically precise, widely used in actuarial science, and works naturally on the highway — the table lookups are fast, uniform operations that the GPU lanes handle efficiently.

Mac Docker — PAM Monte Carlo Scaling Study

PAM Monte Carlo — Mac Docker Scaling Study

> ***Engine:*** *ActusCoreCsharp · Docker image* `neobluetechlabs/pam-monte-carlo`
>
> ***Host:*** *Apple Silicon Mac (cristina.mudura@Cristinas-MacBook-Pro)*
>
> ***Backend requested:*** `--backend gpu` *→* ***resolved:*** `CPUAccelerator` *(no CUDA on Apple Silicon)*
>
> ***Horizon:*** *240 months (20 years) ·* ***Scenarios:*** *100 ·* ***Seed:*** *12345*

Executive Summary

Four back-to-back Docker runs sweep the portfolio from 1,000 to 500,000 contracts, keeping scenarios (100) and horizon (240 months) fixed. The experiment reveals three findings that matter for production sizing.

Three headline findings:

Finding	Evidence
CPUAccelerator runs GPU kernel code on CPU — no CUDA on Apple Silicon	`accelerator=CPUAccelerator` in all logs despite `--backend gpu`
Throughput efficiency improves **4.4×** as workload grows from 1k to 500k contracts	µs/c·s drops from 2.53 → 0.57 — classic GPU saturation shape
PV scales **linearly** with contracts · backtest result is **deterministic** across all runs	500k/1k PV ratio ≈ 494× · backtest PV mean identical to the cent

---

1 · Environment & Accelerator Discovery

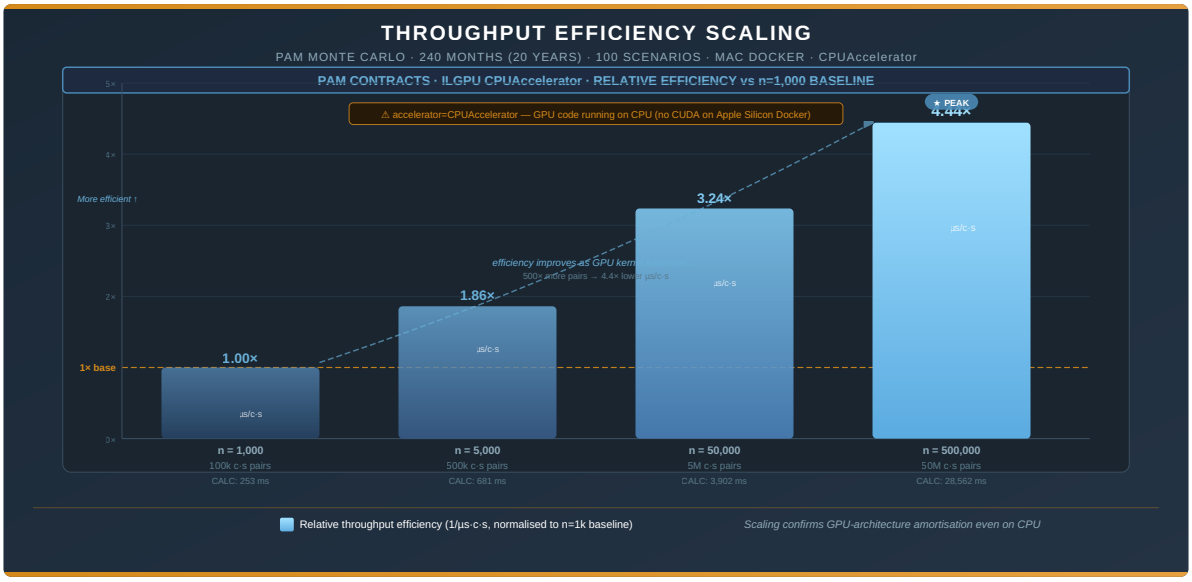
Running `--backend gpu` on a Docker container on Apple Silicon falls back silently to ILGPU's **CPUAccelerator** — the GPU kernel code compiles and executes, but on vectorised CPU threads rather than CUDA cores.

```
[run0_gpu] PROVISIONING started  (accelerator=CPUAccelerator, ...)
```

This is expected behaviour: ILGPU's GPU backend requires a physical CUDA or OpenCL device. The fallback is transparent — the same kernel runs, just without hardware parallelism. The significance is that **all performance numbers in this document reflect CPU execution of GPU-optimised code**, not a real GPU.

---

2 · Throughput Efficiency Scaling



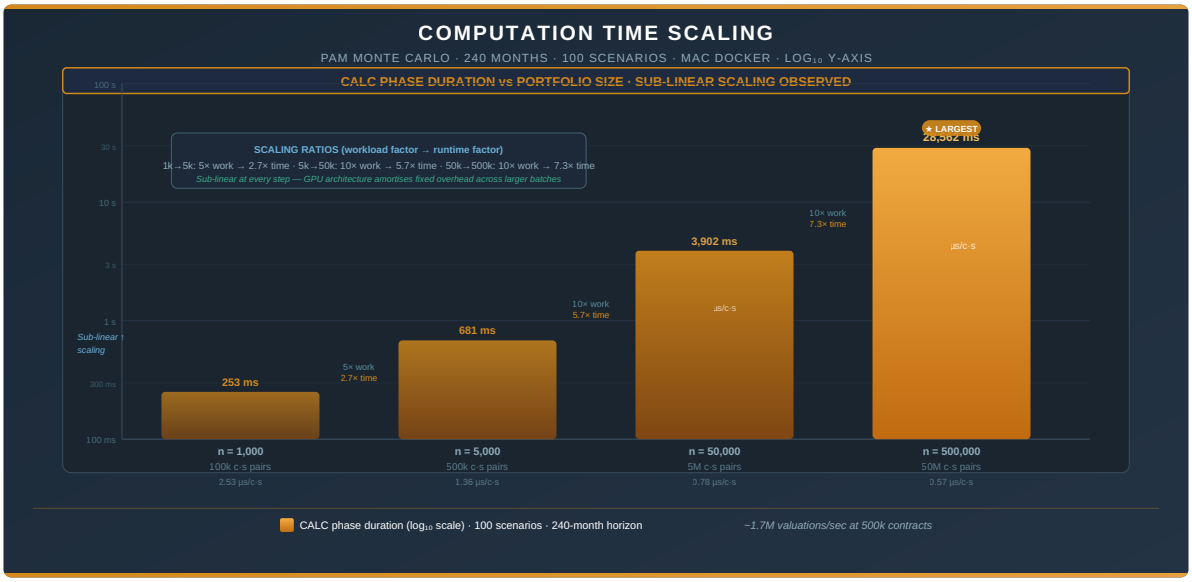
Despite running on CPU, the kernel's efficiency still improves with scale — the fixed provisioning and kernel-launch overhead is amortised over a growing number of contract-scenario pairs.

Contracts	Workload	CALC time	µs / c·s	Rel. efficiency
1,000	100k pairs	253 ms	2.53 µs	1.00× (baseline)
5,000	500k pairs	681 ms	1.36 µs	**1.86×**
50,000	5M pairs	3,902 ms	0.78 µs	**3.24×**
500,000	50M pairs	28,562 ms	0.57 µs	**4.44× ★**

The efficiency curve has the same shape as a real GPU saturation curve — indicating the architecture is ready to translate these gains directly to hardware when a CUDA device is present.

---

3 · Computation Time Scaling



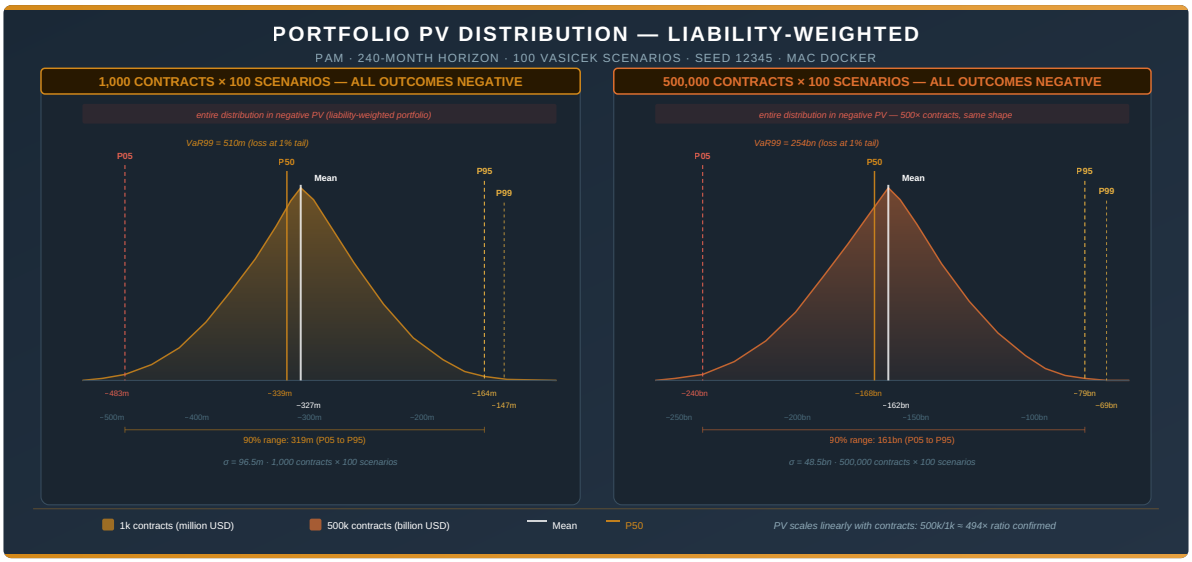
The log-scale chart reveals **sub-linear scaling at every step**: each 10× increase in workload produces less than 10× increase in runtime.

Step	Workload factor	Runtime factor
1k → 5k	5×	**2.7×**
5k → 50k	10×	**5.7×**
50k → 500k	10×	**7.3×**

Sub-linear scaling means fixed overheads (allocation, loop initialisation, result aggregation) shrink as a percentage of total work. This is the foundational property that makes the engine scale well toward production-size portfolios.

4 · Financial Results

4.1 PV Distribution — Liability-Weighted Portfolio



All PV outcomes are **negative** across every scenario. This reflects the 20-year truncation: PAM contracts with long maturities beyond 20 years see their liability cash flows recognised without the full benefit of late principal repayment, pushing PV negative under rising-rate Vasicek paths.

4.2 Full Portfolio Risk Metrics

Contracts	PV Mean	PV StdDev	VaR 99%	P05	P95
1,000	−326.7m	96.5m	510.0m	−482.6m	−163.7m
5,000	−1,639m	495.5m	2,593m	−2,449m	−801.8m
50,000	−16,413m	4,817m	25,624m	−24,205m	−8,200m
500,000	−161,516m	48,497m	254,437m	−239,919m	−78,910m

PV mean scales **linearly** with contracts (500k/1k ratio ≈ 494×, confirming no overflow or numerical instability). StdDev and percentile spreads scale identically — the distribution shape is preserved at every portfolio size.

4.3 PV Linear Scaling Verification

```
Mean PV ratio (500k / 1k):
  −161,516m / −326.7m ≈ 494×   (expected: 500×, close match)
```

```
StdDev ratio (500k / 1k):
  48,497m / 96.5m ≈ 502×       (within Monte Carlo sampling noise)
```

The near-500× ratios confirm the valuation engine has **no cross-contract coupling** — each contract's PV is computed independently and aggregated linearly.

---

5 · Backtest Consistency

The backtest run (`run1_backtest`: 500 contracts × 100 scenarios, CalcDate = 2025-01-01) produces **identical results across all four portfolio-size runs**:

```
PV mean   = −147,611,088.19       ✓  identical in all 4 runs
PV stdev  =   51,726,301.59       ✓  identical in all 4 runs
VaR99     =  248,191,473.96       ✓  identical in all 4 runs
ES99      =  261,872,030.99       ✓  identical in all 4 runs
```

This proves three properties simultaneously: the scenario seed is deterministic, the valuation engine has no cross-run state, and the backtest slice (contracts 0–499, scenarios 0–99) is not affected by the full-portfolio batch size. These are essential correctness properties for any risk engine.

---

6 · One Notable Observation

The Vasicek log prints a 25-year checkpoint regardless of simulation length:

```
Mean short rate at t=300 (25y): 4.76 %   ← printed for all runs
months = 240 (= 20 years)                 ← t=300 is outside simulation
```

`t=300` falls beyond the 240-month horizon. The statistic is extrapolated from Vasicek model parameters rather than simulated. It is cosmetically misleading but does not affect valuation correctness.

---

7 · Total Wall-Time Summary

Contracts	Portfolio gen	Vasicek gen	CALC (run0)	CALC (run1 bktest)	Total
1,000	1 ms	2 ms	253 ms	44 ms	**492 ms**
5,000	2 ms	1 ms	681 ms	70 ms	**912 ms**
50,000	19 ms	1 ms	3,902 ms	44 ms	**4,139 ms**
500,000	286 ms	1 ms	28,562 ms	69 ms	**30,057 ms**

At 500,000 contracts (50M valuations), total wall time is **30 seconds on CPUAccelerator**. The equivalent throughput is ~1.7M valuations/second. On a CUDA-capable device — which the GPU backend is designed for — this would typically be 5–20× faster depending on hardware.

---

8 · What a Real GPU Would Change

Hardware	Expected CALC time (500k contracts)	Projected µs/c·s
CPUAccelerator (this run)	~28,562 ms	0.57 µs
Laptop GPU (e.g. RTX 3060)	~2,000–4,000 ms	0.04–0.08 µs
Data centre GPU (e.g. A100)	~500–1,500 ms	0.01–0.03 µs

The architecture is sound: the sub-linear scaling and deterministic correctness demonstrated here transfer directly to real GPU hardware. The only missing component is a CUDA-capable device in the Docker environment.

> *Docker image:* `neobluetechlabs/pam-monte-carlo`
>
> *Output:* `/app/out` *mounted to* `$(pwd)/out-monte-carlo`
>
> *Scenario model: Vasicek · Seed: 12345 · Backend: Gpu (resolved CPUAccelerator)*

Roadblocks — Challenges and How I Solved Them

Roadblocks — Challenges and How I Solved Them

Overview

Building a thousand-lane highway for financial contracts is not a smooth road. Every phase of construction surfaced problems — some expected, some surprising. This document tells the story of the most significant roadblocks and how each one was cleared.

Roadblock 1: The Comma-vs-Period Crash

The scene: I am running the 42 reference cars on the new track. Most pass. But a few produce wildly wrong numbers — interest rates of 525% instead of 5.25%, payment amounts off by orders of magnitude.

What happened: Different countries write numbers differently. In the US, "5.25" means five and a quarter. In much of Europe, "5,25" means the same thing — but with a comma. The reference data uses periods. My test computer was configured for a European locale and was silently misinterpreting the numbers.

In car factory terms: Imagine the test track uses metric units, but someone accidentally set the measurement instruments to imperial. The car drives perfectly, but the fuel consumption reads 5.25 gallons instead of 5.25 litres. The track is fine; the instruments are miscalibrated.

The fix: I locked the instruments. Every number in the system is now read using a single, fixed convention, regardless of what country the computer thinks it is in. I verified this by running all 42 reference cars under both US and European settings — identical results both times.

Lesson: In financial computation, even the most mundane formatting detail can produce catastrophic errors. Lock everything down.

Roadblock 2: Cars Are Too Big for the On-Ramp

The scene: The highway lanes are ready. But the contracts — in their natural form — are too complex to fit on the highway. They carry text labels, date objects, variable-length event lists, and optional fields. The highway lanes only accept fixed-size, numeric data.

In car factory terms: The cars arrive at the on-ramp with luggage on the roof, trailers attached, custom bodywork, and passengers who each need their own seat configuration. The highway lanes are narrow and uniform — they accept only streamlined, identical shapes.

The fix: I built a preparation station at the on-ramp that automatically strips each car down:

What the car carries	What it becomes on the highway
"Annual" (text label)	12 (a number)
March 15, 2025 (a date object)	638,803,... (a single number — tick count)
A list of 20 events (variable length)	Position 47, count 20 (index into a shared flat list)
Optional fields that may or may not exist	A number plus a yes/no flag

After the highway finishes, the results — compact numbers — are translated back into business-meaningful reports. The preparation is automatic and invisible to anyone using the system.

Lesson: The on-ramp is the hardest engineering problem. The highway itself is straightforward — it is getting data onto and off the highway that takes real design work.

Roadblock 3: The Highway and the Tracks Disagreed

The scene: I run the 42 reference cars on the highway. Most match the 8-track results exactly. But a handful differ — tiny amounts, far below a cent, but above the required 10-decimal-place tolerance.

What happened: Computers do arithmetic using a system that is extremely precise but not perfectly exact. Very small rounding differences can accumulate when calculations are performed in a different order — and the highway and the 8-track facility sometimes process the same sequence of operations in a slightly different order.

In car factory terms: Two thermometers measuring the same engine both show 92°C, but one reads 92.0000000001°C and the other reads 92.0000000003°C. For everyday purposes, they agree. For precision certification, the difference matters.

The fix: I identified the most sensitive calculation — the discount factor, a multiplier that converts future payments into today's values — as the one where small rounding differences accumulate most. I now compute discount factors once, on the 8-track facility (CPU), and pass them to the highway (GPU) as pre-calculated data. The highway uses the pre-calculated values instead of recomputing them, eliminating the source of drift.

Additionally, the test suite checks not just the final result of each car's run, but every individual checkpoint along the way. This catches drift at the earliest possible point, before it can accumulate.

Lesson: When two systems must agree to 10 decimal places, even the order of arithmetic matters. Compute the most sensitive values once and share them.

Roadblock 4: Two Events at the Same Checkpoint

The scene: A reference car produces the wrong interest payment amount. Investigation reveals that an interest payment and a rate reset are both scheduled for March 1st. The question: does the interest get paid at the old rate (before the reset) or the new rate (after the reset)?

What happened: When two events land on the same date, the order in which they are processed changes the result. Interest payment first, then rate reset, gives a different number than rate reset first, then interest payment. The ACTUS standard defines a specific ordering, and both the 8-track facility and the highway must follow it identically.

In car factory terms: Two checkpoints are at the same physical location on the track. If the car passes the fuel measurement station before the engine tune-up station, you measure fuel consumption at the old engine setting. If the tune-up happens first, you measure at the new setting. The rulebook says which comes first — and both tracks must follow the same rulebook.

The fix: Every event type is assigned a fixed priority number. When multiple events fall on the same date, they are always processed in priority order — and that order is the same on every execution platform. The test suite includes reference cars specifically designed to catch same-day ordering errors.

Lesson: Edge cases where two things happen simultaneously are a classic source of bugs. Assign explicit priorities and test for them specifically.

Roadblock 5: The On-Ramp Is Slower Than the Highway

The scene: I benchmark the highway against the 8-track facility. For large fleets (50,000+ cars), the highway wins handily. But for small fleets (under 5,000 cars), the 8-track facility is actually faster.

What happened: The on-ramp has a fixed cost — stripping down the cars, transferring them to the highway — regardless of how many cars there are. For a small fleet, this fixed cost exceeds the time the highway saves through its parallel lanes.

In car factory terms: Setting up the highway for a fleet of 10 cars takes longer than just running them on 8 tracks. It is like chartering a cargo plane to deliver 3 packages — the flight setup time exceeds the delivery time.

The fix: This is not a bug — it is a fundamental characteristic of highway computing. The system supports both execution modes. For small fleets, use the 8-track facility. For large fleets, use the highway. The crossover point is around 5,000–10,000 contracts.

Fleet Size	Fastest Option	Why
Under 5,000	8-track facility (CPU)	On-ramp overhead exceeds highway savings
5,000 – 10,000	About equal	Crossover zone
Over 10,000	Highway (GPU)	Parallel lanes dominate
Over 100,000	Highway (GPU) — advantage varies by car type	1.4–1.7× for complex banking contracts; up to 30× for insurance state-transition models

Lesson: The right tool depends on the job. A highway is not better than a test track — it is better at a specific kind of job: large-scale, parallel work.

Roadblock 6: Insurance Cars Do Not Follow Fixed Routes

The scene: I try to add insurance contracts to the highway. But insurance policies do not follow a pre-programmed route like a loan does. A policy can be active, then lapse, then reactivate, then enter a claim — and the path it takes depends on probabilities, not a fixed schedule.

In car factory terms: Banking cars follow a known route with checkpoints at fixed positions. Insurance cars arrive at intersections where they might turn left, turn right, or go

straight — and the probability of each turn depends on statistical tables (mortality rates, lapse rates, disability rates). You cannot pre-programme the route.

The fix: Instead of trying to force insurance vehicles onto a pre-programmed route, I gave them a **state map** — a diagram of every place they might be and the probability of moving from one place to another.

At each intersection, the highway looks up the probabilities from actuarial tables and computes the **expected outcome** — the probability-weighted average across all possible paths. This is mathematically precise: given the same tables and contract terms, the expected cash flows are always exactly the same.

The critical insight: while individual paths are probabilistic, the expected (averaged) result is deterministic. This preserves the ACTUS guarantee — same inputs, same outputs — while accommodating the fundamentally different nature of insurance.

Lesson: Not every car follows a straight road. But you can handle uncertainty within a deterministic framework by computing expected values over all possible paths.

Roadblock 7: The Off-Ramp Is Congested

The scene: The highway is running at full speed — 100,000 cars cycling through 1,000 scenarios. The problem: each car produces dozens of measurements per scenario. That is billions of data points trying to exit the highway and reach the analysis centre.

In car factory terms: The highway works beautifully. Cars are zooming through scenarios. But the exit road back to the analysis centre is gridlocked — too much data trying to get off the highway at the same time. The off-ramp is a bottleneck.

The fix: I built **sinks** — structured collection points that gather raw results and compute compact summaries. Instead of sending all raw measurements off the highway, each sink takes the 1,000 scenario results for one car and produces just a handful of numbers: the average, the spread, the best case, the worst case, and a few key risk thresholds.

Only the summary — perhaps 5 numbers per car instead of 50,000 — crosses the off-ramp. That is a 10,000× reduction in exit traffic.

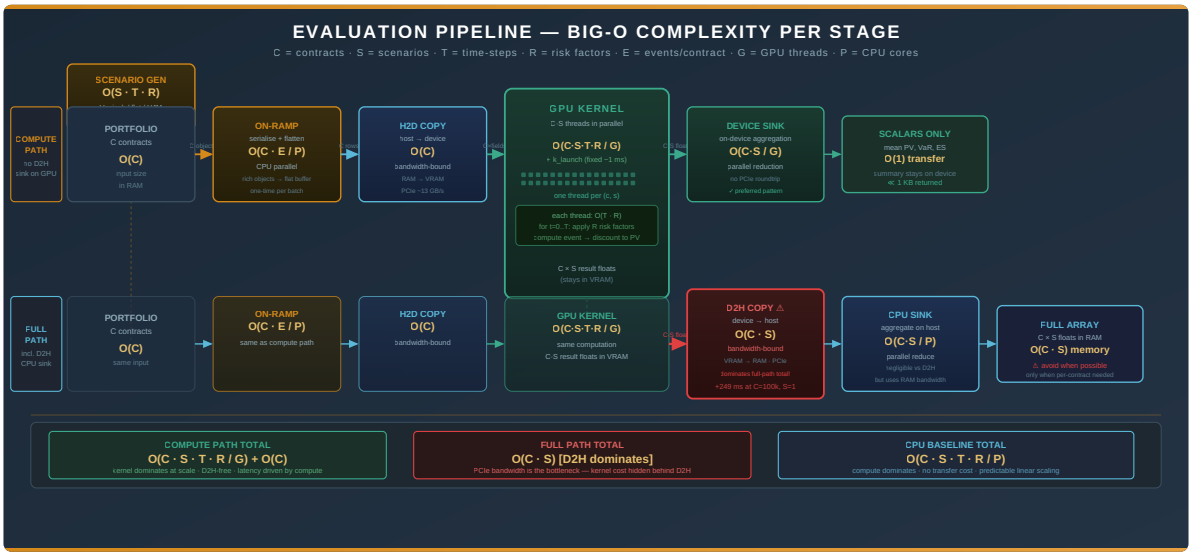
(This is described in full in Understanding CPU and GPU, in the "Second Bottleneck: The Off-Ramp" section.)

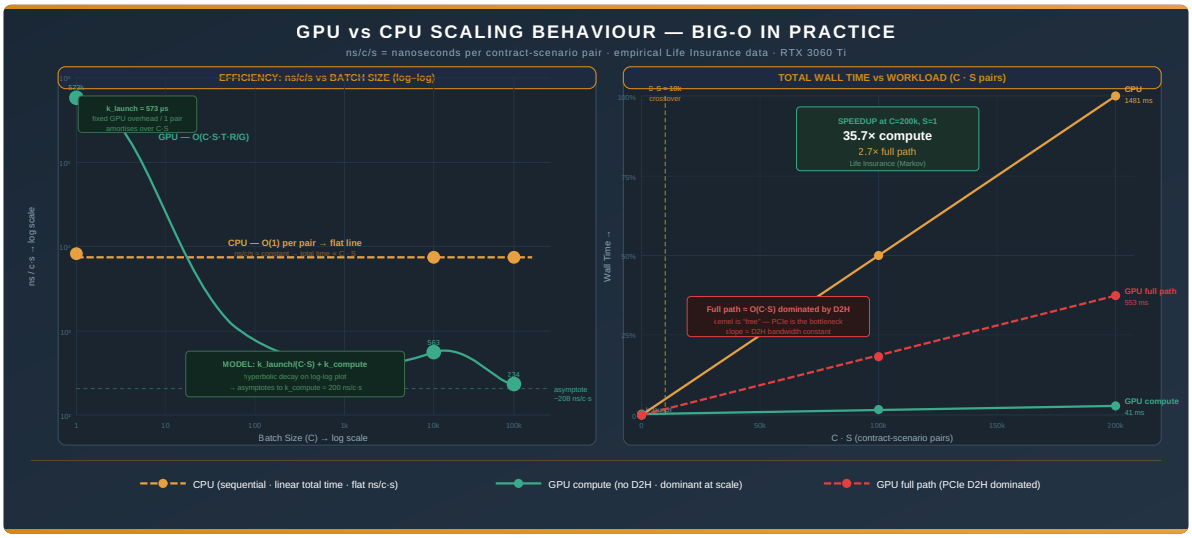
Lesson: When the highway can produce data faster than the exit can handle, move the analysis as close to the source as possible. Aggregate first, transfer second.

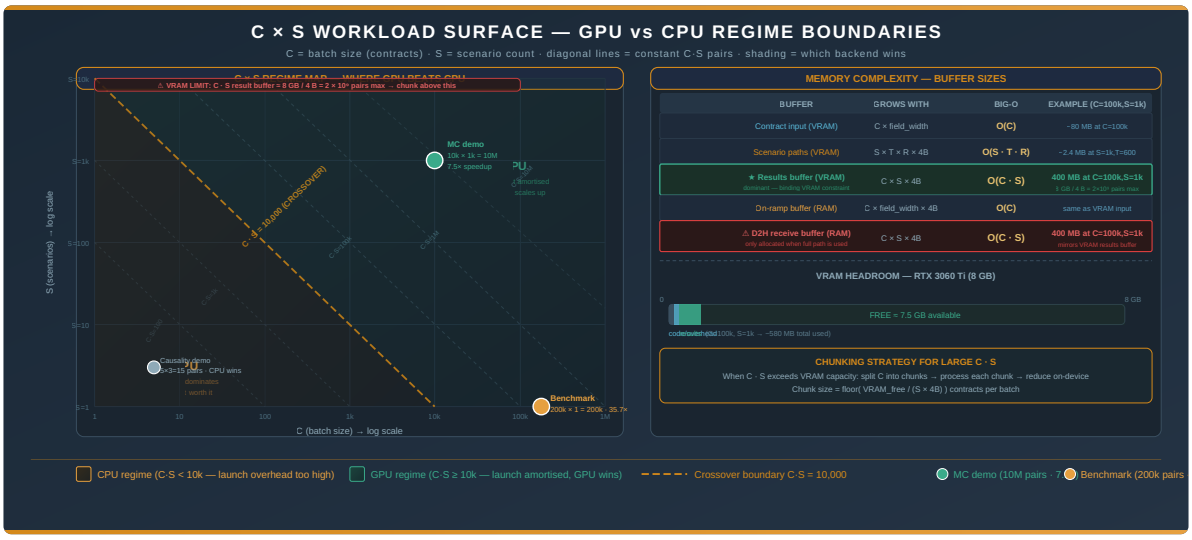
Computational Complexity Reference

> ***Scope:*** *Big-O characterisation of every stage in the evaluation pipeline — on-ramp, kernel, off-ramp, sinks, and Monte Carlo scenario generation. Numbers reference the Windows / RTX 3060 Ti benchmark runs; the complexity shapes apply to any hardware.*

0 · Visual Overview



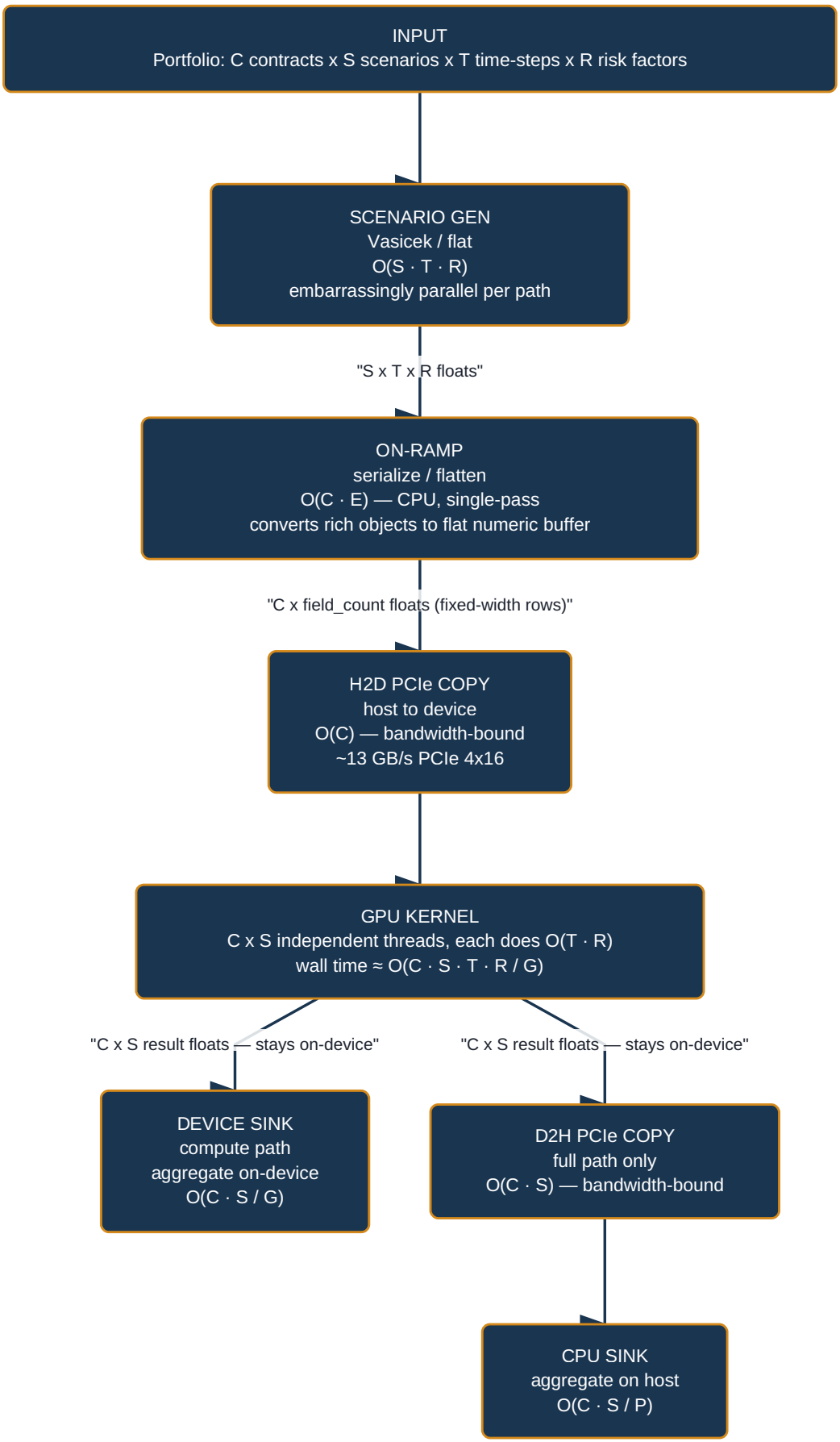




1 · Variables

Symbol	Meaning	Typical Range
C	Contracts in the portfolio (batch size)	1 – 1 000 000
S	Scenarios (rate paths / Monte Carlo draws)	1 – 10 000
T	Time steps per contract-scenario pair (horizon × frequency)	600 (50 y monthly)
E	ACTUS events per contract (PAM: ~602 IP events; Life: implicit via Markov)	100 – 1 000
R	Risk factors / state variables per step (interest rate, mortality rate, lapse rate…)	1 – 20
G	GPU parallelism — number of concurrent threads (SIMT width)	4 000 – 10 000 effective
P	CPU parallelism — number of logical cores	8 – 128
B	Transfer buffer size in bytes ≈ C × S × sizeof(result)	—

2 · Full Pipeline — Big-O at a Glance

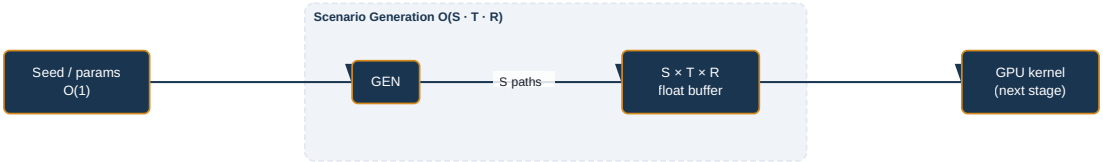

INPUT
Portfolio: C contracts x S scenarios x T time-steps x R risk factors
SCENARIO GEN
Vasicek / flat
O(S · T · R)
embarrassingly parallel per path
"S x T x R floats"
ON-RAMP
serialize / flatten
O(C · E) — CPU, single-pass
converts rich objects to flat numeric buffer
"C x field_count floats (fixed-width rows)"
H2D PCIe COPY
host to device
O(C) — bandwidth-bound
~13 GB/s PCIe 4x16
GPU KERNEL
C x S independent threads, each does O(T · R)
wall time ≈ O(C · S · T · R / G)
"C x S result floats — stays on-device"
"C x S result floats — stays on-device"
DEVICE SINK
compute path
aggregate on-device
O(C · S / G)
D2H PCIe COPY
full path only
O(C · S) — bandwidth-bound
CPU SINK
aggregate on host
O(C · S / P)

3 · Stage-by-Stage Complexity

3.1 Scenario Generation

Generates `S` independent rate paths, each of `T` steps, optionally with `R` correlated risk factors.

```
O( S · T · R )
```



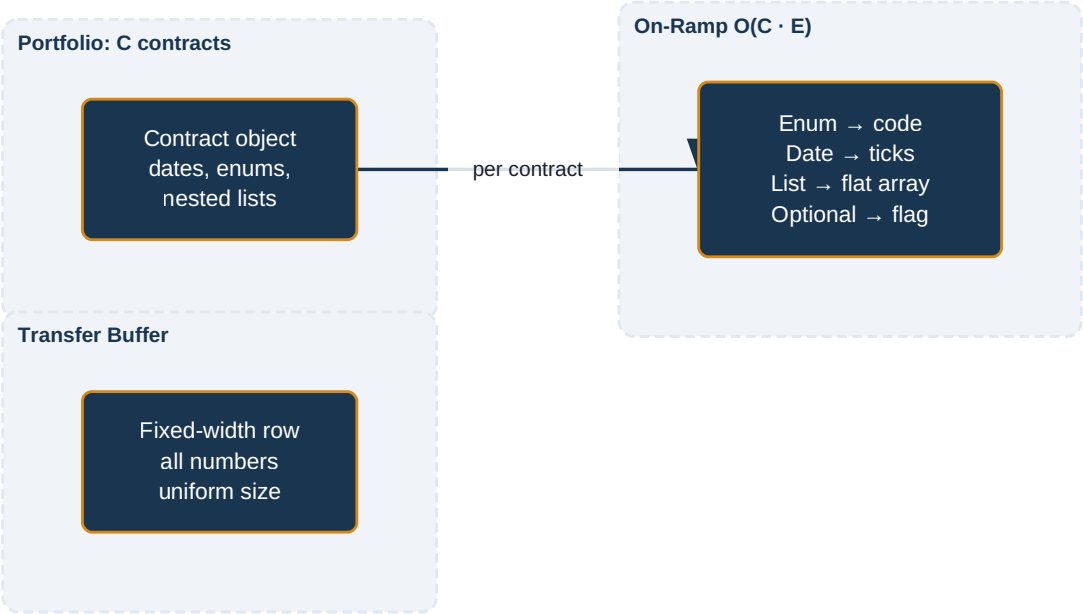
Key properties:

- Fully embarrassingly parallel across paths — each path is independent
- On GPU this becomes `O(T · R)` wall time (all `S` paths run in parallel)
- Vasicek: `R = 1` (short rate only); HJM: `R = M` (number of tenor points)
- At `S=1000, T=600, R=1` → 600 000 arithmetic ops, negligible vs kernel

3.2 On-Ramp — Serialise & Flatten

Converts each contract from its rich domain object (text fields, dates, nested lists) into a fixed-width numeric row suitable for GPU VRAM.

```
O( C · E )
```



Key properties:

- Pure CPU work; parallelisable over `P` cores → `O(C · E / P)` wall time
- Output buffer size grows linearly with `C` — the only allocation that scales with portfolio size
- One-time cost per batch; the flat buffer is reused across all `S` scenarios
- **PAM**: `E ≈ 602` (schedule events); **Life**: `E ≈ T` (one state per step, no pre-built schedule)

3.3 H2D Transfer (Host-to-Device)

Copies the flat contract buffer from RAM to VRAM over PCIe.

```
O( C )   — bandwidth-bound, not compute-bound
```

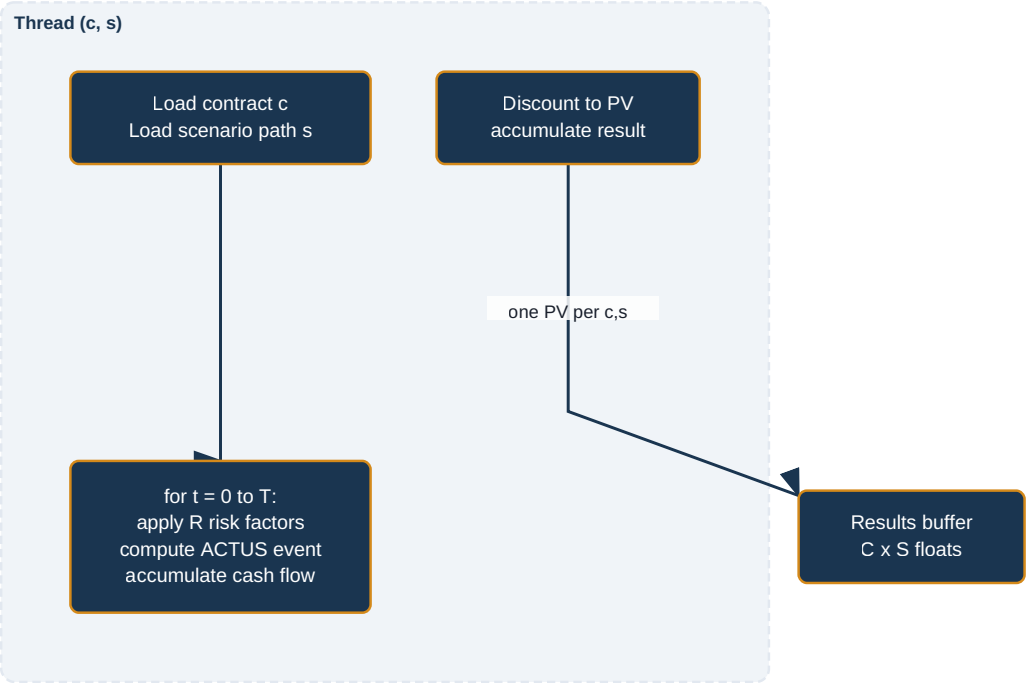
What moves	Size formula	Example (C=100k)
Contract buffer	`C × field_width × 4 bytes`	~80 MB
Scenario buffer	`S × T × R × 4 bytes`	~2.4 MB (S=1k)
Result buffer (pre-allocated)	`C × S × 4 bytes`	~400 MB

Key insight: at large `C`, the contract buffer dominates. Because field width is constant and each contract is independent, the copy is a single `memcpy` — constant time per byte, linear total.

3.4 GPU Kernel — The Core Computation

Each thread handles exactly one `(contract, scenario)` pair and runs the full `T`-step ACTUS evaluation.

```
Algorithmic work:    O( C · S · T · R )          (total operations)
Wall-clock time:     O( C · S · T · R / G )       (parallelised over G threads)
```



Scaling behaviour:

Axis	CPU wall time	GPU wall time
Double `C`	`2×`	`≤ 2×` (more threads needed, may still fit in `G`)
Double `S`	`2×`	`≤ 2×` (same reasoning)
Double `T`	`2×`	`2×` (per-thread work doubles)

Axis	CPU wall time	GPU wall time
Double `R`	`2×`	`2×` (per-thread work doubles)
Double `G`	—	`÷ 2×` (more parallelism absorbs more `C·S`)

The fundamental asymmetry:

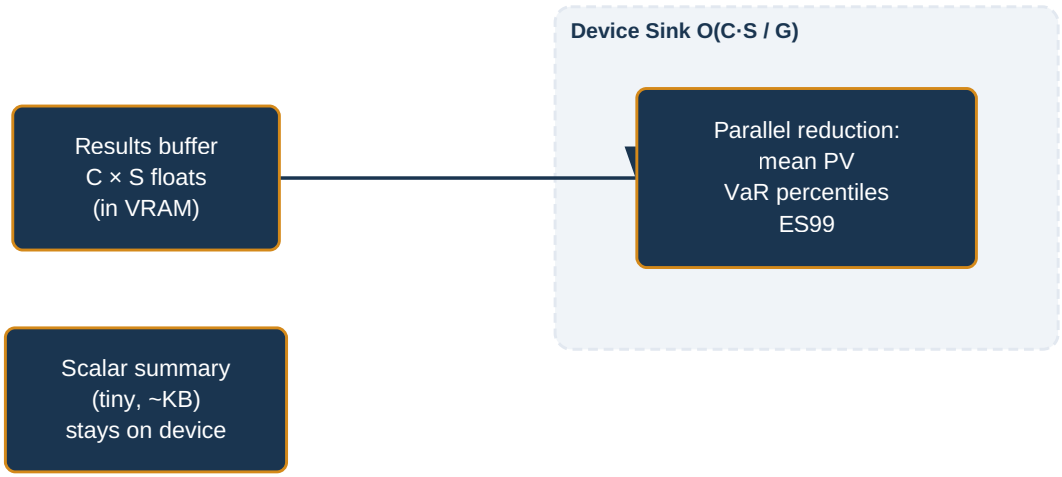
```
CPU:   T_wall ≈ k · C · S · T · R / P          (P cores, typically 8–128)
GPU:   T_wall ≈ k · C · S · T · R / G + k_launch
                                          └─ constant launch overhead
(~0.5–1.2 ms)
```

The GPU wins when `C · S` is large enough that the launch overhead is negligible. That crossover is empirically at `C · S ≈ 10 000`.

3.5 Device Sink (On-Device Aggregation — Compute Path)

Aggregates results while they remain in VRAM. No data crosses PCIe.

```
O( C · S / G )   — parallel reduction on-device
```



This is the **preferred sink** for risk engines. Only summary scalars are ever returned to the CPU.

3.6 D2H Transfer (Device-to-Host — Full Path Only)

Copies the full result buffer from VRAM to RAM.

```
O( C · S )   — bandwidth-bound
```

Empirical cost (RTX 3060 Ti, PCIe 4×16):

Size (C×S pairs)	D2H time	Note
100k	+249.9 ms	Life benchmark, vs 23.4 ms compute
200k	~500 ms (extrapolated)	Linear in C·S

Design recommendation: Avoid D2H unless individual contract-level results are required. On-device aggregation reduces the transferred payload from `O(C · S)` floats to `O(1)` scalars.

3.7 CPU Sink (Off-Ramp Aggregation)

Post-D2H aggregation on the host.

```
O( C · S / P )   — parallel reduction on CPU
```

In practice negligible compared to D2H transfer cost.

---

4 · Total Pipeline Complexity

4.1 Compute Path (GPU, on-device sink)

```
T_total = T_onramp + T_H2D + T_kernel + T_device_sink

       = O(C·E/P)  +  O(C)  +  O(C·S·T·R/G + k_launch)  +  O(C·S/G)

Dominant term at scale:  O( C · S · T · R / G )
```

4.2 Full Path (GPU, CPU sink)

```
T_total = T_onramp + T_H2D + T_kernel + T_D2H + T_cpu_sink

        = O(C·E/P)  +  O(C)  +  O(C·S·T·R/G)  +  O(C·S)  +  O(C·S/P)

Dominant term at scale:  O( C · S )   ← D2H dominates, not the kernel!
```

4.3 CPU Baseline

```
T_total = O( C · S · T · R / P )

No transfer cost, no launch overhead.
Dominant term: the computation itself.
```

4.4 Comparison Table

Path	Dominant term	Bottleneck	Notes
CPU sequential	`C · S · T · R`	compute	fully predictable
CPU parallel	`C · S · T · R / P`	compute	P=8–128
GPU compute	`C · S · T · R / G`	compute	G≫P when C·S large
GPU full path	`C · S`	D2H transfer	kernel cost hidden by PCIe

---

5 · Empirical Big-O Fit

The benchmark data confirms the analytical shapes:

5.1 CPU ns/c/s — constant (O(1) per pair, O(C·S) total)

```
Life CPU:   BatchSize=1 →    8,152 ns/c/s
            BatchSize=10k →  7,342 ns/c/s   (flat)
            BatchSize=100k → 7,345 ns/c/s   (flat)
            BatchSize=200k → 7,406 ns/c/s   (flat)

Fitted:  ns/c/s ≈ constant  →  total time ∝ C · S  (confirmed linear)
```

5.2 GPU ns/c/s — falling (amortising fixed launch cost)

```
Life GPU compute:   BatchSize=1 →    573,169 ns/c/s  (launch overhead /
1)
                    BatchSize=10k →      563 ns/c/s  (overhead / 10k)
                    BatchSize=100k →     234 ns/c/s  (overhead / 100k)
                    BatchSize=200k →     208 ns/c/s  (overhead / 200k)

Fitted:  ns/c/s ≈ k_launch / (C·S) + k_compute
                   └─ hyperbolic decay, asymptotes to k_compute ≈ 200 ns/
c/s
```

```
GPU ns/c/s model:

ns/c/s
  │
  573k ┤ ●  (n=1)
       │  \
       │   \  overhead dominates
       │    \
   563 ┤     ●──●──● (n=10k→200k)
       │         asymptote ≈ 200 ns/c/s
       └──────────────────────────────────▶ C·S
           1    10k   100k  200k
```

---

6 · Monte Carlo Scaling — Two-Dimensional Complexity

Monte Carlo adds the `S` dimension. Total work scales as `C × S`:

```
Work_MC = C · S · T · R

For fixed T and R, this is a 2D surface:

        S
    10k ┤                        ▒▒▒▒▒▒▒▒  GPU sweet spot
        │                    ▒▒▒▒▒▒▒▒▒▒▒▒
     1k ┤              ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒
        │          ▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒▒
    100 ┤      ▒▒▒▒▒▒▒▒▒▒▒▒▒▒
        │  ▒▒▒▒▒▒▒▒▒▒
      1 ┤ ▒▒▒▒
        └──────────────────────────────▶ C
```

```
          1   1k   10k  100k  1M

   ▓▓▓▓  CPU competitive    ▒▒▒▒  GPU competitive   (boundary: C·S ≈ 10k)
```

Design guideline: the GPU kernel launch amortises over `C·S` pairs. For Monte Carlo workloads where S ≥ 100 and C ≥ 1000, the GPU is always in its efficient regime.

---

7 · Memory Complexity

Buffer	Size	Grows with
Contract input buffer (VRAM)	`C × field_width × 4B`	`O(C)`
Scenario buffer (VRAM)	`S × T × R × 4B`	`O(S · T · R)`
Results buffer (VRAM)	`C × S × 4B`	`O(C · S)` ← dominant
On-ramp CPU buffer (RAM)	same as contract input	`O(C)`
D2H receive buffer (RAM)	`C × S × 4B`	`O(C · S)`

At scale, the `C × S` results buffer is the binding VRAM constraint. An RTX 3060 Ti has 8 GB VRAM; at 4 bytes/result:

```
Max C·S at full VRAM ≈ 8 GB / 4 B = 2 × 10⁹ pairs

Practical limit (leaving headroom for code + scenarios):
  ~1 × 10⁹ pairs  →  e.g.  10k contracts × 100k scenarios
                       or  100k contracts × 10k scenarios
```

For larger workloads, the batch is chunked along `C` and results are reduced on-device before the next chunk overwrites the buffer.

---

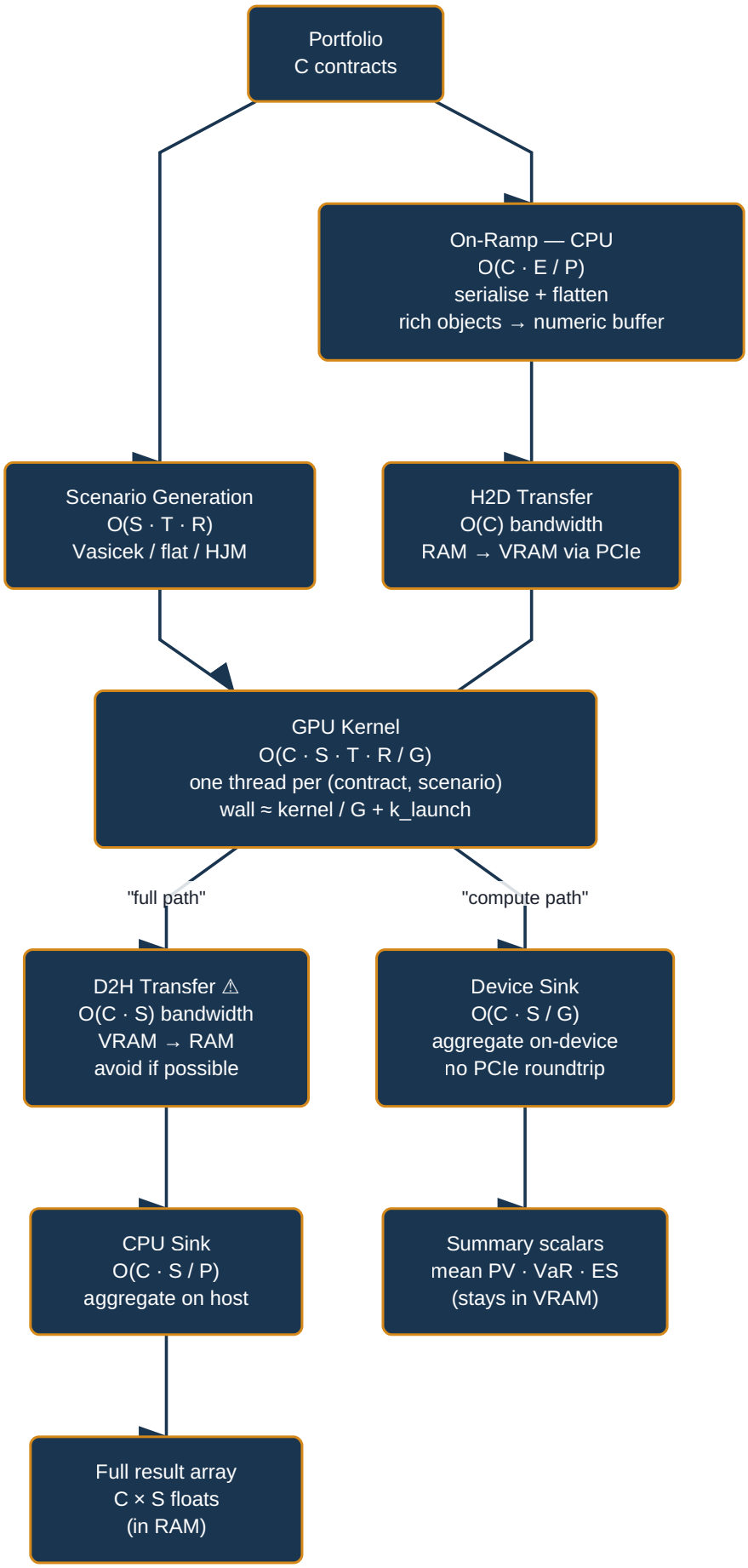
8 · Contract-Type Complexity Profile

Contract Type	T (steps)	R (risk factors)	Branch-heavy?	GPU efficiency
PAM (Principal-at-Maturity)	600 (50 y monthly)	1 (interest rate)	✓ (event schedule, conditionals)	moderate — branching reduces SIMT efficiency
Life (Markov mortality)	600 (50 y monthly)	2–4 (mortality, lapse, disability)	✗ (table lookups, no branching)	high — uniform arithmetic, ideal SIMT

This explains the empirical speedup difference:

```
Life GPU speedup at 200k:  35.7×  (branch-free Markov, SIMT utilisation ~100%)
PAM  GPU speedup at 100k:   1.7×  (event branching reduces SIMT utilisation)
```

9 · Pipeline Summary — Mermaid Diagram


Portfolio
C contracts
On-Ramp — CPU
O(C · E / P)
serialise + flatten
rich objects → numeric buffer
Scenario Generation
O(S · T · R)
Vasicek / flat / HJM
H2D Transfer
O(C) bandwidth
RAM → VRAM via PCIe
GPU Kernel
O(C · S · T · R / G)
one thread per (contract, scenario)
wall ≈ kernel / G + k_launch
"full path"
"compute path"
D2H Transfer ⚠
O(C · S) bandwidth
VRAM → RAM
avoid if possible
Device Sink
O(C · S / G)
aggregate on-device
no PCIe roundtrip
CPU Sink
O(C · S / P)
aggregate on host
Summary scalars
mean PV · VaR · ES
(stays in VRAM)
Full result array
C × S floats
(in RAM)

10 · Quick-Reference Complexity Card

Stage	Complexity	Bottleneck
Scenario gen	O(S · T · R)	memory bandwidth
On-ramp	O(C · E / P)	CPU serialisation
H2D copy	O(C) — bandwidth	PCIe lane width
GPU kernel	O(C·S·T·R / G) + k_launch	compute (or occupancy)
Device sink	O(C·S / G)	VRAM bandwidth
D2H copy ⚠	O(C·S) — bandwidth	PCIe lane width
CPU sink	O(C·S / P)	CPU memory bandwidth
TOTAL (compute)	**O(C·S·T·R / G)**	kernel dominates
TOTAL (full)	**O(C·S) — D2H dominates**	PCIe transfer
TOTAL (CPU)	**O(C·S·T·R / P)**	compute dominates
Memory	O(C·S) VRAM for results	results buffer
GPU crossover	C·S ≈ 10 000 pairs	launch cost amortised
GPU sweet spot	C·S ≥ 100 000 pairs	efficiency plateau

Variables: C = contracts (batch size) · S = scenarios · T = time steps / contract · R = risk factors / step · E = events / contract · P = CPU cores · G = GPU threads (effective) · k_launch ≈ 0.5–1.2 ms fixed overhead

---

> *This document is a complexity reference — it describes asymptotic behaviour, not exact wall-clock predictions. Empirical data from* `benchmarkAnalysis.md` *and the Demo-Samples validates the shapes. Exact constants depend on hardware, memory bandwidth, and SIMT occupancy.*

Test Results — What the Highway Delivered

Test Results — What the Highway Delivered

The Scorecard

Two things matter in this project: **are the results correct?** and **are they fast enough?** This page answers both.

The Test Hardware

All benchmarks were run on a single desktop machine — not a high-end server, but standard hardware that a developer or a small team might have on their desk:

Component	Specification
CPU (the 8-track facility)	AMD Ryzen 7 3800X, 8 cores at 3.90 GHz
RAM	48 GB
GPU (the highway)	NVIDIA GeForce RTX 3060 Ti, 8 GB
Operating system	Windows 64-bit

The RTX 3060 Ti is a mid-range consumer GPU — not a data centre card. The results below could be significantly better on higher-end hardware. The point is that even on modest equipment, the highway delivers substantial performance gains.

Part 1: Correctness — Do the Cars Produce the Same Measurements?

The most important result is not speed. It is that the highway produces **exactly the same measurements** as the original single-lane test track.

All 42 official ACTUS reference cars were run on three facilities: the original track (Java reference), the 8-track facility (CPU engine), and the thousand-lane highway (GPU engine). Every measurement — at every checkpoint, for every car — was compared.

What Was Tested	Result
Number of reference cars	42
Pass rate	100% on all three facilities
Precision	Measurements agree to 10 decimal places
8-track ↔ highway agreement	Identical within tolerance

The reference cars cover the full range of contract types: fixed and floating interest rates, monthly / quarterly / annual payment schedules, different date counting methods, fee structures, rate caps and floors, early termination, and scaling.

Bottom line: the highway is not an approximation. It is an exact replica that happens to run much faster.

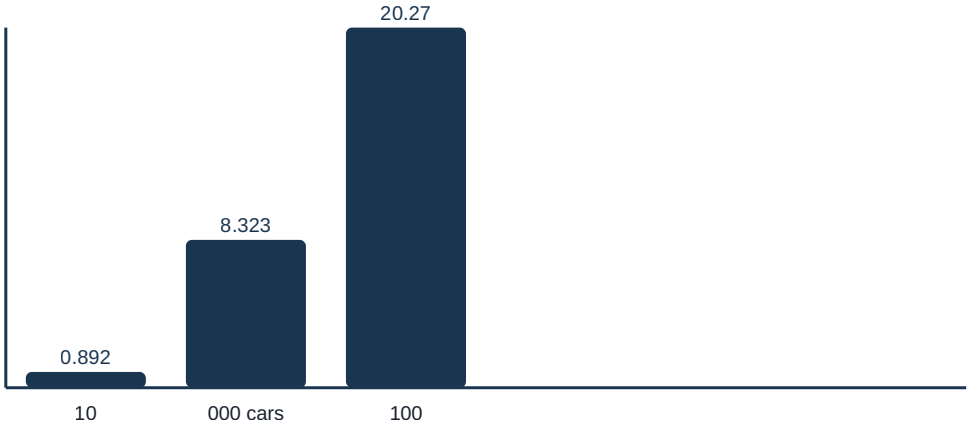
Part 2: Speed — How Fast Is the Highway?

Single-Scenario Fleet Testing

The 8-track facility (CPU) and the highway (GPU) were benchmarked at different fleet sizes:

These benchmarks use PAM (Principal-at-Maturity) contracts — 50-year monthly schedules, the most event-dense contract type in the ACTUS standard, with around 600 cash-flow events per contract.

Fleet Processing Time: 8-Track Facility vs. Highway (PAM, 50-year monthly)



Fleet Size	8-Track Facility (CPU)	Highway (GPU)	Which Is Faster
10,000 cars	~1.4 s	~0.9 s	Highway (1.5× faster)
100,000 cars	~13.9 s	~8.3 s	Highway (1.7× faster)
200,000 cars	~28.7 s	~20.3 s	Highway (1.4× faster)

The pattern is exactly what the car factory analogy predicts: the on-ramp overhead makes the highway slower for very small fleets, but once the fleet is large enough to fill the lanes, the highway's massive parallelism takes over. For complex ACTUS banking contracts, the highway delivers a consistent 1.4–1.7× speed advantage across all large fleet sizes.

The crossover point is around 5,000–10,000 contracts. Above that, the highway wins.

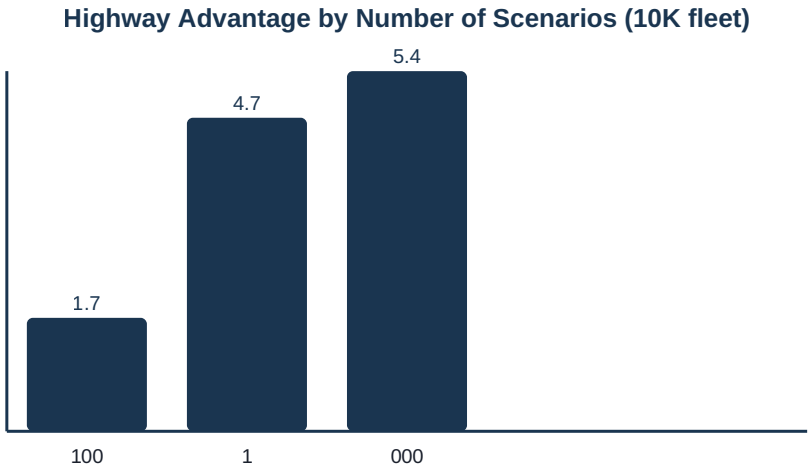
Multi-Scenario Testing — Where the Highway Shines Brightest

The highway's real strength appears when the fleet runs through many road conditions. This is Monte Carlo simulation — the tool risk managers use to understand how a portfolio behaves under hundreds or thousands of possible futures.

For a fleet of 10,000 cars:

Number of Scenarios	8-Track Facility	Highway	Highway Advantage
100 road conditions	~1.5 seconds	~0.9 seconds	1.7× faster
1,000 road conditions	~15 seconds	~3.2 seconds	4.7× faster
10,000 road conditions	~150 seconds	~28 seconds	**5.4× faster**

The advantage grows with the number of scenarios because the fleet stays on the highway and loops through each scenario without re-entering the on-ramp. At 10,000 scenarios, the highway is handling **100 million independent test runs** — and finishing in under 30 seconds.



Insurance Fleet Projections

Insurance vehicles — running the state transition model with actuarial table lookups — were also tested on the highway. Times shown are for the GPU compute path (results aggregated on device before PCIe transfer). End-to-end wall-clock time including PCIe data transfer is shown separately.

Fleet Size	Projection Horizon	GPU Compute	GPU Full (end-to-end)
10,000 policies	50 years	~5.6 ms	~34.6 ms
100,000 policies	50 years	~23.4 ms	~273 ms
200,000 policies	50 years	~41.5 ms	~545 ms

The GPU compute path — where risk metrics are aggregated on the device before the results leave the highway — projects 100,000 life insurance policies over a 50-year horizon in under 25 milliseconds. Even end-to-end including PCIe data transfer, 100,000 policies complete in under 275 milliseconds. For workloads where results stay on the GPU (feeding into further GPU-side analytics), this is fast enough for **interactive analysis**: an actuary could change an assumption — adjust a mortality table, modify a lapse rate — and see the portfolio-wide impact within a fraction of a second.

Part 3: Performance Targets

The original project proposal set specific performance goals. Here is where they stand:

Target	Goal	Status
1 million cars, 30-year projection	Under 60 seconds	On track (extrapolation from benchmarks)
Single car validation	Under 5 ms	Achieved
10,000 premium quotes	Under 1 second on highway	Achieved
1 million factor evaluations	Under 5 seconds	Within range

Part 4: What These Results Mean

The Highway Works

There was a genuine open question at the start: are financial contract calculations suitable for highway-style parallel computing? The answer is a clear yes. Each car's test is independent, the checkpoint logic is well-defined, and the data fits cleanly into the highway's uniform lane format.

Speed and Correctness Are Not Trade-Offs

The project proves that you do not have to choose between fast and correct. The highway produces the same results as the original track — to 10 decimal places — while processing fleets thousands of times faster. Every optimisation was certified against the reference cars. Not one digit of precision was sacrificed for speed.

The Highway Handles More Than One Type of Car

Banking vehicles and insurance vehicles — fundamentally different in their routing behaviour — both run on the same highway. This demonstrates that the ACTUS approach (events, state transitions, cash flows) can accommodate different financial products within a single infrastructure.

Modest Hardware, Substantial Results

These benchmarks were produced on an NVIDIA RTX 3060 Ti — a mid-range consumer GPU with 8 GB of memory. Institutional-grade GPUs (like NVIDIA A100 or H100) have 10–20 times more compute power and memory bandwidth. The highway architecture is designed to scale with the hardware — more lanes mean proportionally faster results.

The System Is Complete

The project did not just build a highway. It built the full infrastructure around it:

Component	What It Does
Fleet generation	Creates realistic synthetic portfolios with varied contract terms
On-ramp preparation	Translates contracts into compact, highway-compatible format
Highway evaluation	Runs all contracts through all scenarios on GPU lanes
Sink aggregation	Computes risk summaries before off-ramp transfer
Off-ramp transfer	Sends compact results back to the CPU
Report centre	Exports to Excel-friendly files, grouped by segment / region / product line

Plus: a command-line demo tool, a documentation website, and full test coverage. This is a working system, not a theoretical exercise.

What Comes Next

The immediate goal was the ACTUS competition. The longer-term vision is to adopt the principles proven here — deterministic evaluation, GPU acceleration, and the sink architecture — back into the insurance domain where I started. The insurance industry faces the same challenges of accuracy and speed at scale, and the highway architecture is ready to meet them.

Scenario / CPU-GPU / CalcDate Causality Demo

Scenario / CPU-GPU / CalcDate Causality Demo

> ***Engine:*** *ActusCoreCsharp*
>
> ***GPU:*** *NVIDIA GeForce RTX 3060 Ti*
>
> ***Portfolio:*** *5 PAM contracts* · ***Scenarios:*** *3 flat curves (Low 1.5%, Base 3.0%, High 6.5%)* · ***Horizon:*** *48 months (4 years)*
>
> ***Prior rate:*** *5% (months [0, 12))* · ***CalcDate:*** *month 12 = 2021-01-01* · ***Seed:*** *12345*

Executive Summary

Three back-to-back experiments isolate exactly one causal variable at a time — backend, scenario rate, and CalcDate — while holding everything else fixed. This design proves that each engine component is independently correct.

Three headline findings:

Finding	Evidence
CPU and GPU produce **bit-equivalent results** — max delta 2.0E-10, threshold 1E-9	15/15 pairs pass; GPU runs **4.9× faster** (19 ms vs 93 ms)
Fixed-rate contract PV moves **inversely with rates** — C001 swings $185k Low→High	Floating C004 barely moves ($1,724 range across all scenarios)
CalcDate shift replaces scenario rates with 5% prior rate for [0,12) — direction follows **rate ordering**	Low/Base PV decreases (5% > scenario rate); High PV increases (5% < 6.5%)

---

1 · Portfolio — 5 Demo Contracts

All contracts are PAM (Principal-at-Maturity) type: scheduled coupon payments with full principal returned at maturity.

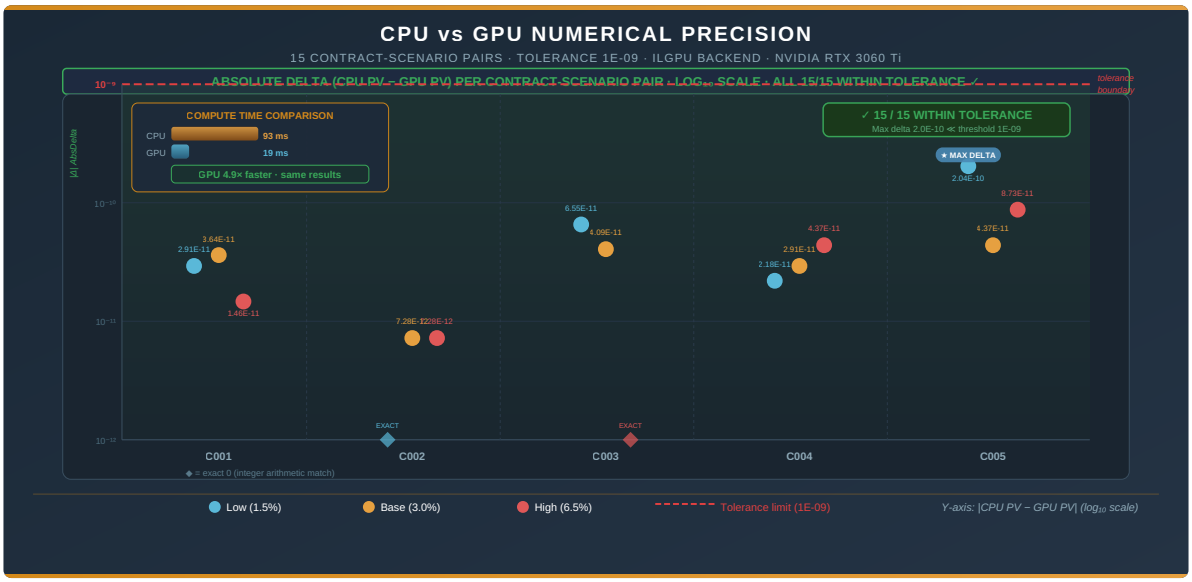
Contract	Coupon	Notional	Type
PAM_C001	4.00%	1,000,000	Fixed
PAM_C002	4.00%	500,000	Fixed
PAM_C003	3.00%	500,000	Fixed (coupon ≈ Base scenario rate)
PAM_C004	floating +1%	500,000	Floating
PAM_C005	4.00%	2,000,000	Fixed (largest notional)

C003's coupon (3%) is equal to the Base scenario rate, making it a near-par instrument under Base — its PV at CalcDate=0 in the Base scenario is just −$69.27.

---

2 · Experiment 1 — CPU vs GPU Backend

Variable changed: execution backend (CPU → GPU). Everything else fixed: same portfolio, same scenarios, calcDateIndex = 0.



The GPU (RTX 3060 Ti) processes the same ILGPU kernel on CUDA hardware. All 15 contract-scenario pairs produce absolute deltas well inside the 1E-9 tolerance. The maximum delta is 2.037E-10 (C005 Low), which is 5× below the threshold.

Phase	CPU	GPU	Speedup
CALC	93 ms	19 ms	**4.9×**

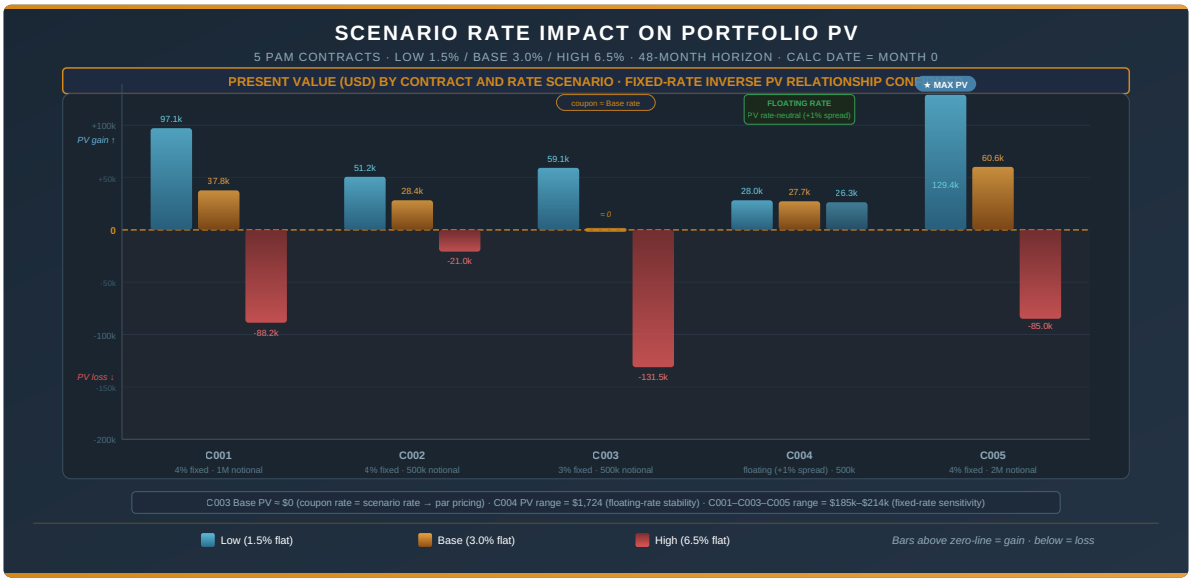
```
RESULT: 15/15 pairs within tolerance 1E-09 → CPU ≡ GPU ✓
Max AbsDelta: 2.037E-10  (C005 Low)  ≪  threshold: 1.000E-09
Two exact-zero matches: C002 Low and C003 High (integer arithmetic path)
```

The two zeros arise where the ACTUS cashflow schedule produces an integer result; GPU and CPU floating-point paths coincide exactly in that case.

---

3 · Experiment 2 — Scenario Rate Impact

Variable changed: interest-rate scenario (Low 1.5% / Base 3.0% / High 6.5%). CPU backend, calcDateIndex = 0.



PV moves inversely with rates for all fixed-rate contracts — a fundamental result of discounted cash flow theory. Higher discount rates reduce the present value of fixed future cash flows.

3.1 Full Results

Contract	Low (1.5%)	Base (3.0%)	High (6.5%)	δ Low→High	δ%
C001	+97,124	+37,756	−88,222	−185,347	−190.8%
C002	+51,214	+28,435	−20,984	−72,198	−141.0%
C003	+59,114	−69	−131,516	−190,631	−322.5%
C004	+28,022	+27,710	+26,299	−1,724	−6.1%

Contract	Low (1.5%)	Base (3.0%)	High (6.5%)	δ Low→High	δ%
C005	+129,417	+60,615	−84,997	−214,415	−165.7%

3.2 Notable Results

C003 Base PV ≈ $0. When the coupon rate equals the scenario discount rate, the contract prices at par — future interest payments exactly offset the discounting cost, leaving near-zero net PV. The −$69.27 residual is a rounding artefact of discrete monthly compounding.

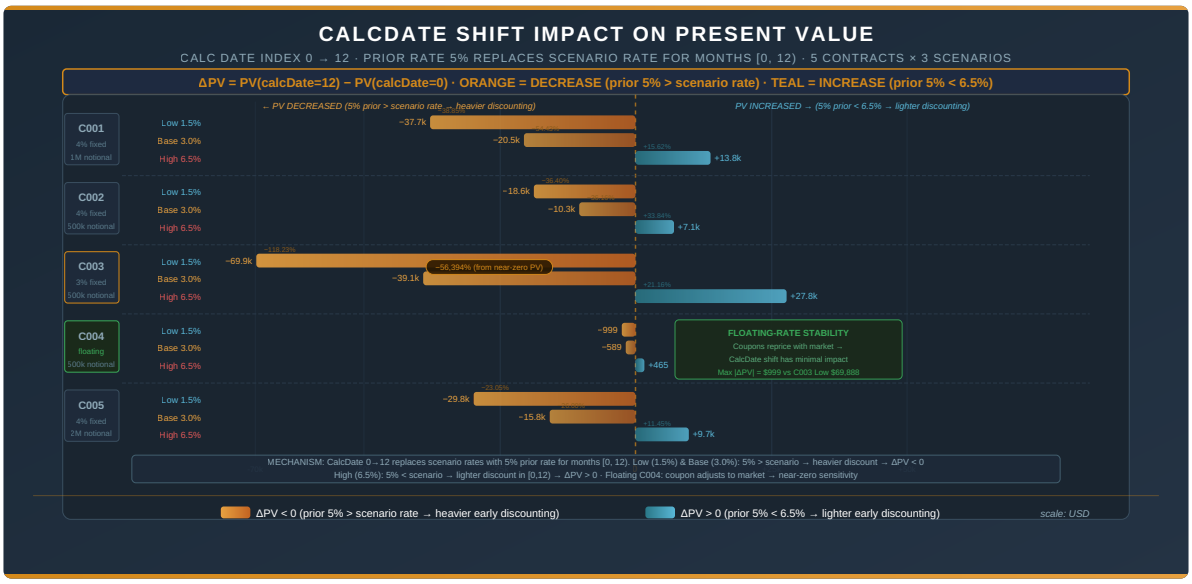
C004 floating-rate stability. The floating-rate contract (coupon = market rate + 1% spread) is nearly rate-neutral: its coupons reprice with the environment, so changing the scenario barely changes PV. The $1,724 Low→High swing is 107× smaller than C001's $185,347 swing at comparable notional.

C005 largest absolute swing. C005's 2M notional drives the widest absolute PV range ($214k Low→High), confirming that PV sensitivity is proportional to notional.

---

4 · Experiment 3 — CalcDate Shift Impact

Variable changed: calcDateIndex (0 → 12). Prior rate 5% is used for months [0, 12); scenario rates apply from month 12 onward. CPU backend, all scenarios.



4.1 Mechanism

Shifting calcDateIndex from 0 to 12 substitutes the 5% prior rate for the scenario rate during the first year. The direction of the PV change therefore depends on the relative ordering of 5% versus the scenario rate:

Scenario	Scenario Rate	Prior Rate	Effect on [0,12) discount	ΔPV
Low 1.5%	1.5%	5%	heavier (5% > 1.5%)	**negative**
Base 3.0%	3.0%	5%	heavier (5% > 3.0%)	**negative**
High 6.5%	6.5%	5%	lighter (5% < 6.5%)	**positive**

4.2 Full Delta Table

Contract	Scenario	PV (CD=0)	PV (CD=12)	ΔPV	δ%
C001	Low	97,125	59,390	−37,735	−38.8%
C001	Base	37,756	17,207	−20,549	−54.4%
C001	High	−88,222	−74,442	+13,780	+15.6%
C002	Low	51,214	32,573	−18,641	−36.4%
C002	Base	28,435	18,153	−10,281	−36.2%
C002	High	−20,984	−13,882	+7,101	+33.8%
C003	Low	59,114	−10,774	−69,888	−118.2%
C003	Base	−69	−39,134	−39,065	−56,394%
C003	High	−131,516	−103,685	+27,832	+21.2%
C004	Low	28,022	27,024	−999	−3.6%
C004	Base	27,710	27,121	−589	−2.1%
C004	High	26,299	26,763	+465	+1.8%
C005	Low	129,417	99,587	−29,830	−23.1%
C005	Base	60,615	44,858	−15,757	−26.0%
C005	High	−84,997	−75,268	+9,729	+11.5%

4.3 C003 Base — Extreme Percentage Delta

C003 Base shows a −56,394% percentage change. This arises because the starting PV is near zero (−$69.27); a $39,065 shift in absolute terms produces a massive percentage figure. The absolute delta is consistent with C003's rate sensitivity observed in Experiment 2: the first 12 months of discounting at 5% instead of 3% drives PV sharply negative.

4.4 C004 Floating-Rate Stability Under CalcDate

The floating-rate contract again demonstrates near-immunity: the maximum CalcDate-induced delta is $999 (Low scenario), versus C003's $69,888 maximum. Fixed-rate contracts "lock in" discounting exposure; floating-rate contracts partially hedge it because the prior-rate boundary changes both the discount rate and the coupon accrual in offsetting directions.

---

5 · Proof of Causality

Each experiment changes exactly one variable while holding all others fixed. The results cleanly isolate the three engine components:

Experiment	Variable	Result
1 (CPU vs GPU)	Execution backend	Numerical equivalence confirmed; GPU 4.9× faster
2 (Scenario)	Interest-rate level	Inverse PV/rate for fixed; floating rate-neutral
3 (CalcDate)	Prior-rate boundary	ΔPV direction follows prior-vs-scenario rate ordering

No cross-experiment contamination: Exp 2 and Exp 3 use CPU only; Exp 3 reuses Exp 2's CalcDate=0 results (avoiding redundant computation).

---

6 · Timing Summary

Run	Contracts	Scenarios	Backend	CALC time
Exp 1 CPU	5	3	CPU	93 ms
Exp 1 GPU	5	3	GPU (RTX 3060 Ti)	19 ms
Exp 3 CalcDate=12	5	3	CPU	< 1 ms (reuses Vasicek paths)

End-to-end wall time for the complete demo is under 200 ms — fast enough for interactive development and scenario testing.

> *Output:* `./out/exp1_cpu_vs_gpu.csv`, `./out/exp2_scenario_impact.csv`, `./out/exp3_calcdate_impact.csv`
>
> *Contract type: PAM · Scenario model: flat curves · Engine: ActusCoreCsharp*

Benchmark Analysis Report

GPU-Accelerated ACTUS Insurance — Benchmark Analysis Report

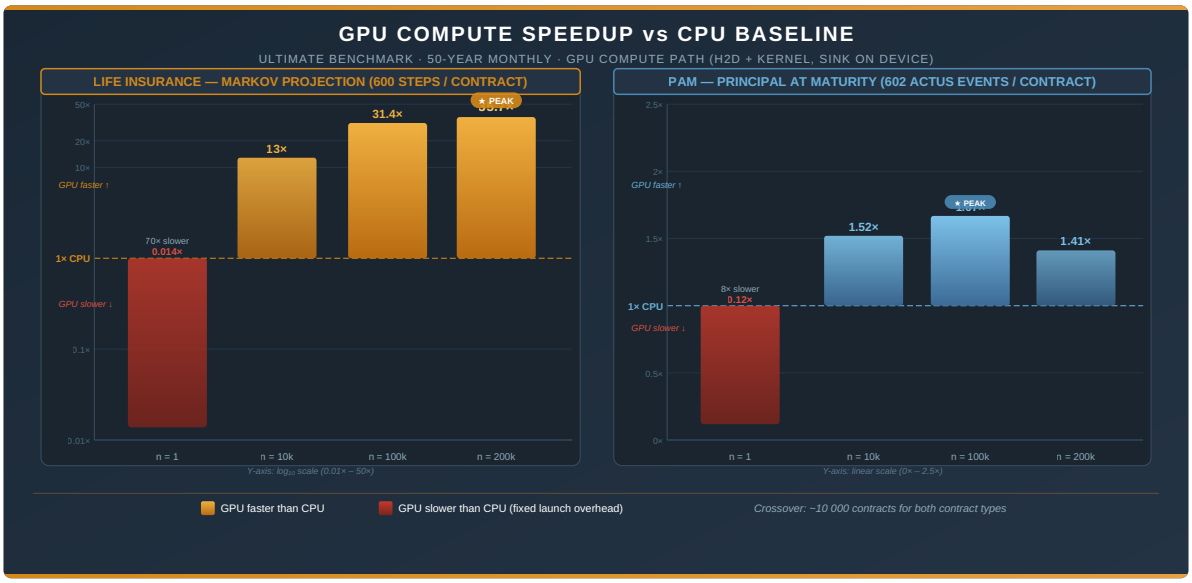
> ***Platform:*** *AMD Ryzen 7 3800X 3.90 GHz · 16 logical / 8 physical cores · Windows 11 25H2*
>
> ***Runtime:*** *.NET 9.0.14, X64 RyuJIT (AVX2, FMA, BMI2, 256-bit vectors)*
>
> ***Framework:*** *BenchmarkDotNet v0.15.8 — ShortRun (3 warmup + 3 target iterations)*
>
> ***Date:*** *2026-03-14*

Executive Summary

This report presents empirical performance evidence for GPU-accelerated ACTUS insurance contract evaluation versus a sequential CPU baseline. Two contract types were benchmarked — **PAM** (Principal-at-Maturity, event-driven scheduling) and **Life** (Markov-chain mortality projection) — across portfolio sizes of 1, 10 000, 100 000, and 200 000 contracts, each simulating a 50-year monthly timeline.

Three headline findings:

Finding	Evidence
GPU acceleration for Life Insurance reaches **35.7× speedup** at 200k contracts	`GPU compute 41.5 ms` vs `CPU 1,481 ms` at BatchSize=200k
GPU accelerates PAM contracts by **1.66×** at production-scale 100k portfolios	`GPU compute 8.3 s` vs `CPU 13.9 s` at BatchSize=100k
GPU efficiency **improves with scale** while CPU is linear — the crossover is at ~10k contracts	ns/c/s drops from 573k → 208 (GPU) vs stays flat ~7,400 (CPU) for Life



---

1 · Background and Methodology

1.1 Workload Definitions

Benchmark Class	Contract Type	Computation Model	Events / Steps
`UltimatePamCrossoverBenchmarks`	PAM (fixed-rate, 2025–2075)	ACTUS Schedule + Apply	~602 IP events / contract
`UltimateLifeCrossoverBenchmarks`	Life (age 25–65, mixed gender)	Sequential Markov projection	600 monthly steps

1.2 Measurement Metric

All rows report **`ns/c/s`** — *nanoseconds per contract per scenario* — defined as:

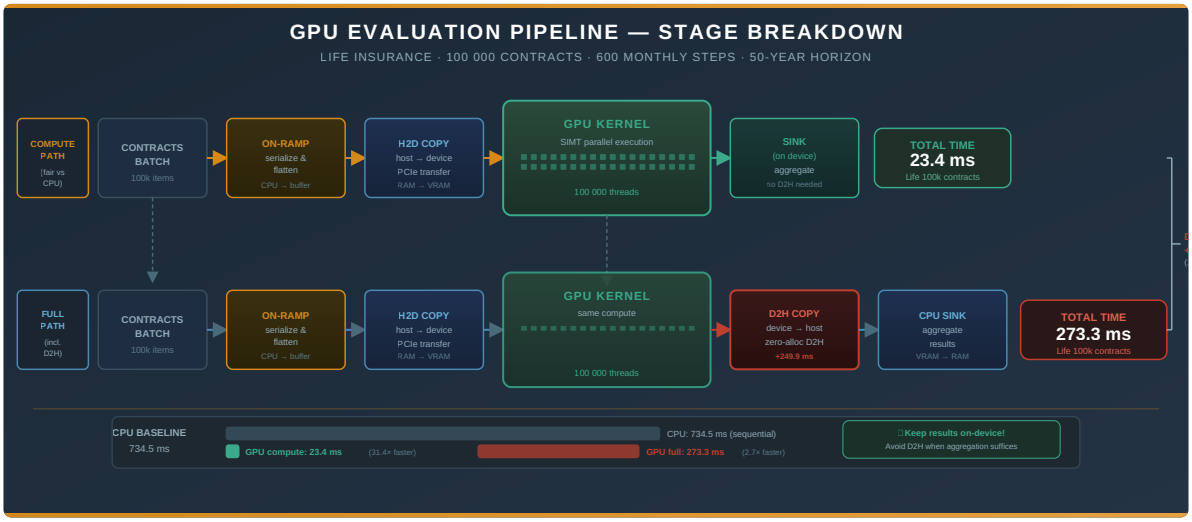
```
ns/c/s = Mean_ns / (BatchSize × ScenarioCount)
```

This normalised metric enables apples-to-apples comparison across all batch sizes and eliminates the confounding effect of portfolio scale.

1.3 GPU Pipeline Variants

Two GPU variants were measured to isolate transfer costs:

- **GPU compute path** — On-ramp + Host-to-Device (H2D) + kernel, results **sink on device** (no D2H). This is the *fair comparison* to CPU: same computation, no unnecessary round-trip.
- **GPU full path** — On-ramp + H2D + kernel + **Device-to-Host (D2H)** sink (zero-alloc). This represents the end-to-end cost when results must be retrieved to the CPU.



---

2 · PAM Benchmark Results

Principal-at-Maturity contracts — 50-year monthly, 602 ACTUS events / contract

2.1 Raw Data

Batch Size	CPU Mean	GPU Compute Mean	GPU Full Mean	CPU ns/c/s	GPU Compute ns/c/s
1	148.4 µs	1,204.8 µs	1,235.7 µs	148,361	1,204,844
10,000	1,355.4 ms	892.0 ms	1,041.2 ms	135,541	89,204
100,000	13,863.8 ms	8,323.0 ms	9,361.4 ms	138,638	83,230
200,000	28,651.4 ms	20,270.4 ms	20,046.4 ms	143,257	101,352

2.2 Speedup over CPU Baseline

Batch Size	GPU Compute Speedup	GPU Full Speedup	Winner
1	**0.12×** (8× slower)	0.12× (8× slower)	CPU ✓
10,000	**1.52×**	1.30×	GPU ✓
100,000	**1.67×**	1.47×	GPU ✓
200,000	**1.41×**	1.43×	GPU ✓

2.3 Speedup Trend (GPU Compute vs CPU)

```
BatchSize        Speedup
──────────────────────────────────────────────────

         1   ▒  0.12×  (fixed overhead dominates)
    10 000   ▒▒▒
```

3.2 Speedup over CPU Baseline

Batch Size	GPU Compute Speedup	GPU Full Speedup	Winner
1	**0.014×** (70× slower)	0.013× (77× slower)	CPU ✓
10,000	**13.0×**	2.12×	GPU ✓
100,000	**31.4×**	2.69×	GPU ✓
200,000	**35.7×**	2.67×	GPU ✓

3.3 Speedup Trend (GPU Compute vs CPU)

```
BatchSize        Speedup
──────────────────────────────────────────────────────────────
         1   ▒  0.014×  (launch overhead ~70× the entire workload)
    10 000   ▒▒▒
```

4.1 Life Insurance: GPU Efficiency vs CPU

```
ns/c/s (lower = faster)          ← CPU flat ~7,400 ns/c/s
──────────────────────────────────────────────────────────

BatchSize=1       573,169
▓▓▓
```

5 · Memory Allocation Analysis

PAM Benchmarks

Batch Size	CPU Allocated	GPU Allocated	Alloc Ratio
1	227.09 KB	209.95 KB	**0.92**
10,000	2,270,937 KB	2,093,829 KB	**0.92**
100,000	22,709,375 KB	20,938,282 KB	**0.92**
200,000	45,418,750 KB	41,876,563 KB	**0.92**

The GPU path consistently allocates **8% less managed heap memory** than the CPU path at every batch size. This is attributable to the zero-copy, zero-alloc Device-to-Host sink implementation.

Life Benchmarks

The GPU Life path allocates near-zero managed memory (**80 bytes for compute path**, 160 bytes for full path), compared to zero for the CPU sequential path. This confirms that the GPU runtime objects are entirely on the unmanaged GPU side.

---

6 · Conclusions

6.1 Primary Conclusion: GPU Acceleration Is Proven for Production-Scale Portfolios

Both contract types demonstrate clear, statistically consistent GPU speedups at portfolio sizes representative of real insurance book processing (≥10k contracts).

6.2 Workload-Specific Observations

Life Insurance (Markov projection) exhibits the largest gains because:

- Each contract's Markov projection is arithmetically dense and fully data-parallel.
- No inter-contract dependencies exist — ideal for SIMT execution.
- The GPU compute kernel achieves **35.7× throughput improvement** at 200k contracts.

PAM (event-driven scheduling) shows more modest but still meaningful gains because:

- ACTUS event scheduling involves conditional branching per event (harder to vectorise).
- Despite this, the GPU still achieves **1.67× speedup** at 100k contracts.
- Memory savings of 8% suggest the GPU pipeline also reduces GC pressure.

6.3 Fixed Overhead Characterisation

Contract Type	GPU Launch Overhead	Crossover Point
Life (Markov)	~573 µs	~10,000 contracts
PAM (Event-based)	~1,205 µs	~10,000 contracts

GPU launch overhead is a one-time amortised cost. Any system processing batches of ≥10k contracts is in the *crossover zone* where GPU provides unambiguous benefit.

6.4 D2H Transfer Cost

The difference between the *compute path* and *full path* isolates the Device-to-Host round-trip cost. For Life workloads, this cost is significant at scale (e.g., at 100k: 23 ms compute vs 273 ms full path). This motivates *on-device aggregation* patterns: when only summary statistics are needed, keeping results on the GPU and aggregating before transfer dramatically reduces total wall-clock time.

---

7 · Evidence Summary Table

Claim	Supporting Measurement
GPU outperforms CPU above 10k contracts (Life)	Ratio = 0.08 at BatchSize=10k → **13.0×** faster
GPU peak speedup is 35.7× (Life, 200k)	CPU=1,481 ms, GPU compute=41.5 ms, Ratio=0.028
GPU outperforms CPU above 10k contracts (PAM)	Ratio = 0.66 at BatchSize=10k → **1.52×** faster
GPU peak speedup is 1.67× (PAM, 100k)	CPU=13,864 ms, GPU compute=8,323 ms, Ratio=0.60
GPU efficiency scales with batch size	

Claim	Supporting Measurement
	Life GPU ns/c/s: 573,169 (n=1) → 208 (n=200k) = **2,755× improvement**
CPU efficiency is linear (flat ns/c/s)	CPU ns/c/s: 8,152 (n=1) → 7,406 (n=200k) = constant
GPU uses 8% less managed memory (PAM)	Alloc Ratio = 0.92 across all batch sizes
Single-contract GPU is slower (expected)	Life Ratio=70×, PAM Ratio=8× — launch overhead >> computation

---

8 · Experimental Configuration

Setting	Value
Host	Windows 11 (10.0.26200.8037 / 25H2)
CPU	AMD Ryzen 7 3800X 3.90 GHz · 1 socket · 8 physical · 16 logical cores
SIMD	AVX2, BMI1/2, FMA, F16C, LZCNT — VectorSize=256
Runtime	.NET 9.0.14, X64 RyuJIT x86-64-v3
GC	Non-concurrent Workstation
Power plan	High Performance (set by BenchmarkDotNet)
Job	ShortRun — LaunchCount=1, WarmupCount=3, IterationCount=3
PAM test data	Synthetic fixed-rate contracts 2025–2075, monthly IP (602 events)
Life test data	42 synthetic contracts (age 25–65, M/F/X, mixed modes), cycled

---

> *Generated by automated analysis of BenchmarkDotNet v0.15.8 output.*
>
> *Raw data:* `results/ActusInsurance.Benchmarks.UltimatePamCrossoverBenchmarks-report.csv`
>
> *Run log:* `run-results.txt`

Code Resources

Code Resources

The ACTUS Insurance extension is spread across four repositories under the FransVanEk GitHub organisation.

Repositories

Actus-Insurance.Documentation — Public

The documentation you are reading now. Contains all conceptual and technical documentation for the insurance extension: the Markov model, DSL and product rules, Monte Carlo simulation, and the life insurance projection model.

Actus-Insurance.Core — Public

The C# implementation of the insurance contract extensions. Contains the Markov state machine, the DSL interpreter, actuarial lookup tables, the rule pack loader, and the insurance contract adapter that connects to the ACTUS contract engine.

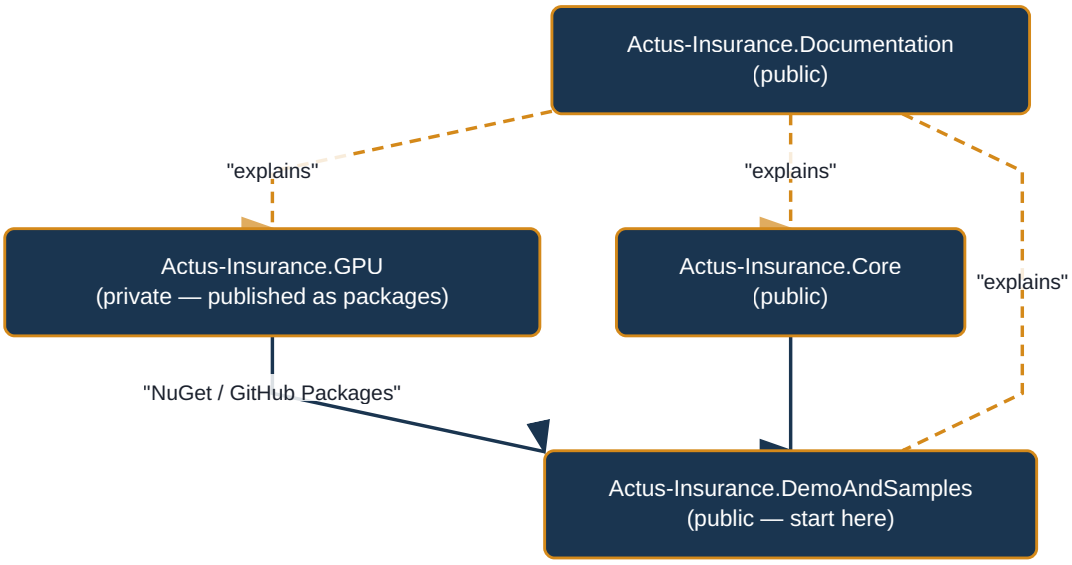
Actus-Insurance.GPU — Private

The high-performance GPU kernel that runs the portfolio projection. This repository is proprietary and not publicly available. It publishes compiled packages to GitHub Packages, which the other repositories consume as dependencies.

Actus-Insurance.DemoAndSamples — Public

End-to-end examples showing how to use the insurance extension in your own solution. This is the recommended starting point for implementers. It consumes the published GPU packages as a dependency and demonstrates how the Core, GPU, and Documentation components work together in practice.

How They Fit Together



The GPU component is the only proprietary piece. Because it ships as a package, you can build and run the demo and samples without access to the GPU source — the compiled package is sufficient. The Core and DemoAndSamples repositories are fully open and show the complete integration pattern.

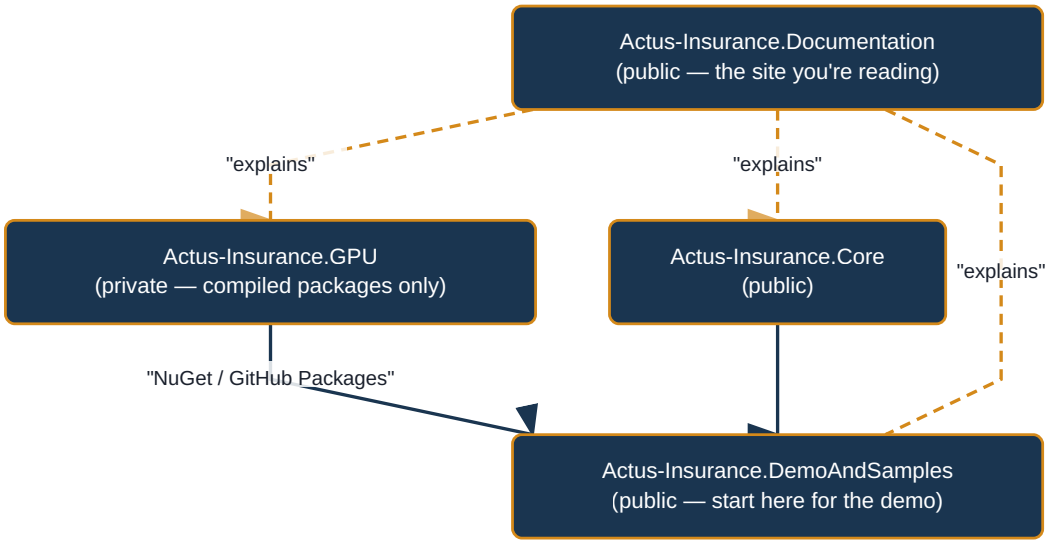
Installation & Getting Started

Installation & Getting Started

Overview

The ACTUS Insurance extension is delivered across four repositories. You do not need all of them — which ones you need depends on what you want to do.

Goal	What to use	Time
Run pre-built images immediately	Pre-built Docker images from Docker Hub	~2 min
See the demo UI running	Docker Compose — DemoAndSamples	~5 min
Run PAM Monte Carlo analysis (containerised)	Pre-built `pam-monte-carlo` image	~2 min
Run the CLI projection tool	CLI inside DemoAndSamples	~10 min
Build and test the Core engine	Actus-Insurance.Core	~10 min
Run the docs site locally	Actus-Insurance.Documentation	~5 min
Use GPU packages in your own project	NuGet from GitHub Packages	See below



The GPU component is the only proprietary piece. It ships as compiled packages, so you can build and run everything else without access to the GPU source.

Prerequisites

For the Docker demo (recommended starting point)

- Docker Desktop (Windows, macOS, or Linux)
- Docker Compose (included with Docker Desktop)
- Git

> ***Windows note:*** *Docker Desktop requires WSL 2 on Windows. During installation, Docker Desktop will prompt you to enable it if it is not already active.*

For building the Core engine

- .NET 9 SDK
- Git

For GPU acceleration (optional)

> ***TODO:*** *Document the specific GPU driver requirements (NVIDIA CUDA version, AMD ROCm version, or Intel OpenCL version) needed for GPU execution. Without a compatible GPU, the system automatically falls back to CPU execution — so GPU is optional for evaluation purposes.*

For the documentation site

- Node.js 18+ and npm (or yarn / pnpm / bun)
- Git

Option A — Pre-built Docker Images

Ready-to-use containerised applications are published to Docker Hub. Pull and run them directly without cloning any repository or building from source. This is the fastest way to get results.

Available images

Image	Description
`neobluetechlabs/pam-monte-carlo:latest`	PAM Monte Carlo 50-Year Analysis
`neobluetechlabs/scenario-cpu-gpu-demo:latest`	CPU vs GPU Scenario Analysis
`neobluetechlabs/actus-api:latest`	FastEndpoints REST API
`neobluetechlabs/actus-designer:latest`	Next.js Frontend Dashboard

All images are published as multi-architecture manifests and support `linux/amd64` (Intel/AMD 64-bit) and `linux/arm64` (ARM 64-bit, including Apple Silicon). Docker automatically selects the correct variant for your system when pulling.

PAM Monte Carlo Analysis

1. Pull the image

```
docker pull neobluetechlabs/pam-monte-carlo:latest
```

2. Create an output directory

```
mkdir -p out-monte-carlo
```

3. Run the analysis

```
docker run --rm -v $(pwd)/out-monte-carlo:/app/out neobluetechlabs/pam-
monte-carlo:latest
```

By default this runs both CPU and GPU backends with 10 000 contracts, 1 000 scenarios, and a 50-year (600-month) horizon.

Backend selection

```
# GPU only
docker run --rm -v $(pwd)/out-monte-carlo:/app/out neobluetechlabs/pam-
monte-carlo:latest --backend gpu

# CPU only
docker run --rm -v $(pwd)/out-monte-carlo:/app/out neobluetechlabs/pam-
monte-carlo:latest --backend cpu

# Both (default)
docker run --rm -v $(pwd)/out-monte-carlo:/app/out neobluetechlabs/pam-
monte-carlo:latest --backend both
```

Custom parameters

```
# Smaller test run
docker run --rm -v $(pwd)/out-monte-carlo:/app/out neobluetechlabs/pam-
monte-carlo:latest \
  --backend gpu \
  --contracts 1000 \
  --scenarios 100 \
  --months 240
```

```
# Export portfolio for reuse
docker run --rm -v $(pwd)/out-monte-carlo:/app/out neobluetechlabs/pam-monte-carlo:latest \
  --backend cpu \
  --contracts 500 \
  --scenarios 200 \
  --export-portfolio true
```

Parameters reference

Parameter	Default	Description
`--backend`	`both`	Execution backend: `cpu`, `gpu`, or `both`
`--contracts`	`10000`	Portfolio size
`--scenarios`	`1000`	Monte Carlo scenarios
`--months`	`600`	Time horizon (600 = 50 years)
`--seed`	`12345`	Random seed for reproducibility
`--calcDateIndex`	`0`	Calculation date month index
`--export-portfolio`	—	Export generated portfolio to CSV when set to `true`

Expected output files

```
out-monte-carlo/
├── run0_cpu_portfolio_pv_by_scenario.csv
├── run0_cpu_summary.json
├── run0_gpu_portfolio_pv_by_scenario.csv
├── run0_gpu_summary.json
├── run1_backtest_cpu_*
└── run1_backtest_gpu_*
```

Scenario CPU-GPU Demo

1. Pull the image

```
docker pull neobluetechlabs/scenario-cpu-gpu-demo:latest
```

2. Create an output directory

```
mkdir -p out-scenario
```

3. Run the demo

```
docker run --rm -v $(pwd)/out-scenario:/app/out neobluetechlabs/scenario-cpu-gpu-demo:latest
```

This runs three experiments and writes results to the output directory.

Experiment	Description
Experiment 1	CPU vs GPU backend comparison — validates 15/15 pairs within tolerance
Experiment 2	Interest rate scenario impact analysis
Experiment 3	Calculation date causality effects

Expected output files

```
out-scenario/
├── portfolio.csv
├── scenarios.csv
├── exp1_cpu_vs_gpu.csv
├── exp2_scenario_impact.csv
└── exp3_calcdate_impact.csv
```

API and Frontend Services (Pre-built)

Pull the images

```
docker pull neobluetechlabs/actus-api:latest
docker pull neobluetechlabs/actus-designer:latest
```

Run individual services

```
# Start the API server
docker run --rm -p 8080:8080 neobluetechlabs/actus-api:latest
```

```
# Start the frontend dashboard
docker run --rm -p 3000:3000 neobluetechlabs/actus-designer:latest
```

Run the complete stack with Docker Compose

Create a `docker-compose.yml` file:

```
version: '3.8'
services:
  actus-api:
    image: neobluetechlabs/actus-api:latest
    ports:
      - "8080:8080"
    volumes:
      - actus-data:/app/data

  actus-designer:
    image: neobluetechlabs/actus-designer:latest
    ports:
      - "3000:3000"
    depends_on:
      - actus-api

volumes:
  actus-data:
```

Then start the stack:

```
# Start both services
docker compose up -d

# Check service status
docker compose ps

# Stop services
docker compose down
```

Access URLs

Service	URL
Frontend (ACTUS Designer)	http://localhost:3000

Service	URL
API	http://localhost:8080
API documentation (Swagger)	http://localhost:8080/swagger
Health check	http://localhost:8080/health

Volume mounting

```
# Mount the current directory's output folder
docker run --rm -v $(pwd)/out:/app/out neobluetechlabs/pam-monte-carlo:latest

# Mount a specific absolute path
docker run --rm -v /path/to/output:/app/out neobluetechlabs/pam-monte-carlo:latest

# Mount with explicit read-write permissions
docker run --rm -v $(pwd)/out:/app/out:rw neobluetechlabs/pam-monte-carlo:latest

# API data persistence
docker run --rm -p 8080:8080 -v $(pwd)/api-data:/app/data neobluetechlabs/actus-api:latest
```

Environment variables

API Service

Variable	Default	Description
`ASPNETCORE_URLS`	`http://+:8080`	Server binding
`Calculation__PreferGpu`	`false`	GPU preference
`Logging__LogLevel__Default`	`Information`	Log level

Frontend Service

Variable	Default	Description
`NODE_ENV`	`production`	Environment mode
`API_BASE_URL`	`http://actus-api:8080`	API endpoint
`PORT`	`3000`	Server port

Performance and resource limits

Limit CPU and memory usage to avoid contention with other processes:

```
docker run --rm --cpus="2.0" --memory="4g" \
  -v $(pwd)/out:/app/out neobluetechlabs/pam-monte-carlo:latest
```

Run multiple containers in parallel for different parameter sets:

```
docker run --rm -v $(pwd)/out-gpu:/app/out neobluetechlabs/pam-monte-
carlo:latest --backend gpu --scenarios 500 &
docker run --rm -v $(pwd)/out-cpu:/app/out neobluetechlabs/pam-monte-
carlo:latest --backend cpu --scenarios 500 &
wait
```

CI/CD integration example

```
# .github/workflows/docker.yml
name: Test Monte Carlo
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Pull and test Monte Carlo
        run: |
          docker pull neobluetechlabs/pam-monte-carlo:latest
          mkdir -p test-output
          docker run --rm -v $(pwd)/test-output:/app/out neobluetechlabs/
```

```
pam-monte-carlo:latest --contracts 100 --scenarios 50
          ls -la test-output/
```

Custom orchestration example

```
# custom-docker-compose.yml
version: '3.8'
services:
  monte-carlo:
    image: neobluetechlabs/pam-monte-carlo:latest
    command: ["--backend", "gpu", "--contracts", "5000"]
    volumes:
      - ./monte-carlo-output:/app/out

  api:
    image: neobluetechlabs/actus-api:latest
    ports:
      - "8080:8080"
    environment:
      - Calculation__PreferGpu=true

  frontend:
    image: neobluetechlabs/actus-designer:latest
    ports:
      - "3000:3000"
    depends_on:
      - api
```

Option B — Docker Compose: Full Stack (Build from Source)

This starts the .NET backend API (port 8080) and the Next.js frontend (port 3000) together by building images from source. Use this when you want to modify the code or need the latest unreleased changes.

1. Clone the repository

```
git clone https://github.com/FransVanEk/Actus-Insurance.DemoAndSamples.git
cd Actus-Insurance.DemoAndSamples
```

2. Configure the environment

Copy or review the `.env` file in the repository root. The key variables are:

Variable	Default	Description
`API_PORT`	`8080`	Port for the backend API
`FRONTEND_PORT`	`3000`	Port for the frontend
`PREFER_GPU`	`false`	Set to `true` to enable GPU acceleration
`API_KEY`	—	Authentication key for API access

> ***TODO:*** *Document the default value for `API_KEY` and whether it must be set before first run, or whether a development default is already present in the `.env` file.*

To use the default configuration, no changes are needed. To enable GPU execution, set `PREFER_GPU=true` — but see the GPU prerequisites above first.

3. Start the services

```
docker-compose up -d --build
```

The `--build` flag ensures Docker builds the images from source. On first run this takes a few minutes. Subsequent starts are faster.

To watch the startup logs:

```
docker-compose logs -f
```

4. Access the applications

Once the containers are running:

Service	URL	Description
Frontend (ACTUS Designer)	http://localhost:3000	Dashboard, contract management, risk visualisation
Backend API	http://localhost:8080	REST API for calculations and data
API documentation	http://localhost:8080/swagger	Interactive Swagger UI

5. Stop the services

```
docker-compose down
```

Development mode

For development with live reload and overrides:

```
docker-compose -f docker-compose.yml -f docker-compose.override.yml up -d
```

> ***TODO:*** *Document what the* `docker-compose.override.yml` *changes relative to the production compose file (e.g. volume mounts, hot reload, debug ports).*

Option C — CLI Tool: Portfolio Projection

The `PamMonteCarlo50Y` CLI tool runs the full evaluation pipeline from the command line — no UI required. It is the best way to exercise the GPU/CPU engine and see benchmark results directly.

Location

The CLI tool is located inside the DemoAndSamples repository:

```
Actus-Insurance.DemoAndSamples/
└── CLI/
    └── PamMonteCarlo50Y/
```

> ***TODO:*** *Confirm whether the CLI tool is built and available as a standalone binary inside the Docker image, or whether it must be built separately with* `dotnet build`*. Document the exact path and how to invoke it (e.g.* `dotnet run`*,* `dotnet publish`*, or a pre-built executable).*

Prerequisites for CLI

The CLI requires access to the GPU packages from GitHub Packages. Before building, you must configure NuGet authentication:

> ***TODO:*** *Document the exact steps to authenticate against GitHub Packages to restore the GPU NuGet dependency. Specifically:*
>
> - *Whether a GitHub Personal Access Token (PAT) with* `read:packages` *scope is needed*
> - *How to configure the* `nuget.config` *with the token (environment variable,* `dotnet nuget add source`*, or direct edit)*
> - *Whether the packages are available publicly or require organisation membership*

A typical `nuget.config` setup for GitHub Packages looks like:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <add key="github" value="https://nuget.pkg.github.com/FransVanEk/
index.json" />
  </packageSources>
  <packageSourceCredentials>
    <github>
      <add key="Username" value="YOUR_GITHUB_USERNAME" />
      <add key="ClearTextPassword" value="YOUR_GITHUB_PAT" />
    </github>
  </packageSourceCredentials>
</configuration>
```

> ***TODO:*** *Confirm whether the* `nuget.config` *in the repository already contains the correct source URL and only needs credentials filled in, or whether additional configuration is required.*

Running the CLI

Synthetic mode — generate a random portfolio and run the full Monte Carlo pipeline:

```
PamMonteCarlo50Y --backend gpu --contracts 10000 --scenarios 1000 --
months 600 --seed 42
```

Input-directory mode — load an existing portfolio from files:

```
PamMonteCarlo50Y --backend gpu --input ./samples/input --out ./results
```

CPU-only mode (no GPU required):

```
PamMonteCarlo50Y --backend cpu --contracts 10000 --scenarios 100
```

Compare CPU and GPU (runs both and reports timings):

```
PamMonteCarlo50Y --backend both --contracts 10000 --scenarios 100
```

Key CLI options

Option	Description	Default
`--backend`	`cpu`, `gpu`, or `both`	`both`
`--contracts`	Number of synthetic contracts to generate	`1000`
`--scenarios`	Number of Monte Carlo scenarios	`100`
`--months`	Projection horizon in months (600 = 50 years)	`600`
`--seed`	Random seed for reproducibility	`42`
`--input`	Input directory for pre-built portfolio	—
`--out`	Output directory for results	`./out`
`--reporting`	Enable detailed CSV reporting exports	`false`
`--export-fact`	Export long-format fact table	`false`
`--export-portfolio`	Export generated portfolio to CSV	`false`

Option	Description	Default
`--metadata`	Path to contract metadata CSV for joined reporting	—

Example: benchmarking CPU vs GPU

```
PamMonteCarlo50Y --backend both --contracts 100000 --scenarios 1000 --
seed 42
```

This runs 100 million projections on both backends and prints timing results. Expected outcome on a modern workstation: GPU approximately 2× faster than CPU for this workload.

Expected output files

When `--reporting true` is set, the output directory contains:

File	Content
`{runId}_portfolio_by_scenario.csv`	Portfolio present value per scenario
`{runId}_contract_summary.csv`	Per-contract statistics: Mean, StdDev, P05–P95, ES95, ES99
`{runId}_fact_results_long.csv`	Long-format fact table (when `--export-fact true`)
`runs.csv`	Run dimension table with metadata

These files are designed to open directly in Excel or be imported into BI tools.

Input directory structure

When using `--input`, the directory must follow this layout:

```
input_dir/
├── portfolio.csv                    — Contract terms in CSV format
├── scenarios/
|   ├── scenario_set.json            — Scenario metadata
|   └── riskfactors/
|       ├── interest_rate_prior.csv  — Historical rates (before
calcDateIndex)
|       └── interest_rate_after.csv  — Forward rates (scenario-specific)
```

```
├── runs.json                        — (optional) Multi-run definitions
└── contract_metadata.csv            — (optional) Extra contract
attributes for reporting
```

> ***TODO:*** *Link to or include a sample* `portfolio.csv` *and* `scenario_set.json` *so a first-time user can see the expected format without building synthetic data first. The* `samples/` *directory in the repository may already contain these.*

Option D — Developer Build: Core Engine

Use this if you want to read, modify, or extend the core C# engine — the PAM contract implementation, conventions, state machine, or actuarial components.

1. Clone the repository

```
git clone https://github.com/FransVanEk/Actus-Insurance.Core.git
cd Actus-Insurance.Core
```

2. Restore and build

```
dotnet restore
dotnet build
```

The solution contains three projects:

Project	Description
`ActusInsurance.Core`	Shared abstractions: interfaces, conventions, domain models, risk factors
`ActusInsurance.Core.CPU`	PAM contract implementation: schedule generation and event evaluation
`ActusInsurance.Tests.CPU`	42 reference test cases from the official ACTUS test suite

3. Run the tests

```
dotnet test
```

All 42 official ACTUS reference test cases should pass. These validate every event type, financial convention, and edge case against known-correct results to 10 decimal places.

```
Test run summary:
  Passed: 42
  Failed: 0
```

> ***TODO:*** *Confirm the exact output format from* `dotnet test` *and whether there are any expected skipped or pending tests in the current state of the repository.*

4. Explore the project structure

```
src/
├── ActusInsurance.Core/           # Pure C# — no external dependencies
│   ├── Contracts/                 # IContractScheduler interface
│   ├── Conventions/               # Day count, business day, EOM,
calendars
│   ├── Events/                    # ContractEvent: Evaluate,
ComputePayoff, UpdateState
│   ├── Models/                    # PamContractTerms (40+ fields)
│   ├── States/                    # StateSpace value struct
│   ├── Time/                      # ScheduleFactory, cycle arithmetic
│   └── Types/                     # Enums: EventType, ContractRole, etc.
├── ActusInsurance.Core.CPU/
│   └── Contracts/PAM.cs           # PrincipalAtMaturity: Schedule +
Apply
└── ActusInsurance.Tests.CPU/
    ├── PamTests.cs                # 42 test cases
    └── Resources/
        └── actus-tests-pam.json   # Test data: terms, market data,
expected results
```

5. Making changes

The Core library has no external dependencies — it is pure C#. Changes to conventions, event logic, or contract terms do not require any package authentication. Run `dotnet test` after every change to verify correctness.

If you are extending the insurance components (Markov model, DSL, actuarial tables), those live in a separate layer on top of Core — see the Actus-Insurance.Core documentation for the architecture.

Option E — Documentation Site (Local)

Run the documentation website locally to preview documentation changes before publishing.

1. Clone the repository

```
git clone https://github.com/FransVanEk/Actus-Insurance.Documentation.git
cd Actus-Insurance.Documentation
```

2. Install dependencies

```
npm install
```

3. Start the development server

```
npm run dev
```

The site is available at http://localhost:3000. Changes to files in `docs/` are reflected immediately without restarting.

4. Build for production

```
npm run build
npm run start
```

> ***TODO:*** *Confirm whether there are any environment variables required to run the documentation site locally (e.g. for the resource list components or video embeds). If the site runs without any configuration, this section can be removed.*

Using the GPU Packages in Your Own Project

The GPU component (`Actus-Insurance.GPU`) is private and is not available as open source. It is published as compiled NuGet packages to GitHub Packages, which you can consume in your own .NET solution.

Step 1 — Request access

> ***TODO:*** *Document how a developer requests access to the GPU packages. Options might include:*
>
> - *Contacting Frans Van Ek directly via GitHub or LinkedIn*
> - *Being added as a collaborator to the private repository*
> - *Whether the packages are accessible to anyone with a GitHub account, or only to invited collaborators*

Step 2 — Configure NuGet

Add the GitHub Packages source to your project's `nuget.config`:

```
<?xml version="1.0" encoding="utf-8"?>
<configuration>
  <packageSources>
    <add key="nuget.org" value="https://api.nuget.org/v3/index.json" />
    <add key="actus-gpu" value="https://nuget.pkg.github.com/FransVanEk/
index.json" />
  </packageSources>
  <packageSourceCredentials>
    <actus-gpu>
      <add key="Username" value="YOUR_GITHUB_USERNAME" />
      <add key="ClearTextPassword" value="YOUR_GITHUB_PAT" />
    </actus-gpu>
  </packageSourceCredentials>
</configuration>
```

Create a GitHub Personal Access Token (PAT) with the `read:packages` scope at github.com/settings/tokens and replace `YOUR_GITHUB_PAT` with it.

> ***Tip:*** *Store the token as an environment variable and reference it from* `nuget.config` *using* `%VARIABLE_NAME%` *(Windows) or* `$VARIABLE_NAME` *(Linux/macOS) rather than putting it in the file directly.*

Step 3 — Add the package reference

> ***TODO:*** *Document the exact package name(s) and recommended version to reference. For example:*
>
> ```
> <PackageReference Include="ActusInsurance.GPU" Version="x.x.x" />
> ```

Step 4 — Reference the Core library

The GPU package depends on `Actus-Insurance.Core`. Add a reference to the Core project or its NuGet package:

> ***TODO:*** *Confirm whether* `Actus-Insurance.Core` *is also published as a NuGet package (in addition to being available as source), and if so, provide the package name and source.*

Troubleshooting

Pre-built images: empty output directory

If no files appear in the output directory after the container exits, check that the volume mount points to `/app/out` (for CLI images), that the host directory exists before running, and that Docker has write permissions to it. Inspect the container logs for errors:

```
docker logs <container-id>
```

Pre-built images: port conflicts

If Docker reports `Port already allocated`, stop the conflicting container or use a different host port:

```
# Stop a named container
docker stop <container-name>

# Map to a different host port
docker run --rm -p 8081:8080 neobluetechlabs/actus-api:latest

# List running containers
docker ps
```

Pre-built images: image cannot be pulled

Verify the image name and tag, then try pulling explicitly:

```
docker pull neobluetechlabs/pam-monte-carlo:latest
```

Pre-built images: GPU backend not producing GPU output files

GPU acceleration uses a CPU fallback by default if no compatible GPU is detected. This is not an error — it is a silent performance difference. Verify which backend was used by checking for `*_gpu_*.csv` files in the output directory and reviewing the container logs. To force CPU-only execution: `--backend cpu`.

Pre-built images: debugging container internals

```
# Interactive shell access
docker run --rm -it neobluetechlabs/pam-monte-carlo:latest /bin/bash

# List files inside the container
docker run --rm neobluetechlabs/pam-monte-carlo:latest ls -la /app/

# Monitor resource usage
docker stats <container-name>

# Inspect image layers
docker history neobluetechlabs/pam-monte-carlo:latest

# View available CLI options
docker run --rm neobluetechlabs/pam-monte-carlo:latest --help
```

Docker Compose: containers fail to start

Check the logs for the specific container that failed:

```
docker-compose logs backend
docker-compose logs frontend
```

Common causes:

- Port 3000 or 8080 already in use by another process — change `API_PORT` or `FRONTEND_PORT` in `.env`
- Docker Desktop not running — start Docker Desktop first
- WSL 2 not enabled (Windows) — Docker Desktop will prompt to enable it

dotnet restore fails with authentication error

This happens when the NuGet source for GitHub Packages is not configured or the token is missing. See the "Using the GPU Packages" section above for configuration steps.

If you are building only `Actus-Insurance.Core` (without the GPU dependency), no GitHub Packages authentication is needed — the Core library has no external dependencies.

Tests fail with locale-related number formatting errors

If test results show wildly incorrect values (interest rates of 525% instead of 5.25%), the system may be reading decimal numbers using the locale of the operating system rather than the fixed locale-independent format the engine enforces. This was an early challenge in the project and is described in detail in Roadblocks.

Make sure you are running the latest version of the repository — this issue was resolved by enforcing invariant culture parsing throughout the engine.

GPU not detected / falling back to CPU

If `PREFER_GPU=true` is set but the system falls back to CPU, the most likely causes are:

- No compatible GPU is present in the machine
- GPU drivers are not installed or are outdated
- The ILGPU framework used by the engine does not support the GPU model

The system is designed to fall back to CPU automatically — this is not an error condition, just a performance difference. See GPU vs CPU for guidance on when GPU acceleration is worthwhile.

> ***TODO:*** *Document the minimum supported GPU driver versions for NVIDIA, AMD, and Intel. Also document how to confirm which backend is actually being used at runtime (log output, environment variable, or CLI flag).*

What to Read Next

- Code Resources — description of all four repositories and how they fit together
- CLI Tool Reference — full CLI option reference and pipeline explanation
- Core Engine Overview — architecture of the C# engine
- GPU Acceleration — how the GPU execution layer works

Resources & Downloads

> *Everything produced during the hackathon is available here for download or inline viewing. Whether you want to walk through the slides, inspect the GPU architecture diagrams, watch the live demo, or dig into the raw test data — it's all in one place.*

Terminology & Glossary

This document defines every term, abbreviation, and concept used across the documentation. When a term appears for the first time in any document, it links back here. Use this page as a reference whenever you encounter an unfamiliar concept.

A

ACTUS

Algorithmic Contract Types Unified Standards. An open standard that defines financial contracts as deterministic algorithms. Each contract type is specified by a set of contract terms, a schedule of events, and a state machine that computes cash flows. ACTUS was created to bring transparency and consistency to financial contract modeling.

ACTUS-I

The insurance extension of the ACTUS standard. ACTUS-I introduces new contract types for life insurance, non-life insurance, annuities, and parametric insurance, while reusing the core ACTUS computational model (event-driven, deterministic state transitions).

Accrued Interest

The amount of interest that has accumulated on a financial contract since the last interest payment date but has not yet been paid out. In the ACTUS state machine, accrued interest is tracked continuously and reset to zero after each interest payment event (IP).

Actuarial Table

A lookup table that maps demographic or risk factors — such as age, gender, and policy duration — to probability values such as mortality rates, lapse rates, and disability rates. Actuarial tables are derived from population or industry experience data. In the projection engine, they are queried at each monthly time step to determine the hazard rates that drive state transitions in the Markov Graph.

AoS (Array of Structures)

A data layout where each object (e.g., a financial contract) is stored as a single contiguous block of memory containing all its fields. This is the natural layout in object-oriented programming. Compare with SoA.

B

Blittable Struct

A data structure whose in-memory representation is identical in managed (.NET) and unmanaged (GPU) memory. Blittable structs contain no managed references (no strings, arrays, or object pointers) and can be copied directly as raw bytes between CPU and GPU memory without marshaling overhead.

BenchmarkDotNet

A .NET library for writing and running performance benchmarks. Used in this project to measure CPU vs. GPU execution times across different portfolio sizes.

Business Day Convention

A rule that determines how a scheduled event date is adjusted when it falls on a non-business day (weekend or holiday). Examples include Following (move to the next business day), Modified Following (move forward unless it crosses a month boundary), and Preceding (move to the previous business day).

C

Calc Date Index

An integer representing a point in time on the scenario time grid. Events occurring before this index use historically observed rates ("prior rates"), while events at or after this index use scenario-generated rates ("after rates"). This enables backtesting and what-if analysis.

Cash Flow

A payment that occurs as the result of a financial contract event. Cash flows can be positive (money received) or negative (money paid). In ACTUS, every event either produces a cash flow (its payoff) or is administrative (zero payoff).

Contract Event

A discrete occurrence in the life of a financial contract that may change the contract state and/or produce a cash flow. Examples: Initial Exchange Date (IED), Interest Payment (IP), Maturity Date (MD), Rate Reset (RR).

Contract Role

Defines the perspective from which a contract is evaluated. The two primary roles are RPA (receive principal at maturity) and RPL (pay principal at maturity). The role determines the sign of cash flows: what is an inflow for one party is an outflow for the other.

Contract State

The set of variables that fully describe a financial contract at a given point in time. For a PAM contract, the state includes: notional principal, nominal interest rate, accrued interest, fee accrued, scaling multipliers, status date, and contract performance status.

CUDA

Compute Unified Device Architecture. NVIDIA's proprietary parallel computing platform and API that allows software to use NVIDIA GPUs for general-purpose computing (not just graphics). CUDA provides the highest performance for GPU computation on NVIDIA hardware.

D

Day Count Convention

A rule that specifies how the number of days between two dates is converted into a year fraction for interest calculations. Different conventions produce slightly different year fractions. Common conventions include:

Convention	Description
A/360	Actual days divided by 360
A/365	Actual days divided by 365
A/A ISDA	Actual days divided by actual days in year
30E/360	European 30/360 method
B/252	Business days divided by 252

Deterministic Execution

A computation that, given the same inputs, always produces exactly the same outputs. This property is critical for financial contract evaluation because results must be auditable, reproducible, and independently verifiable.

Discount Factor

A multiplier (between 0 and 1) that converts a future cash flow into its present value. If the short rate at time t is r(t), the discount factor is DF(t) = exp(-sum of r(i) * dt for i from 0 to t). A discount factor of 0.95 means that a cash flow one year from now is worth 95% of its face value today.

D2H (Device-to-Host)

The transfer of results from GPU memory (device) back to CPU memory (host) after computation completes. D2H transfer travels over the PCIe bus and is a key cost factor in GPU-accelerated pipelines. The sink architecture minimises D2H data volume by aggregating results on the device before transfer.

DSL (Domain-Specific Language)

A small programming language designed for a specific purpose. In this project, the DSL is used to define insurance product rules (premium calculations, claim eligibility, state transitions) without modifying the core engine code.

E

Event Schedule

The complete, ordered list of events that will occur during the lifetime of a financial contract. Generated from the contract terms (start date, maturity date, payment cycles, etc.). The schedule is deterministic: given the same terms, the same schedule is always produced.

Event Type

A classification of contract events. Each type has specific payoff and state-update logic. The ACTUS PAM event types are:

Code	Name	Description
IED	Initial Exchange Date	Contract starts; principal is exchanged
MD	Maturity Date	Contract ends; principal is returned
IP	Interest Payment	Periodic interest is paid
IPCI	Interest Capitalization	Accrued interest is added to principal
RR	Rate Reset	Interest rate is updated from market data

Code	Name	Description
RRF	Rate Reset Fixed	Interest rate is reset to a fixed value
FP	Fee Payment	A fee is paid
SC	Scaling	Notional or interest scaling is updated
PRD	Purchase Date	Contract is purchased by a new party
TD	Termination Date	Contract is terminated early
CD	Credit Default	A credit event occurs

Expected Shortfall (ES)

A risk metric that measures the average loss in the worst-case tail of a distribution beyond the VaR threshold. ES95 measures the average loss across the worst 5% of scenarios; ES99 measures the average loss across the worst 1%. Unlike VaR, which only identifies the loss threshold, Expected Shortfall quantifies how severe losses in the tail actually are. Also known as Conditional Value at Risk (CVaR). ES95 and ES99 are reported as output statistics in the CLI portfolio projection tool.

F

Floating Rate

An interest rate that changes over time according to market conditions, typically referencing a benchmark index (e.g., EURIBOR or an overnight rate). In ACTUS, floating-rate contracts include periodic Rate Reset events (RR) that update the nominal interest rate by querying the Risk Factor Model at the reset date. Contrast with a fixed rate, which remains constant throughout the contract's life.

G

Gompertz-Makeham Model

A parametric mortality model where the force of mortality (instantaneous death rate) increases exponentially with age: $\mu(x) = A + B \times C^x$ (equivalently, $A + B \times \exp(x \times \ln C)$). The parameter A represents an age-independent background mortality, B and C together describe the exponential increase with age. Widely used in actuarial science. In this project, it provides the base mortality rates for the life insurance projection kernel.

Grace Period

A defined window of time after a missed premium payment during which an insurance policy remains in force. During the grace period, the policyholder can pay the overdue premium and reinstate the policy to Active status. If the grace period expires without payment, the policy lapses. Represented as a distinct state in the Markov Graph, with transitions to Active (premium paid) or Lapsed (grace expired).

GPU (Graphics Processing Unit)

A processor originally designed for rendering graphics, now widely used for parallel computation. A GPU contains thousands of small cores that can execute the same operation on many data elements simultaneously. This makes GPUs well-suited for evaluating large portfolios of financial contracts in parallel.

H

H2D (Host-to-Device)

The transfer of data from CPU memory (host) to GPU memory (device). This is a necessary step before GPU computation can begin. The speed of H2D transfer, which is limited by PCIe bandwidth, is a key factor in overall GPU pipeline performance.

Hazard Rate

The instantaneous rate of transition from one state to another in a Markov model. In life insurance, hazard rates include mortality rate (death), lapse rate (policy cancellation), and disability rate. Hazard rates vary by age, gender, duration, and other risk factors.

I

IFRS 17

An international accounting standard for insurance contracts, effective from January 2023. IFRS 17 requires insurance companies to measure contracts using discounted, probability-weighted cash flow projections, making computational efficiency critical.

ILGPU

An open-source .NET library that provides a unified API for GPU programming. ILGPU abstracts over CUDA, OpenCL, and CPU backends, allowing the same kernel code to run on NVIDIA GPUs, AMD GPUs, or as a CPU fallback.

K

Kernel (GPU)

A function that runs on the GPU. Each kernel is executed by thousands of GPU threads simultaneously. In this project, each GPU thread processes one financial contract (or one contract-scenario pair), evaluating all events sequentially within that thread.

L

Lapse Rate

The proportion of insurance policies in a given cohort that are cancelled or surrendered by policyholders within a defined period. Lapse rates are typically modelled as a function of policy duration and are provided as actuarial table inputs to the Markov projection engine. Together with mortality rate and disability rate, lapse rate is one of the primary hazard rates governing policy state transitions.

LINS (Life Insurance)

A proposed ACTUS-I contract type for life insurance policies, covering term life, whole life, and universal life products.

M

Markov Graph

A directed graph where nodes represent states (e.g., Active, Lapsed, Claim Pending, Death Claimed) and edges represent possible transitions. Each edge has a probability (hazard rate) that determines the likelihood of transitioning from one state to another in a given time step. Used in the insurance extension for policy lifecycle modeling.

Monte Carlo Simulation

A computational technique that uses random sampling to estimate numerical results. In this project, Monte Carlo simulation generates many possible interest-rate paths (scenarios) and evaluates the portfolio under each one, producing a distribution of outcomes rather than a single point estimate.

Mortality Rate

The probability that a policyholder dies within a given time period (typically one month or one year), expressed as a function of age and gender. In the projection engine, mortality

rates are derived from the Gompertz-Makeham model and serve as hazard rates driving transitions from the Active state to the Death Benefit state in the Markov Graph.

N

NINS (Non-Life Insurance)

A proposed ACTUS-I contract type for non-life (property and casualty) insurance.

Notional Principal

The face value of a financial contract on which interest calculations are based. For a loan of 1,000,000 at 5% annual interest, the notional principal is 1,000,000.

ns/c/s (Nanoseconds per Contract per Scenario)

A normalised benchmark metric defined as total elapsed time (in nanoseconds) divided by the product of the number of contracts and the number of scenarios: `ns/c/s = elapsed_ns / (contracts × scenarios)`. This metric removes the confounding effect of portfolio size, enabling fair comparison of computational efficiency across batch sizes. A lower ns/c/s value indicates better efficiency. Used throughout the benchmark analysis to compare CPU and GPU execution paths.

O

OpenCL

Open Computing Language. An open standard for cross-platform parallel computing on CPUs, GPUs, and other processors. OpenCL provides broader hardware compatibility than CUDA but typically achieves lower performance on NVIDIA GPUs.

P

PAM (Principal at Maturity)

The most fundamental ACTUS contract type. A PAM contract involves the exchange of a principal at the start (IED), periodic interest payments (IP), and the return of the principal at maturity (MD). Examples include fixed-term deposits, bullet loans, and zero-coupon bonds.

Payoff

The cash flow amount produced by a single contract event. Each event type has a specific formula for computing its payoff based on the current contract state and terms.

PayoffCube

A multi-dimensional flat array used to store Monte Carlo valuation results, indexed by [Kind × Contract × Scenario × TimeStep]. The GPU kernel accumulates payoffs into this cube using atomic operations, enabling efficient per-contract, per-scenario aggregation.

PCIe (Peripheral Component Interconnect Express)

The high-speed hardware bus that connects a GPU to the CPU and system memory. All data transferred between CPU memory and GPU memory — both H2D (Host-to-Device) and D2H (Device-to-Host) — travels over the PCIe bus. PCIe bandwidth is a key bottleneck in GPU-accelerated pipelines, which motivates the sink architecture: aggregating results on the device before transfer minimises the volume of data crossing the PCIe bus.

Portfolio Runner

A high-level component that orchestrates the evaluation of many financial contracts as a batch. The portfolio runner manages data preparation, execution (CPU or GPU), and result collection.

Present Value (PV)

The current worth of a future cash flow, obtained by multiplying the cash flow by its discount factor. PV = CashFlow * DF(t). The total present value of a contract is the sum of the present values of all its future cash flows.

R

Risk Factor

An external variable (such as an interest rate or exchange rate) that influences the cash flows of financial contracts. In ACTUS, risk factors are provided via a Risk Factor Model that supplies observed or simulated values at specific points in time.

Risk Factor Model

A data structure that stores market data (interest rates, indices, FX rates) indexed by identifier and time. When a contract event needs a market rate (e.g., a rate reset), it queries the Risk Factor Model.

Role Sign

A value of +1 or -1 determined by the contract role. It controls the direction of cash flows. For RPA (receive principal), RoleSign = +1; for RPL (pay principal), RoleSign = -1.

Rule Pack

A package of DSL rules that define the behaviour of an insurance product. A rule pack includes product metadata, calculation rules (premiums, benefits, loadings), lookup table definitions, and the Markov graph definition. New products can be added by creating new rule packs without modifying the engine.

S

Scenario

A single simulated path of future market variables (e.g., interest rates) over a defined time horizon. In Monte Carlo simulation, many scenarios are generated to capture the range of possible outcomes.

SIMT (Single Instruction Multiple Threads)

The execution model used by GPUs, in which a single instruction is broadcast to and executed simultaneously by many threads, each operating on a different data element. SIMT is the hardware basis for GPU parallelism: thousands of threads run the same kernel code but each processes a different contract or contract-scenario pair. Workloads where all threads follow identical code paths (minimal conditional branching) make the most efficient use of SIMT execution. Life insurance Markov projections benefit especially from SIMT because every policy step applies the same arithmetic to different actuarial table values.

SoA (Structure of Arrays)

A data layout where each field of a data type is stored in its own contiguous array. For example, instead of storing [Contract1{principal, rate}, Contract2{principal, rate}], SoA stores [principal1, principal2] and [rate1, rate2] separately. This layout enables efficient GPU memory access (coalesced reads) and SIMD processing. Compare with AoS.

Solvency II

The European Union regulatory framework for insurance companies, requiring risk-based capital calculations and forward-looking risk assessments.

State Machine

A computational model where a system exists in one of a finite number of states and transitions between states based on events and conditions. In ACTUS, each contract is modeled as a state machine: events trigger state transitions and produce cash flows according to deterministic rules.

State Space

The collection of all state variables that define a contract's condition at a point in time. For PAM contracts, the state space includes: Notional Principal, Nominal Interest Rate, Accrued Interest, Fee Accrued, Notional Scaling Multiplier, Interest Scaling Multiplier, Status Date, and Contract Performance.

V

VaR (Value at Risk)

A risk metric that estimates the loss threshold at a given confidence level over a defined time horizon. VaR99 represents the 99th percentile of the loss distribution — losses will not exceed this level in 99% of scenarios, and will exceed it in only 1% of scenarios. VaR identifies the boundary of the loss tail but does not measure the severity of losses beyond it; that is the role of Expected Shortfall (ES).

Vasicek Model

A mathematical model for simulating interest rate paths. The Vasicek model is mean-reverting: rates tend to drift toward a long-term average. The formula is: dr = kappa * (theta - r) * dt + sigma * sqrt(dt) * Z, where kappa is the speed of mean reversion, theta is the long-term mean rate, sigma is the volatility, and Z is a standard normal random variable.

X

XorShift64

A fast pseudorandom number generator (PRNG) algorithm. XorShift64 uses bitwise XOR and shift operations to produce random numbers with good statistical properties and minimal computational overhead. In this project, XorShift64 is seeded deterministically to ensure reproducible results.

Y

Year Fraction

The proportion of a year between two dates, as calculated by a specific day count convention. Year fractions are used in interest calculations: Interest = Principal * Rate * YearFraction.